



بسم الله الرحمن الرحيم

جزوه‌ی درس: داده ساختار پیشرفته

استاد مربوطه: خانم دکتر فاطمی

تهیه کننده: محمد رحیمی

ترم دوم سال تحصیلی ۱۴۰۱-۱۴۰۰



داده ساختار پیشرفته

سخنی با خواننده:

با سلام و احترام خدمت تمام عزیزانی که این جزوه را مطالعه می‌فرمایند. ضمن تشکر و قدردانی از سرکار خانم دکتر فاطمی که زحمت تدریس درس داده ساختار پیشرفته را به عهده دارند و مطالب بسیار معتبر و به‌روزی که در این درس ارائه کردند این جزوه بر اساس نکات گفته در کلاس ایشان تهیه و آماده شده است. خواهشمند است در صورت مواجه با هر نوع مشکلی که ممکن است در ساختار نگارشی یا مطالب نوشته شده در جزوه وجود دارد ایمیلی ارسال و این موضوع اعلام شود تا مورد اصلاح قرار بگیرد.

این جزوه با هدف کمک به تمام دانشجویانی که درس داده ساختار پیشرفته را انتخاب کرده‌اند تهیه شده و در اختیار همه قرار می‌گیرد. هدف دیگر از تهیه‌ی جزوه، ایجاد منبع فارسی برای تمام دانشجویان دکتری است که درس داده ساختار پیشرفته را به عنوان یکی از دروس امتحان جامع خود در دانشگاه اصفهان انتخاب کرده‌اند.

این جزوه با اجازه‌ی سرکار خانم دکتر فاطمی در اختیار همه قرار می‌گیرد و استفاده از آن برای همه‌ی افراد به صورت رایگان وجود دارد و هیچ ممانعتی وجود ندارد.

با تشکر فراوان از استاد گرانقدر، سرکار خانم دکتر فاطمی

با آرزوی موفقیت برای همه‌ی خوانندگان عزیزان

آدرس ایمیل: rahimi_mohammad23@yahoo.com

تلفن همراه: 09351160913 – محمد رحیمی



مطالب

جلسه‌ی سوم.....	۴
جلسه‌ی چهارم.....	۷
جلسه‌ی پنجم.....	۹
جلسه‌ی ششم.....	۱۴
جلسه‌ی هفتم.....	۱۷
جلسه‌ی هشتم.....	۲۱
جلسه‌ی نهم.....	۲۶
جلسه‌ی دهم.....	۳۰
جلسه‌ی یازدهم.....	۳۵
جلسه‌ی دوازدهم.....	۳۹
جلسه‌ی سیزدهم: اسلاید دانشگاه استنفورد.....	۴۱
جلسه‌ی چهاردهم: مبحث درهم سازی.....	۴۵
جلسه‌ی پانزدهم.....	۵۰
جلسه‌ی شانزدهم.....	۶۱
جلسه‌ی هفدهم.....	۷۱
جلسه‌ی هجدهم.....	۷۵



جلسه سوم

داده ساختارهای زمانی، داده‌هایی هستند که سابقه‌ی تغییرات را حفظ می‌کنند. در این درس دو داده ساختار مورد بررسی قرار می‌گیرد:

داده ساختار ماندگار: داده ساختارهایی هستند که سوابق تغییرات حفظ می‌شود. نمونه‌ای آن‌ها Version control هستند.

معطوف به زمان: داده ساختارهایی هستند که اولاً سوابق تغییرات آن‌ها قابل تغییر هستند و در ثانی اگر متغیر، داده ساختار و یا رویدادهایی باشند که از این داده ساختار تاثیر می‌پذیرند آن‌ها نیز تغییر می‌کنند.

انواع داده ساختارهای ماندگار

داده ساختارهای نسبی (Partial): داده ساختارهایی هستند که تنها قادرند سوابق تغییر خود را ذخیره کند. این سوابق قابل تغییر نیستند. به این نوع داده ساختارها Branch universal نیز گفته می‌شود.

داده ساختارهای کامل (Full): داده ساختارهایی هستند که سوابق تغییرات در آن‌ها ذخیره می‌شوند و این سوابق قابل تغییر هستند. اصطلاح Parallel universal نیز به آن‌ها اطلاع می‌شود.

مدل Pointer Machin: مدلی است که از یک مجموعه گره تشکیل شده است. هر گره شامل یک اندازه‌ی محدود است و شامل داده‌ی واقعی (همان مقدار گره با فرمت‌های مختلف) و یک اشاره گر است. عملیات که بر روی یک داده ساختار انجام می‌شود شامل ایجاد، بازیابی، انتساب، عملیات داده و تخریب است و هر داده ساختار جدیدی ایجاد می‌شود باید شامل این پنج عملیات باشد.

1. x = new Node()

2. x = y.field

3. x.field = y

4. x = y + z, etc (i.e. data operations)

5. destroy(x) (if no other pointers to x)

Where x, y, z are names of nodes or fields in them.

ایجاد گره جدید
بازیابی گره
انتساب

عملیات داده
از بین بردن



داده ساختار پیشرفته

داده ساختار ماندگار: داده ساختارهایی هستند که سوابق تغییرات آن‌ها حفظ می‌شود. در واقع تمام نسخه‌های داده ساختار حفظ می‌شود. تعریف دیگر آن به این صورت است که هر داده ساختاری که توانایی پاسخگویی به کوئری‌ها در مورد حالت‌های گذشته‌ی خود را داشته باشند یک داده ساختار ماندگار است.

ماندگاری نسبی یا Impartial: آخرین نسخه از داده ساختار قابل به‌روزرسانی است. دو عملیات در آن وجود دارد.

۱- عملیات خواندن: که به صورت $\text{Read}(\text{var}, \text{version})$ است.

۲- عملیات به‌روزرسانی: که به صورت $\text{newversion} = \text{write}(\text{var}, \text{val})$ است.

یک ترتیب خطی در آن وجود دارد. با لیست پیوندی، آرایه و درخت قابل پیاده‌سازی است.

مانایی کامل یا Full: در این داده ساختار می‌توان بر روی نسخه‌های قبلی کوئری زد یعنی نسخه‌های قبلی قابل خواندن و تغییرپذیر هستند. عملیات خواندن و نوشتن به صورت زیر است.

۱- عملیات خواندن: بر روی هر نسخه‌ای قابل انجام است و به صورت $\text{read}(\text{var}, \text{version})$ نشان داده می‌شود.

۲- عملیات نوشتن: بر روی هر نسخه‌ای قابل انجام است و به صورت $\text{newversion} = \text{write}(\text{var}, \text{version}, \text{val})$ نشان داده می‌شود.

ماندگاری متلاقی یا Confluent: در اینجا علاوه آنکه نسخه‌های قبلی قابل خواندن و نوشتن هستند عملیات ترکیب نیز وجود دارد. با ترکیب کردن ناسازگاری از بین می‌رود. در اینجا DAG وجود دارد.

۱- عملیات خواندن بر روی هر نسخه‌ای قابل انجام است و به صورت $\text{read}(\text{var}, \text{version})$ نشان داده می‌شود.

۲- عملیات نوشتن بر روی هر نسخه‌ای قابل انجام است و به صورت $\text{newversion} = \text{write}(\text{var}, \text{version}, \text{val})$ نشان داده می‌شود.

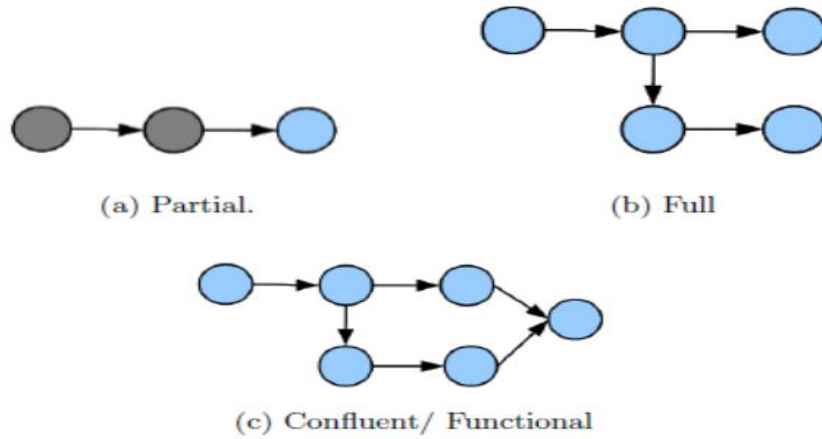
۳- عملیات ترکیب بر روی هر نسخه‌ی نرم‌افزار قابل انجام است و به صورت $\text{newversion} = \text{combine}(\text{var}, \text{val}, \text{version1}, \text{version2})$ تعریف می‌شود.

برنامه‌نویسی Functional: نوعی برنامه‌نویسی است که آبجکت‌ها تغییر ناپذیرند. یعنی اگر آبجکتی بخواهد تغییر کند یک آبجکت جدید ایجاد می‌شود و مقدار تغییر یافته در آبجکت جدید ذخیره می‌شود. در اینجا عملیات شامل ایجاد گره‌ی جدید و مقداردهی در گره‌ها داریم. ولی عملیات نوشتن نداریم. به نظر می‌رسد کاربرد زیادی ندارد. حجم حافظه مصرفی بالا است.



داده ساختار پیشرفته

نکته: تفاوتی که داده ساختار Functional با سایر داده ساختارهای مانا دارد این است که در این داده ساختارها زمانی که که قرار باشد یک نود جدید اضافه شود از همه ی نودها یک نمونه ی جدید ایجاد می شود. البته این داده ساختار از لحاظ مصرف حافظه بهینه نمی باشد ولی زمانیکه ورودی ها کم باشند می تواند مفید واقع شود.





پیاده سازی الگوریتم Partial persistence:

آیا پیاده سازی کارا برای Partial persistence وجود دارد؟ بله، وجود دارد. با استفاده مدل Pointer Machin و شرط محدودکننده از نودهای داده در زمان $O(1)$ قابل پیاده سازی است.

فرض شود یک لیست پیوندی معمولی داریم. اگر بخواهیم لیست پیوندی به لیست پیوندی Partial تبدیل شود لازم است گره ها توسعه داده شوند. برای توسعه ی گره ها دو کار اصلی انجام می شود

۱- بخشی به نام Modification اضافه می شود و تغییرات اتفاق افتاده برای هر گره در این بخش ثبت می شوند.

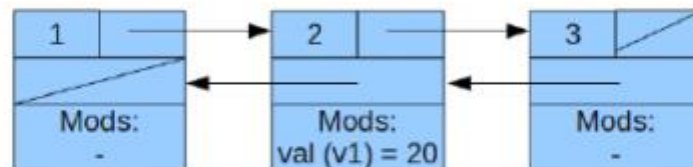
۲- همچنین باید امکان ایجاد نسخه های متفاوت از یک نود وجود دارد. با ایجاد این نسخه ها باید امکان برقراری ارتباط بین نسخه ی جاری یا Current با نسخه ی قبلی یا Old وجود داشته باشد که اینکار از طریق اضافه کردن یکسری اشاره گر انجام می شود.

نکته: بخش Modification می تواند توسط یک آرایه پیاده سازی شود و پس از آنکه مجموعه ای از تغییرات آرایه را پر کرد یک کپی جدید از نود گرفته می شود و مقادیر جدید مجددا در آن ذخیره می شود. البته باید محدودیت تعداد نسخه ها را در آن در نظر گرفت.

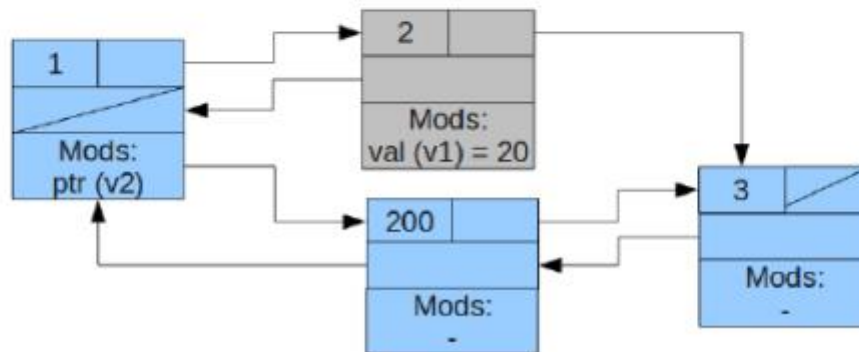
نکته: در پیاده سازی باید این محدودیت در نظر گرفته شود که از هر نود دقیقا چند نسخه می تواند ایجاد شود. در مثال زیر در بخش الف یک لینک لیست ساده وجود دارد که ریشه ی آن به گره ای با مقدار ۲ اشاره می کند. برای تبدیل کردن این لینک لیست به Partial persistence لازم است مقدار گره ها تقویت شود. بنابراین بخش modification به این بخش اضافه می شود. در بخش b مشاهده می شود که مقدار گره ی دوم از ۲ به ۲۰ اصلاح می شود. این تغییرات در بخش c قابل مشاهده است. دقت شود که عملیات نوشتن تنها بر روی آخرین نسخه ایجاد شده از هر گره قابل اعمال است.



(a) Ephemeral linked structure. It has one data field and one pointer field



(b) The structure in 1.2a partially persistent. Showing one update: `write(root.ptr.val = 20)`



(c) The structure in 1.2b after a second update: `write(root.ptr.val = 200)`. Gray indicates the data node is read only

پیاده سازی الگوریتم **Full persistence**: در حالت Full باید امکان تغییر در تمام نسخه ها وجود داشته باشد. در Full به ازای هر گره می توان یک سابقه درختی ایجاد کرد. مسئله ای که وجود دارد این است که ورژن های جدید در کجای ساختار قرار می گیرند.

باید درخت به نحوی پیمایش شود تا بتوان مشخص کرد زمانیکه یک نسخه ی جدید اضافه می شود قبل و بعد از کدام نسخه ها قرار می گیرد. درخت یک نمایش کارا برای پیاده سازی Full persistency است و فضا و زمان خطی برای ذخیره سازی نیاز است. در اینجا درخت باید به صورت Post order پیمایش انجام شود. با انجام این پیمایش محل یک نسخه ی جدید در درخت می تواند مشخص شود. در پیمایش درخت `ba` و `ea` به معنای `begin a` (اولین ملاقات) و `end a` (دومین ملاقات) است.



جلسه پنجم

ماندگاری کامل از تعمیم ماندگاری نسبی ایجاد می‌شود. فرض بر این است که از مدل ساده‌ی ماشین پوینتر تبعیت می‌شود. همچنین تعداد محدودی از اشاره‌گرها وجود دارد. در پیاده‌سازی ماندگاری کامل^۱ دو بحث مطرح می‌شود.

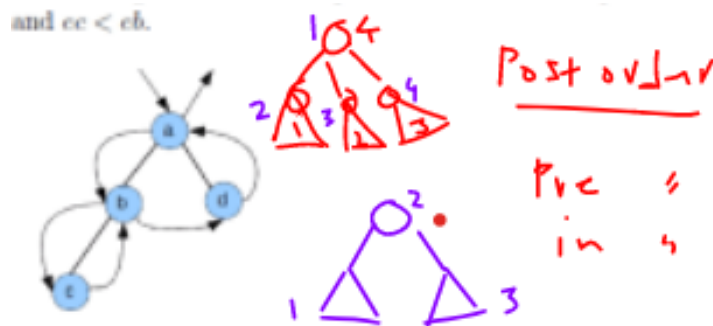
۱- در مسئله اول تمام ورژن‌ها باید به راحتی نمایش داده شوند. به این معنی که ترتیب بین ورژن‌ها به راحتی قابل نمایش باشد و دقیقاً مشخص شود کدام نسخه قبل از نسخه‌ی دیگر قرار می‌گیرد. درج یک نسخه‌ی جدید در زمان کارا باید انجام شود. پیمایش درخت `version-tree` از مرتبه‌ی تعداد ورژن‌هاست.

۲- همچنین می‌خواهیم امکان نوشتن برای تمام ورژن‌ها ایجاد شود. در اینجا ورژن `old` و `current` وجود ندارد. در اینجا بین تمام ورژن‌ها یک ترتیب ایجاد وجود دارد و لازم است نوشتن و خواندن توسط تمام ورژن‌ها حمایت شود.

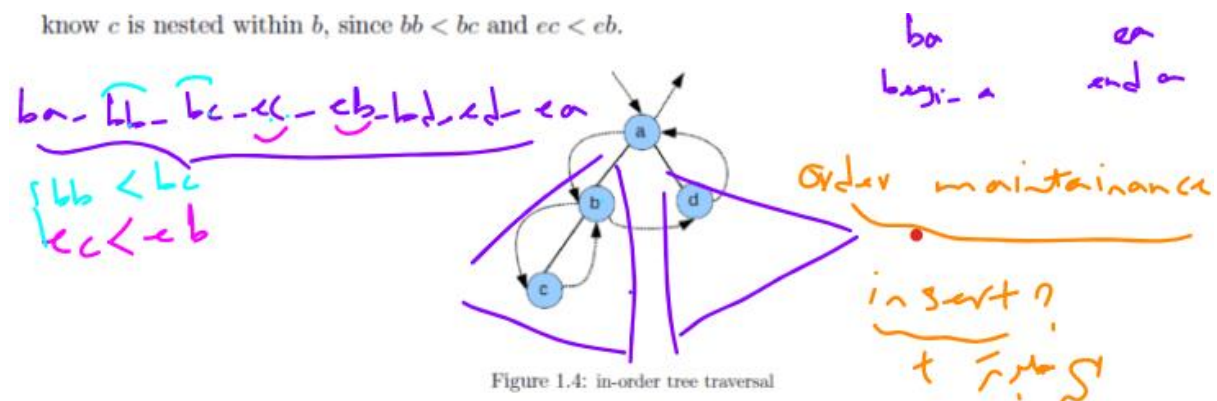
بنابراین در اینجا می‌خواهیم امکانی ایجاد کنیم که امکان نوشتن و ذخیره‌سازی بر روی درخت ورژن‌ها ایجاد شود.

مسئله نمایش ورژن‌ها با نگهداری یک درخت حل می‌شود. نحوه‌ی نمایش درخت باید به صورت خطی باشد. نوع پیمایش درخت به صورت `Postorder` است. به طور کلی سه نوع پیمایش معروف وجود دارد. که دو نوع آن مربوط به درخت عمومی و یک نوع آن مربوط به درخت باینری است. در پیمایش `postorder` در درختان عمومی به اینصورت است که باید یک ترتیب در نظر گرفته شود. در اینجا ترتیب از چپ به راست است. طبق رابطه‌ی ترتیب، ابتدا فرزندان و سپس ریشه پیمایش می‌شود. در پیمایش `postorder` فرزند چپ، فرزند راست و سپس ریشه پیمایش می‌شود. پیمایش `Preorder` به اینصورت است که ابتدا ریشه، فرزند چپ و بعد از آن فرزند راست پیمایش می‌شود و این منطق در زیردرخت‌ها ادامه می‌یابد. این منطق برای درختان عمومی قابل تعریف است. پیمایش `Inorder` برای درختان باینری اعمال می‌شود. نحوه‌ی پیمایش به اینصورت است که ابتدا فرزند چپ، سپس ریشه و در انتها فرزند راست مورد پیمایش قرار می‌گیرد. این پیمایش برای درختان ورژن مناسب نیست. زیرا درختان ورژن ممکن است باینری نباشند.

¹ Full persistence



برای حل مسئله‌ی ورژن‌ها از درختی استفاده می‌شود که ساختار نسخه‌بندی را در خود حفظ می‌کند. سپس این درخت به صورت پیمایش Postorder ذخیره می‌شود و به اینصورت نمایش داده می‌شود. در این درخت، زمانی که یک گره اولین بار ملاقات می‌شود با ba که مخفف $begin a$ است نمایش داده می‌شود و وقتی که کار با آن گره تمام شد با ea که مخفف $end a$ است نمایش داده می‌شود. این درخت به صورت postorder پیمایش می‌شود. در درخت زیر، می‌خواهیم پیمایش postorder انجام دهیم. ابتدا ریشه‌ی درخت مورد توجه قرار می‌گیرد که ba می‌شود. سپس زیردرخت چپ و بعد از آن زیر درخت راست پیمایش می‌شود. برای زیردرخت چپ، ابتدا ریشه و سپس فرزند چپ و بعد از آن فرزند راست پیمایش می‌شود که در نهایت پیمایش به صورت زیر است:



$ba, bb, bc, ec, eb, bd, ed, ea$

در نهایت حفظ ورژن‌ها با ترتیب بالا حفظ می‌شود. مزیت این روش این است که می‌توان به عنوان مثال تشخیص داد که c در دل b قرار دارد که این موضوع با توجه به روابط زیر مشخص می‌شود:

$ec < eb$

$bb < bc$

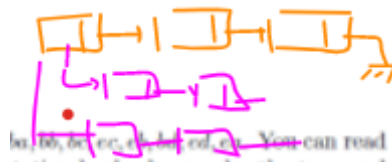
در اینجا یک ترتیب پیمایش حفظ می‌شود. از این به بعد بتوانیم بر روی داده ساختار عملیات انجام دهیم.



داده ساختار پیشرفته

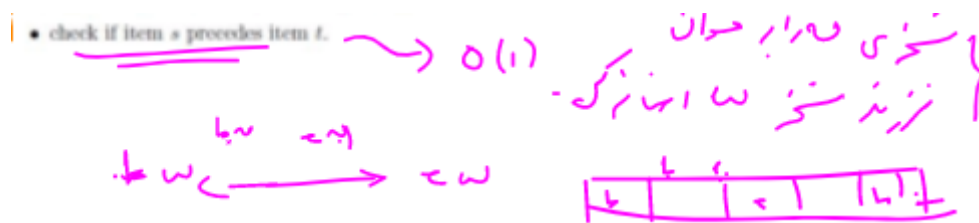
- ۱- درج کردن یک آیتم قبل یا بعد از یکسری آیتم‌ها
- ۲- بررسی کنیم که آیتم S قبل یا بعد از آیتمی مانند D است یا خیر؟

در اینجا هر کدام از نودها یک قلم داده در نظر گرفته می‌شود. فرض کنید مسئله‌ی ما یک لیست پیوندی Full persistent است. در اینجا برای داشتن نسخه‌های متفاوت از یک ورژن می‌توانیم لیست پیوندی از اعداد داشته باشیم و این لیست، نسخه‌های متفاوت را نمایش دهد.

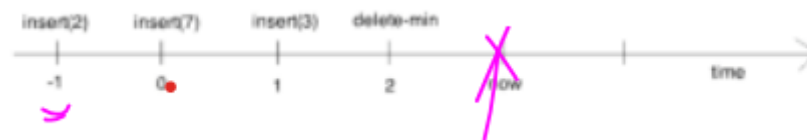


همچنین امکان پیاده‌سازی به صورت BST نیز وجود دارد که پیاده‌سازی آن متفاوت است. در واقع مفاهیم صف اولویت متفاوت است. در هر کدام از این موارد با یکسری مقایسه می‌خواهیم یکسری ورژن tree مانند $bb < bc$ و $ec < eb$ را پیدا کنیم. می‌خواهیم ببینیم ورژن b فرزند ورژن c است یا خیر؟ یا ورژن c قبل از ورژن b درج شده است یا خیر؟ برای یافتن پاسخ بالا کفایت مقایسه انجام دهیم. بنابراین پرس‌وجو در مرتبه‌ی $O(1)$ قابل انجام است.

فرض شود قرار است نسخه‌ی b بعد از نسخه‌ی w درج شود. برای اینکار bw و ew مورد جستجو قرار می‌گیرد سپس ورژن b بین ew و bw قرار می‌گیرد. عملیات به‌روزرسانی تنها با دو مقایسه و از مرتبه‌ی $O(1)$ قابل انجام است. در اینجا فرض می‌شود مکان‌های ew و bw از قبل یافت شده است و یا یافتن این دو عنصر از مرتبه‌ی بسیار پایین است.



داده ساختارهای زمانی: ایده‌ی اصلی این است که وقتی تغییری در گذشته رخ می‌دهد این تغییر کاملاً پذیرفته شود و اثر آن تغییر تا زمان حال دیده شود. در واقع یک تایم‌لاین از یکسری آپدیت‌ها و کوئری‌ها داشته باشیم. در واقع می‌خواهیم تایم‌لاینی از یکسری آپدیت‌ها و کوئری‌ها داشته باشیم. به طوریکه در هر زمان ببینیم چه اتفاقی افتاده است. در تایم‌لاین زیر در زمان 1- درج عدد 2 صورت گرفته است. سپس درج عدد 7 در زمان صفر اتفاق افتاده است و ...



به فرض آنکه شروع عملیات از زمان صفر اتفاق بیفتد. یعنی عدد 7 در زمان صفر، سپس عدد 3 در زمان 1 درج شود. اگر بخواهیم در این زمان عملیات delete-min انجام دهیم عدد 3 از صف اولویت حذف می شود. حال اگر در زمان 1- عدد 2 درج شود و بخواهیم همان عملیات delete-min را مجدداً انجام بدهیم باید عدد 2 از صف اولویت حذف شود. در واقع درج عدد 2 در زمان 1- باعث تاثیر بر روی عملیات delete-min شده است (بر روی داده ساختار تاثیر گذاشته است). در اینجا می خواهیم عملیاتی در گذشته حذف یا درج شود و تاثیر آن تا زمان حال بررسی شود.

در ساختمان داده های سنتی بر روی یک قلم داده یا مجموعه ای از داده ها رشته هایی از به روزرسانی داشتیم. در واقع داده ساختار با هر تغییر، تغییر می کرد و آپدیت های قبلی را از بین می برد. حال ما می خواهیم یک تایم لاین داشته باشیم و عملیاتی که انجام می شود عملیات ساده ی روی داده ساختار نباشد. یعنی می خواهیم عملیات جدیدی اضافه کنیم. در داده ساختار زمان دار، درج ما یک عمل به روزرسانی است. بنابراین در داده ساختار زمان دار سه عمل جدید داریم:

- Insert (t, update): درج عملیات / به روزرسانی در زمان t
- Delete (t): حذف عملیات در زمان t
- Query (t, query): کوئری گرفتن از داده ساختار در زمان t

ایده ی معطوف به گذشته بودن یا زمان دار بودن این امکان را می دهد که بتوانیم علاوه بر داده، عملیات نیز اضافه شود. سه نوع داده ساختار معطوف به زمان پیاده سازی می شود:

۱- داده ساختار معطوف به زمان جزئی: کوئری ها در زمان حال انجام می شود. یعنی عملیات کوئری در زمان حال انجام می شود.

۲- داده ساختار معطوف به زمان کامل: تاثیرات تغییرات در هر لحظه قابل مشاهده است. در واقع کوئری در هر زمان قابل انجام است.

۳- در حالت سوم هر کاری (درج، حذف و کوئری) در هر زمانی قابل انجام است. (حذف، درج و کوئری)

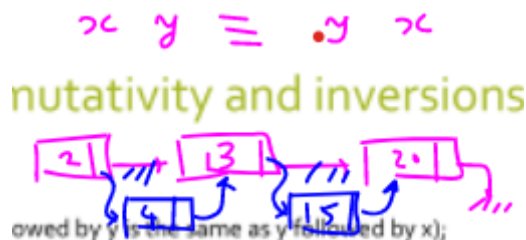
دو فرض اولیه: در اینجا دو فرض اولیه در نظر گرفته می شود. این دو فرض ممکن است همیشه اتفاق نیفتند. یعنی با توجه به داده ساختار و داده ها ممکن است اصلاً اتفاق نیفتد.

فرض اول: در اینجا عملیات در طول زمان تغییر می کند. برخی از عملیات خاصیت جابه جایی دارند. فرض کنید یک لیست پیوندی مرتب شده داریم. می خواهیم عملیات درجی انجام دهیم. به طوریکه ابتدا $x=4$ و



داده ساختار پیشرفته

سپس $y=15$ به داده ساختار اضافه می شود. اگر عملیات درج x و y به صورت y و x جابه جا شود حاصل تغییری نمی کند. در واقع تغییر ترتیب عملیات تغییری در حاصل ایجاد نمی کند. عملیات در اینجا خاصیت جابه جایی دارند. در مجموع عملیاتی که در زمان $t=-1$ انجام شده است می تواند در زمان حال انجام شود.



فرض دوم: زمانی وجود دارد که یکسری عملیات داریم که وارون پذیر هستند. فرض کنید در محور زمان، عملی در زمان t انجام شده است و این عمل قابل حذف باشد. باید بتوانیم وارون این عمل در زمان حال را انجام بدهیم. وارون عمل حذف در زمان t برابر با درج یک عنصر در زمان حال است. به طور مثال لیست پیوندی وجود دارد. در زمان $t=1$ عمل درجی انجام شود. وارون این عمل، عملیات حذف در زمان حال است. یعنی در زمان حال، عملیات $\text{delete}(1)$ انجام شود.

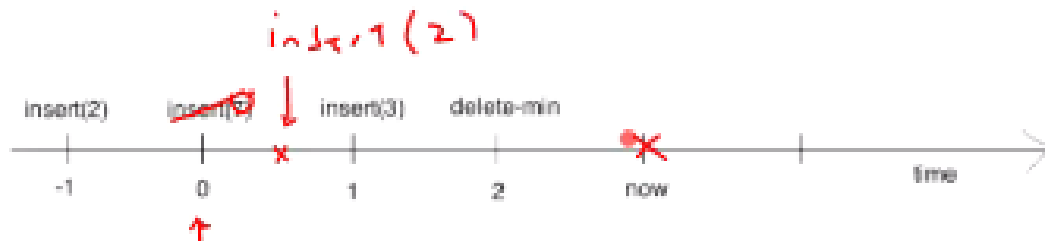
$$x, x^{-1} = () \Rightarrow \text{Delete}(t, \text{op}) = \text{Insert}(\text{now}, \text{op}^{-1})$$

جابه جایی پذیری و وارون پذیری نشان می دهد که برای **retroactive** شدن داده ساختار کافی است در زمان حال دستور مناسب به داده ساختار اضافه شود.

این پیاده سازی به درد امتحان می خورد. اگر بخواهیم یک داده ساختار معطوف به زمان طراحی کنیم کافی است این داده ساختار دو خاصیت جابه جایی پذیری و وارون پذیری داشته باشد.



داده ساختارهای معطوف به زمان: در داده ساختارهای معطوف به زمان، هدف اضافه یا حذف کردن عملیاتی است که تا این لحظه اتفاق افتاده است.



در این داده ساختار سه عملیات $\text{Insert}(t, \text{update})$ و $\text{Delete}(t)$ و $\text{Query}(t, \text{query})$ وجود دارد. عملیات $\text{Query}(t, \text{query})$ گزارشی از وضعیت ساختمان داده در زمان t را به ما نشان می دهد. $\text{Insert}(t, \text{update})$ عملیات درج در زمان t را نشان می دهد. $\text{Delete}(t)$ نیز به طور مشابه عملیات حذف در زمان t را نشان می دهد.

سه نوع داده ساختار معطوف به زمان وجود دارد.

داده ساختار Partial: در نوع اول اگر داده ساختار دارای ویژگی های بازگشت پذیر و معکوس پذیر باشد یک عملیات در زمان گذشته می تواند به زمان حال منتقل شود و اجرا گردد بدون آنکه تغییری در داده ساختار ایجاد شود به عبارت دیگر اگر عمل کوئری گرفتن انجام شود داده ساختار هیچ تغییری ندارد.

Invertible updates: There exists an operation x^{-1} , such that

$$x, x^{-1} = () \Rightarrow \underline{\text{Delete}}(t, \text{op}) = \underline{\text{Insert}}(\text{now}, \text{op}^{-1})$$

داده ساختار Full: در اینجا مسئله ی جستجو در یک داده ساختار مورد بررسی قرار می گیرد. هدف از ایجاد داده ساختارها ذخیره و دستکاری داده هاست. دستکاری می تواند بر روی همه ی داده ها انجام می شود و یا برخی از داده ها انجام شود (مسئله ی جستجو). در مسئله ی جستجو، یک مجموعه ای از اشیا طوری نگهداری می شود که بتوان عملیات Insert ، Delete و Query انجام شود. مسائل جستجوی تفکیک پذیر همان مسئله جستجو است با این تفاوت که اگر کوئری بخواهد اجرا شود باید شرط زیر اجرا شود:

$$\text{query}(x, A \cup B) = f(\text{query}(x, A), \text{query}(x, B))$$

اگر کوئری به طور مجزا بر روی دو مجموعه ی A و B انجام بپذیرد نباید تفاوت با حالتی داشته باشد که کوئری به طور مجزا بر روی A و B رخ دهد.



داده ساختار پیشرفته

به عنوان مثال می توان به مثال تقسیم و غلبه اشاره کرد که از این سیاست پیروی کرد. یعنی هنگام شکستن مسئله به زیر مسئله ها باعث می شود یک بخشی از فضای جستجو کنار برود. اگر در تعریف بالا تابع f تابع سبکی باشد آنگاه می توان با کارایی بالاتر و مرتبه ی پایین تر عملیات ترکیب را به راحتی انجام داد.

مسئله ی نزدیکترین همسایه: فرض کنیم فضایی وجود دارد و در این فضا می خواهیم از بین یک مجموعه همسایه، نزدیک ترین همسایه به یک نقطه را پیدا کنیم. کافیست مسئله به دو بخش تقسیم شود در هر بخش نزدیکترین همسایه به نقطه ی مورد نظر پیدا شود و در نهایت فاصله ی این دو همسایه تا نقطه ی اصلی مورد مقایسه قرار گیرد و نزدیکترین همسایه انتخاب می شود.



ادعاها در مورد داده ساختار معطوف به زمان:

Full retroactivity: مربوط به زمانی است که کوئری در هر زمانی می تواند انجام بشود.

ادعای اول:

- اگر مسئله جستجو تفکیک پذیر داشتیم و عملیات جابه جایی پذیر و معکوس پذیر باشند می توان در زمان $O(\log m)$ که m تعداد عملیات است داده ساختار **Full retroactivity** داشته باشیم. این ادعا توسط ساختمان داده های آرایه، لیست و ... قابل پیاده سازی است. این ادعا با ساختمان داده ی **Segment tree** اثبات شده است. این درخت برای ذخیره کردن یکسری بازه استفاده می شود. علت آن این است که یک قلم داده می تواند طول عمر داشته باشد. یعنی ابتدا عملیات **Insert** و سپس عملیات **delete min** انجام می شود. بنابراین یک بازه ی زمانی برای یک قلم داده وجود دارد. این درخت یک نوع درخت **BST** است که راس، دو فرزند دارد و هر راس یک بازه را نشان می دهد. در این درخت بازه ی هر فرزند زیر مجموعه ای از بازه ی پدر است. بنابراین یک درخت بازه بر حسب زمان ساخته می شود. در این درخت برگ ها زمان را نشان می دهند هر المان در یک بازه ی زمانی زندگی می کند. بسته به عملیات انجام شده بر روی درخت، هر کدام از عملیات در $\log(m)$ انجام می شود. زمانی که می خواهیم بر روی درخت کوئری بزنیم باید بدانیم در یک بازه ی زمانی خاص چه عملیاتی انجام شده است. چون پرس و جو تفکیک پذیر است می توان در زمان $\log m \times \text{Node}$ جستجو کرد و نتیجه را ترکیب کرد.



- **حالت عمومی Retriactivity:** اگر شرطهای مسئله‌ی قبل وجود نداشته باشد از روش Rollback استفاده می‌شود. رول‌بک روش برگشت به عقب و تکرار است. فرض می‌شود یک Timeline وجود دارد که در هر زمان یک عملیاتی را بر روی ساختمان داده انجام داده است. حال اگر بخواهیم عملیات جدیدی را اضافه کنیم و هر موقع بخواهیم یک Query بگیریم به طوریکه آن کوئری اتفاقات قبل را نشان بدهد اینکار را می‌کنیم که از محل تغییر تا محل کوئری آن دستورات را یکی یکی اجرا کنیم (یا تکرار می‌کنیم) و تغییر می‌دهیم. سربار این روش بسیار زیاد است در مرتبه‌ی $O(r)$ اجرا می‌شود.

مثال: فرض کنیم ساختمان داده‌ی ما دو مقدار X و Y را دارد که در ابتدا صفر هستند. یک ساختمان داده وجود دارد که دو مقدار X و Y در آن وجود دارد. و یکسری عملیات خاصی وجود دارد که پشتیبان این داده ساختار هستند. این عملیات شامل انتشار، جمع، ضرب، کوئری هستند. در این ساختمان داده با استفاده از این عملیات می‌خواهیم مقدار چند جمله‌ای زیر را محاسبه کنیم.

$$X = x, Y = \Delta, Y = X \cdot Y, \text{query } 'Y?' \leftarrow Y \cdot X$$

$$Y = a_n \cdot X^n + a_{n-1} \cdot X^{n-1} + \dots + a_0$$

$$Y_+ = a_n, Y = X \cdot Y, Y_+ = a_{n-1}, Y = X \cdot Y_+, \dots + a_0$$

This slide left blank for whiteboard

در مثال بالا همانطور که در شکل نشان داده شده است ابتدا گام به گام معادله ساخته می‌شود. هر گام در تایم‌لاین به طور مجزا نمایش داده می‌شود. سپس در انتهای تایم‌لاین اگر یک Query خورده شود مقدار X با توجه به زمان مقداردهی می‌شود و مقدار کوئری حل می‌شود.

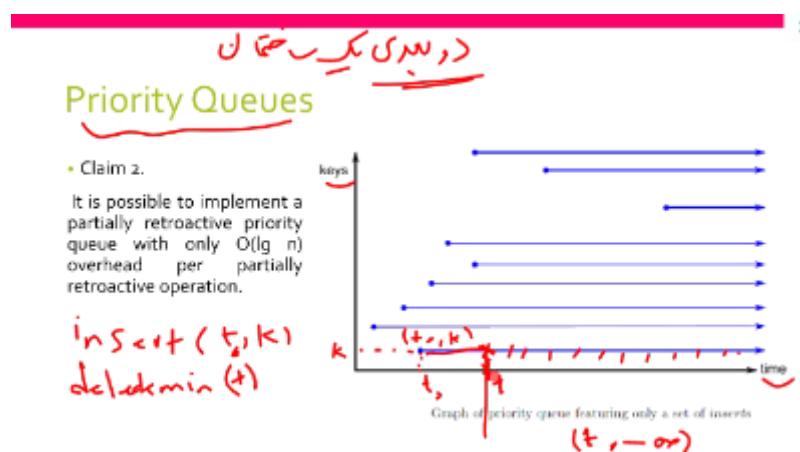


جلسه هفتم

- ادعای دوم: در اینجا تمرکز بر روی صف اولویت است. صف اولویت به این دلیل انتخاب می‌شود که عملیات جابه‌جاپذیر نیست. در صف اولویت دو عمل Insert و Delete وجود دارد. عملیات Delete برحسب نوع صف ممکن است Delete min و Delete max باشد. بنابراین اگر در گذشته یک عملیات Insert انجام شود که مقدار درج شده کمتر از سایر مقادیر باشد آنگاه عملیات Delete min همه چیز را در این صف به هم می‌ریزد. صف اولویت به این دلیل انتخاب شده است که به راحتی قابل پیاده‌سازی است.

قضیه: می‌توان نشان داد که یک صف اولویت retroactive نسبی را می‌توان با هزینه‌ی $O(\lg n)$ پیاده‌سازی کرد. می‌خواهیم این موضوع را اثبات کنیم. برای درک بهتر این موضوع، ساختمان داده به صورت دو بعدی در نظر گرفته می‌شود تا مشکل بودن انجام عملیات در retroactive نمایش داده شود.

روش نمایش ساختمان داده‌ی دوبعدی retroactive: عملیات Delete min، کوچکترین عنصر را در ساختمان داده حذف می‌کند. در این نمایش ساختمان داده در فضای دو بعدی نمایش داده می‌شود. محور X زمان و محور Y کلیدها را نشان می‌دهد. هر گاه یک عمل $\text{Insert}(t, k)$ انجام می‌شود در جدول مختصات یک فلش افقی یا اشعه افقی با مشخصات t و k درج می‌کند. این فلش‌ها یا اشعه‌ها مدت زمان زنده بودن این کلید را نشان می‌دهد. اگر یکسری عمل درج انجام شود تعدادی از اشعه‌ها یا فلش‌ها ایجاد می‌شود. در این نمایش، اگر یک عمل delete min داشته باشیم مانند این است که یک اشعه عمودی ایجاد می‌شود به طوریکه که از منفی بی نهایت شروع می‌شود و زمانی که به اولین اشعه افقی برخورد می‌کند آن را حذف می‌کند. از زمان t_0 تا زمان برخورد (t_1) طول عمر یک کلید نامیده می‌شود.

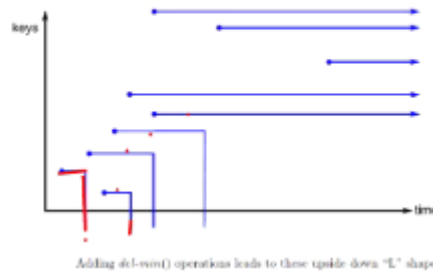


روش گفته شده استفاده می‌شود تا مفاهیم گفته شده به سادگی بیان شود. هر کجا یک L برعکس داشته باشیم حذف انجام می‌شود و اشعه‌های افقی درج را نشان می‌دهد. شکل زیر نشان می‌دهد که ۹ عمل درج و ۴ عمل حذف انجام می‌شود.

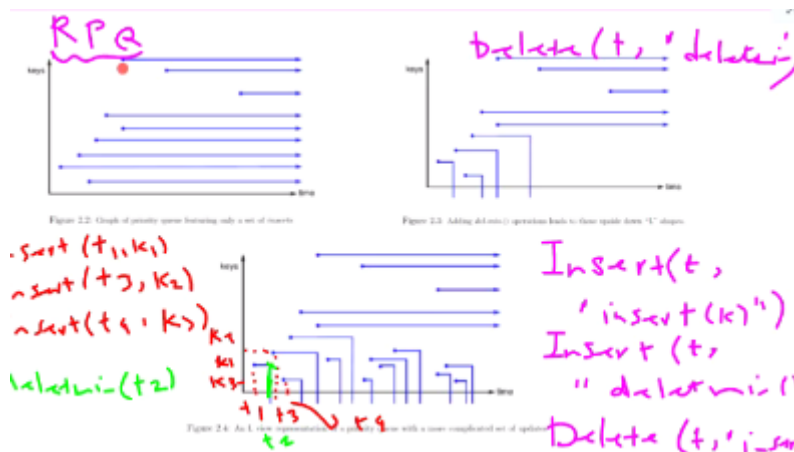


Priority Queues

Every `delete-min()` operation creates a vertical ray that starts at $(t, -\infty)$ and shoots upwards, stopping at the horizontal ray of the element it deletes. Thus, the horizontal ray becomes a line segment with end points (t, k) and (t_k, k) , where t_k is the time of key k 's deletion.



در شکل زیر عملیات زیر به ترتیب انجام می شود:



- 1- Insert (t_1, k_1)
- 2- Insert (t_3, k_2)
- 3- Insert (t_4, k_3)
- 4- Delete min (t_2, k_4)

در این مثال مقدار k_3 از k_1 کوچکتر است ولی زمان تولد آن دیرتر از k_1 است. بنابراین با انجام عملیات حذف در زمان t_2 ، k_3 حذف می شود. اگر بخواهیم عملیات ذکر شده به صورت عملیات retroactive پیاده سازی شود:

- ۱- Insert $(t, \text{insert}(k))$
- ۲- Insert $(t, \text{deletemin}(k))$
- ۳- Delete $(t, \text{insert}(k))$
- ۴- Delete $(t, \text{delete}(k))$

بنابراین صف اولویت retroactive باید دو چیز مشخص داشته باشد:

- ۵- نحوه ی طراحی داده ساختار و الگوریتم های آن بررسی شود
- ۶- چهار عمل بالا باید بررسی شود



داده ساختار پیشرفته

Delete کردن یک عمل delete یعنی $Delete(t, insert(k))$ معادل عملیات $Insert(t, insert(k))$ است. در جزو فقط یک عملیات بررسی شده است. زیرا استدلال می‌شود که اگر یک عملیات Insert پیاده‌سازی شود بقیه‌ی عملیات نیز از روی عملیات پیاده‌سازی شده قابل پیاده‌سازی است.

اگر صف اولویت Retroactive داشته باشیم با انجام یک سری عملیات retroactive چه تغییراتی اتفاق می‌افتد. در اینجا فرض کنیم یک عمل $Insert(Insert(t_0, k_0))$ داریم به طوریکه در زمان t_0 یک k_0 درج شده است. تغییری که در اینجا ایجاد می‌شود این است که عملیات delete min ، k_0 را به جای k_1 حذف می‌کند که باعث می‌شود کل عملیات به هم بریزد و نتیجه‌ی کوثری زده شده تغییر می‌کند.

سوال (تکلیف): یک صف اولویت retroactive به صورت Partial و Full پیاده‌سازی کنید.

روش پیاده‌سازی: برای پیاده‌سازی می‌توان از مفهوم پل استفاده کرد. معنی پل این است که اگر بتوانیم یک زمان t پیدا کنیم که وضعیت داده ساختار بعد از آن تغییر پیدا نکرده باشد. به این زمان یک پل گفته می‌شود. ممکن است چیز جدیدی زیاد شده باشد ولی حذف نداریم.

روش پیاده‌سازی: با استفاده از مقاله پیشنهادی می‌توان این کار را پیاده‌سازی کرد. در این مقاله از BST استفاده شده است.

داده ساختار هندسی:

داده ساختارهای معمولی معمولاً اشیا ساده‌ای از قبیل متن، عدد و ... ذخیره می‌شود. در دنیای خارج اشیا پیچیده هستند که برای ذخیره‌ی آنها داده ساختارهای هندسی پیچیده پیشنهاد می‌شود.

مثال اول: مسئله‌ی Visibility: به عنوان مثال مسئله‌ی Visibility مطرح می‌شود. در این بازی یک نفر می‌خواهد به اهدافی در دوردست تیراندازی کند. اگر مانعی جلوی فردی باشد و مانع برخورد شو جواب مسئله منفی است و اگر مانعی وجود نداشته باشد و به هدف برخورد هدف مشاهده پذیر است. این مسئله در فضاهای مسطح و پیچیده مطرح می‌شود. در دنیای واقعی فضاها پیچیده هستند. برخی از داده ساختارهای هندسی در حل این مسئله به ما کمک می‌کند.

مثال دوم: فضای وجود دارد که انواع مختلفی از اشیا داریم که ویژگی‌های هر شی از شی دیگر متمایز است. در واقع اشیا پیچیده داریم. مثلاً کوه نمی‌تواند به صورت شی منظم در نظر گرفته شود. در اینجا مسئله‌ی visisbilty بسته به موقعیت هدف و موقعیتی که در آن قرار داریم بیان می‌شود.

مثال سوم: فضاهایی است که در آن مهندسان معمار نقشه‌ای را با نرم افزار طراحی می‌کنند. ممکن است برخی فضاها وجود داشته باشد که قابل مشاهده «مانند پشت دیوار» نباشند اما با تغییر زاویه می‌توان آنها را مشاهده



داده ساختار پیشرفته

پذیر کرد. اشیائی مانند در دیوار و ... که موقعیت‌های متفاوتی ایجاد می‌کند جواب‌های مختلفی را ایجاد می‌کند.

مثال چهارم: این اسلاید پیدا کردن یک اشیا خاص در یک زیر مجموعه از فضاهاى هندسى را بیان کند. مثلاً در این عکس می‌خواهیم فیل‌ها را مشخص کنیم. این فیل‌ها می‌تواند به شکل‌های مختلف وجود داشته باشد. بنابراین لازم است این اشکال به صورت‌های مختلف مورد جستجو قرار دهیم. در اینجا مسائل مختلف مطرح می‌شود. فیل از زوایای مختلف می‌تواند اشکال مختلف داشته باشد.

مسئله‌ای که در دنیای واقعی وجود دارد باید به صورتی شبیه‌سازی شود که برای ما قابل درک باشد. مثلاً فیل به صورت نقطه و سایر اشیا به صورت زیر فضا شناخته می‌شود. در اینجا مسئله این است که نقطه‌ها در زیر فضاها پیدا شود.

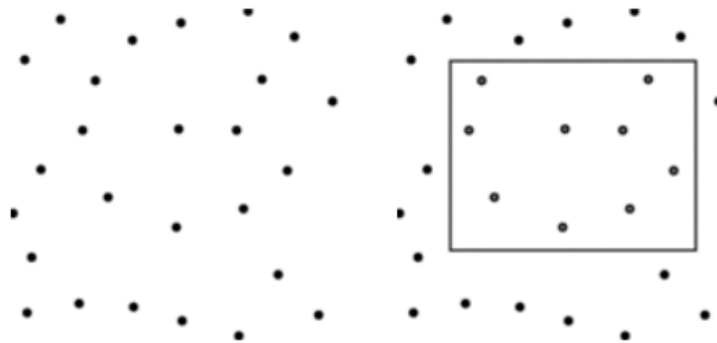


جلسه هفتم

مسئله یافتن اشیا درون یک قاب: این مسئله یافتن اشیائی را مورد نظر قرار می دهد که لزوماً مستطیلی نیستند. به عنوان مثال فرض کنید در شکل زیر اشکالی را می خواهیم پیدا کنیم که قابلیت درگ کردن داشته باشند.



هر چه مسئله پیچیده تر در نظر گرفته شود سخت تر حل می شود. مسئله می تواند به صورتی مدل سازی شود که یکسری نقطه درون یک فضای سه بعدی مورد جستجو قرار می گیرند. این مسئله Range searching نامیده می شود. این مسئله می تواند دو بعدی یا سه بعدی باشد.



مسئله یافتن تقاطع اشیا: این مسئله محل برخورد دو شی در یک محیط است. مثلاً محل پیدا کردن محل تلاقی گلوله (یا سهمی ها) به صفحه ی روبه رو بررسی می شود. اگر این مسئله به صورت پویا فکر شود یعنی اشیا بصورت پویا در حال تغییر باشند مشکل بودن آن چند برابر می شود.



مسائل مورد بررسی در این درس:

- ۱- درخت جستجوی باینری و **kd-trees**: در این نوع درخت یک رنج یک بعدی یا دو بعدی برای کوئری‌ها در نظر گرفته می‌شود.
 - ۲- درخت‌های محدوده یا **Range trees**: برای جستجوی یکسری نقاط در یک بازه مورد استفاده قرار می‌گیرد.
 - ۳- درخت بازه یا **interval trees**: برای گزارش بازه‌ها در یک محدوده مورد استفاده قرار می‌گیرد به طوریکه شامل یکسری اشیا هستند.
 - ۴- درخت قطعه: بر روی یک خط همه‌ی بازه‌هایی گزارش می‌شود که شامل یک نقطه‌ی خاص می‌شود.
- برای برخی مسائل از دو یا چند تکنیک بالا استفاده می‌شود.

الگوریتم SWEEP: برای پیدا کردن یکسری ناحیه‌ی که از هم قابل رویت هستند استفاده می‌شود.

مسئله‌ی Point location: در این دوره‌ی آموزشی آموزش داده نمی‌شود. درخت بالانس برای حل این مسائل استفاده می‌شود.

مسئله‌ی BSP trees: این درخت‌ها بالانس نیستند ولی برای حل برخی از مسائل استفاده می‌شود.

مسئله‌ی جستجوی محدوده

جستجوی محدوده در فضای یک بعدی: زمانی که فضا یک بعدی است. یک خط و یکسری نقاط بر روی خط داریم. سپس می‌خواهیم یک مجموعه نقاطی را پیدا کنیم که در بازه‌ی بین $[a, b]$ قرار دارد. نحوه‌ی حل مسئله و پیچیدگی آن چقدر است؟ یک راه این است که یک آرایه مرتب از مقادیر ایجاد کنیم. هر چیزی که بین مقادیر a و b وجود دارد مجموعه نقاط مورد نظر ما هستند. یک راه برای جستجو این است که مقادیری را پیدا می‌کنیم که از a بزرگتر و از b کوچکتر باشد. مجموعه نقاط بین a و b مجموعه نقاط مورد نظر را

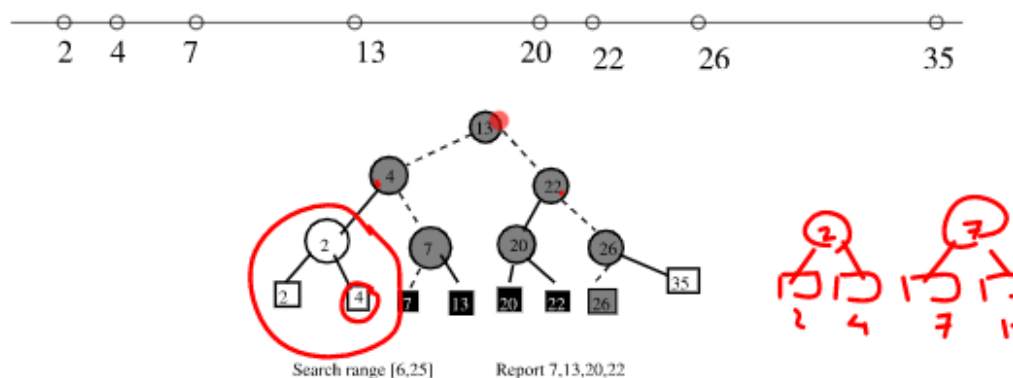


داده ساختار پیشرفته

نشان می‌دهند. در بدترین حالت از مرتبه‌ی $O(n)$ است. راه دیگر استفاده از الگوریتم جستجوی باینری است که مقادیر a و b از طریق جستجوی باینری یافت می‌شوند. مرتبه‌ی جستجو با توجه به این راه حل برابر با $\log n$ است. بنابراین مرتبه‌ی این مسئله در هنگامی که نقاط ثابت هستند برابر با $O(\log n + k)$ است که از راه حل قبلی بهتر است.

برای محل مسئله در حالت از درخت $\text{binary leaf search}$ استفاده می‌شود. نحوه‌ی ساخت درخت به این گونه است که برگ‌ها نشان دهنده‌ی نقاط و گره‌های غیربرگ حاوی بزرگترین مقدار در زیر درخت سمت چپ هستند. اگر بخواهیم محدوده‌ای مانند $[a, b]$ را پیدا کنیم ابتدا a را به صورت باینری درون درخت پیدا می‌کنیم و سپس به صورت ترتیبی گره‌ها را جلو می‌رویم تا به مقدار b برسیم. مقادیر مشاهده شده، مقادیر مورد نظر ما هستند.

پس از پیدا کردن راه حل‌های مختلف لازم است کارایی هر راه حل مورد بررسی قرار گیرد و بهترین راه حل انتخاب شود.



- We use a binary leaf search tree where leaf nodes store the points on the line, sorted by x -coordinates.

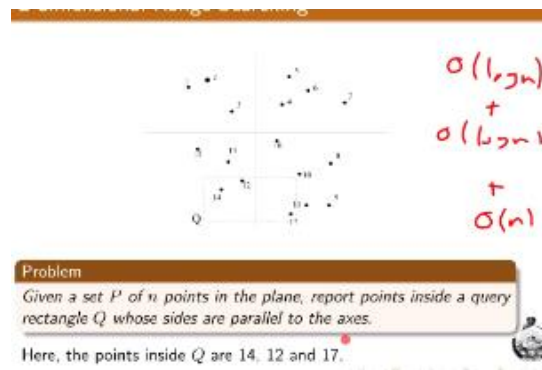
حل مسئله‌ی جستجو به صورت دو بعدی: ما یک مجموعه از نقاط در صفحه‌ای مانند p داریم. در این صفحه یک سری نقاط وجود دارند. می‌خواهیم یک سری نقاط را با استفاده از یک کوئری پیدا کنیم. برای پیدا کردن یک محدوده کافیسست کوئری به صورت یک مستطیل در نظر گرفته شود. اگر این کوئری به شکل لوزی باشد نیاز است یک سری محاسبات هندسی نیز انجام شود ولی برای راحتی کار معمولاً این کوئری به صورت مستطیل در نظر گرفته می‌شود. این مسئله را می‌توان به دو شکل بررسی کرد. در حالت اول این مسئله به دو مسئله‌ی تک بعدی تجزیه می‌شود. یک بعد افقی و یک بعد عمودی. اگر بخواهیم این مسئله به صورت مجزا حل کنیم استفاده از درخت kd-tree است. این درخت برای حل مسئله‌ی range-search استفاده می‌شود.



داده ساختار پیشرفته

نحوه‌ی عملکرد به این صورت است که ابتدا داده‌ها را بر روی مختصات x و سپس بر روی مختصات y تفکیک می‌کند. اگر بعد بیشتری داشته باشیم این مسئله به صورت ذکر شده در بعدهای بالاتر تجزیه می‌شود.

همچنین مسئله به صورت دو مسئله‌ی پرس‌وجوی تک بعدی نیز قابل حل است. یعنی یک بار بر روی x نگاشت و حل شود و بار دیگر بر روی y نگاشت و حل شود و سپس بر روی آن‌ها اشتراک گرفته می‌شود. به عنوان نمونه می‌توان از داده‌ساختاری استفاده کرد که مرتبه‌ی اجرای آن از $O(\log n)$ باشد و سپس هزینه اشتراک این دو مجموعه راه‌حل به مرتبه‌ی $O(\log n)$ اضافه می‌شود. در بدترین حالت هزینه‌ی اشتراک $O(n)$ است.



راه‌حل بهتر، استفاده از kd -tree است. این داده‌ساختار، یک داده‌ساختار $space$ -partitioning است و برای بخش‌بندی فضا استفاده می‌شود. زمانی استفاده می‌شود که کلید جستجو چند بعد دارد استفاده می‌شود. kd -tree حالت خاصی از درخت $space$ -partition-tree است. در این درخت هر گره شامل k بعد است و هر گره‌ی غیر برگ فضا را به دو بخش چپ و راست تقسیم می‌کنند. این درخت فضا را به دو بخش up و $down$ تقسیم می‌شود.

نحوه‌ی ساخت درخت: فرض کنید یک $Hyper$ plan داریم. در این $Hyper$ plan نقاط را بر حسب x میانی به دو بخش تقسیم می‌کنیم. در واقع مقادیر را بر حسب x پیدا می‌کنیم و x میانی را می‌یابیم و بر حسب آن، $hyper$ -plan را به دو بخش تقسیم می‌کنیم. بنابراین یکسری نقاط سمت چپ و یکسری سمت راست قرار می‌گیرند. سپس نقاط بر حسب y به دو بخش تقسیم می‌شوند. بنابراین نقاط به دو بخش بالا و پایین تقسیم می‌شوند. و این عملیات برای سایر ابعاد نیز ادامه می‌یابد. بنابراین زمانی که پرس‌وجویی مطرح می‌کنیم عملیات جستجو شبیه جستجو در درخت باینری می‌شود. پرس‌وجو در قالب یک مستطیل مطرح می‌شود. این مستطیل حالت‌های خاص ایجاد می‌شود. حال مستطیل یا کوئری می‌تواند به صورت جزئی با دو یا چند بخش همپوشانی دارد و یا می‌تواند در یک بخش به طور کامل وجود داشته باشد. برای هر کدام از حالت‌ها به یک شیوه عمل می‌کنیم. اگر مستطیل درون یک ناحیه قرار بگیرد تمام نقاط آن شامل جواب کوئری می‌شود. اگر به صورت جزئی با چند ناحیه همپوشانی داشته باشد باید از درخت باینری پایین بیابیم و نقاط درون این



داده ساختار پیشرفته

درخت مورد جستجو قرار بدهیم. پیچیدگی برای هر نقطه در حدود $\log$ و برای تمام نقاط به صورت $n \log n$ است.

مثال از جزوه بررسی شود

اگر بخواهیم یک kd-tree بسازیم باید عملیات زیر را داشته باشد. پیچیدگی ساخت از عملیات زیر می آید:

- ۱- ساختن
- ۲- اضافه کردن المانهای جدید
- ۳- حذف المانهای موجود
- ۴- ارتفاع بالانس: عمل بالانس سازی باید پیاده سازی شود.
- ۵- عمل جستجو یا nearest naber search است که بعدا تبدیل به rang search است.



جلسه نهم

بهترین راه فهم این داده ساختارها این است که یکسری نقاط با مشخصات (x, y) در صفحه‌ی مختصات قرار داده می‌شود و الگوریتم‌های مربوط به داده ساختارها بر روی آن‌ها اعمال می‌شود.

اگر به تحلیل kd-tree پرداخته شود پیچیدگی آن از مرتبه‌ی $O(\log n)$ است. همچنین k نود مورد بررسی به زمان اجرای ما اضافه می‌شود. یعنی این مقدار در مجموع برابر با $O(\log n + k)$ است. این مرتبه با استفاده از داده ساختارهای دیگر قابل بهبود است. به عنوان نمونه می‌توان از Range-trees یا درخت محدوده استفاده کرد. Range Tree از نوع درخت‌های مرتب است یعنی مشابه kd-trees مقادیر گره‌ها بر اساس کلیدها مرتب می‌شوند و سپس عملیات جستجو بر روی درخت مرتب شکل می‌گیرد.

با وجود آنکه Range-tree ها و kd-tree ها مسائل جستجوی بازه را حل می‌کنند تفاوت آن‌ها در این است که Range-tree ها عملیات جستجوی را در مرتبه‌ی $\log^2 n$ انجام می‌دهد در حالیکه kd-tree ها عملیات جستجو را در مرتبه‌ی $O(\log n)$ انجام می‌دهند. ولی فضای مورد استفاده آن‌ها نسبت به kd-trees بسیار بیشتر است. ایده‌ی پشت تمام این درخت‌های Binary-Search است. یعنی یک درخت باینری وجود دارد که هر نود دارای یک کلید است. تمام کلیدهای سمت چپ از ریشه کوچکتر و تمام کلیدهای سمت راست از ریشه بزرگتر هستند.

در kd-tree از افراز کردن نقاط با توجه به ایده‌ی درخت باینری استفاده می‌شود. در واقع $n/2$ نقاط سمت چپ و $n/2$ سمت راست قرار می‌گیرد. همین عملیات بر روی $n/2$ گره‌ی سمت اعمال می‌شود. به عبارتی n نقطه به چهار دسته تقسیم می‌شود و دسته‌بندی با توجه به درخت باینری اعمال می‌شود. در Range-tree نیز ایده‌ی درخت باینری بر روی نقاط پیاده‌سازی می‌شود.

فرض کنید یک مجموعه‌ی S حاوی n نقطه داریم. اگر بخواهیم ساختمان داده‌ی Range-trees را بسازیم مرتبه‌ی ساخت از $O(n \log n)$ است ولی زمان جستجو از مرتبه‌ی $O(\log^2 n)$ است که البته مقدار k نیز که نشان‌دهنده‌ی تعداد گره‌های مورد گزارش است به آن اضافه می‌شود.

درخت رنج سرچ می‌تواند بهبود یابد. در واقع این داده ساختارها بعد پژوهشی دارد. با استفاده از تکنیک frac casc که زمان جستجو را از نظر زمان بهبود می‌دهد. همچنین از Range-tree به نحوی استفاده شود که از اتلاف حافظه جلوگیری شود.

ایده‌ی مطرح شده این است که مشابه kd-trees، بر اساس مشخصه‌ی مقادیر x درخت را مرتب کنیم و یک درخت جستجوی باینری می‌سازیم. وقتی کوئری مورد پرسش قرار می‌گیرد این کوئری در یک فضای جستجو دو بعدی که شامل یکسری نقاط است اعمال می‌شود. مستطیل ساده‌ترین فرم رنج کوئری است و شامل نقاطی



داده ساختار پیشرفته

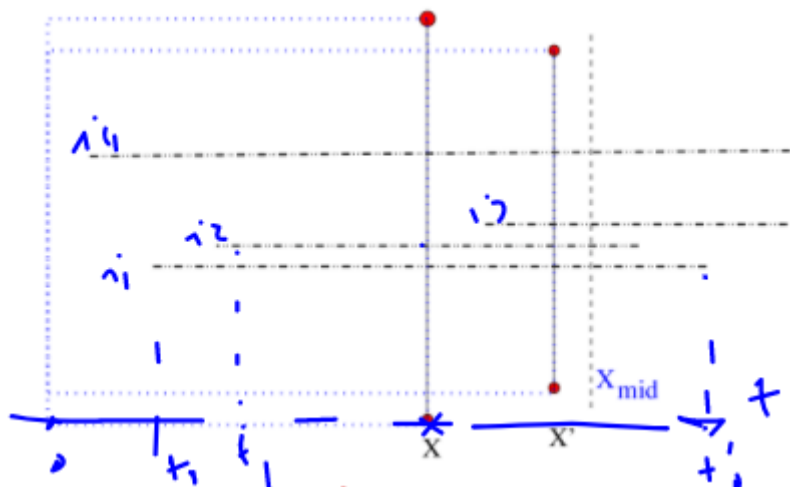
است که x آن‌ها در بازه‌ی بین $[a, b]$ و y آن‌ها بین بازه‌ی $[c, d]$ قرار دارد. ایده‌ی دیگر این است که مختصات رئوس مستطیل داده می‌شود.

ایده‌ای که در Range tree این است که درخت باینری بر حسب مقدار x ساخته می‌شود. در این درخت هر کدام از گره‌ها شامل یکسری نقاط است که این نقاط همان y ها یا بعد دوم نقاط هستند. در واقع در هر گره یک BST قرار می‌گیرد. زمانیکه می‌خواهیم جستجو انجام دهیم مقدار x یک نقطه مانند a را با مقدار x ریشه مقایسه می‌کنیم و در اینجا مشخص می‌شود که به سمت چپ برویم یا راست. بعد از مشخص شدن این موضوع، مقایسه در سطح دوم بر حسب y های مرتب شده انجام می‌شود. همچنین Range Tree می‌تواند به صورت‌های دیگر تعریف شود. ایده در اینجا این است که گره‌ها ساده نباشد. در واقع هر گره یک داده ساختار باشد که بر حسب بعد دوم مرتب شده باشد. Range-tree بیشتر برای فضاهای دو بعدی استفاده می‌شود ولی kd-trees برای k بعد مناسب است.

توصیه می‌شود ۱۰ یا ۲۰ نقطه در نظر گرفته شود و بر اساس آن kd-tree و Range tree ساخته شود.

ایده‌ی Range tree: در شکل زیر هر کدام از گره‌ها یک زیر درخت است که بر حسب مشخصات y ساخته و مرتب شده است. بر جستجوی بازه‌ای مانند $[a, b]$ کافیهست که درخت مورد جستجو قرار بگیرد تا مقدار a پیدا شود. سپس از نقطه‌ی یافت شده به جلو حرکت می‌کنیم تا به نقطه‌ی b برسیم.

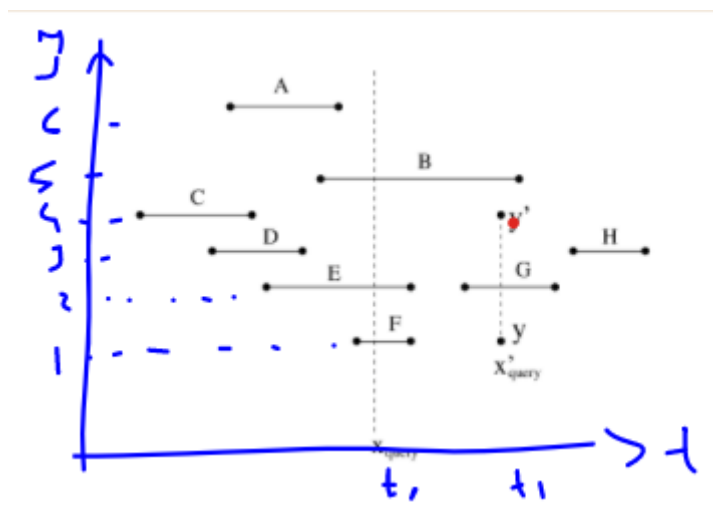
Interval Query: این داده ساختار از مسئله‌ی windowing problem آمده است. در واقع حالت خاصی از Range query نیست. در اینجا مسئله این است که یک مجموعه از بازه‌ها داریم که می‌تواند به هر چیزی از جمله زمان تعبیر شود. این بازه‌ها با i نشان داده می‌شود. و یک نقطه پرس‌وجو مانند x داریم. می‌خواهیم تمام بازه‌های را پیدا کنیم که نقطه‌ی x در آن وجود دارد. در واقع می‌خواهیم ببینیم کدام داده‌ها در زمان x زنده هستند. شبیه مسئله‌ی deletemin در صف اولویت است.





داده ساختار پیشرفته

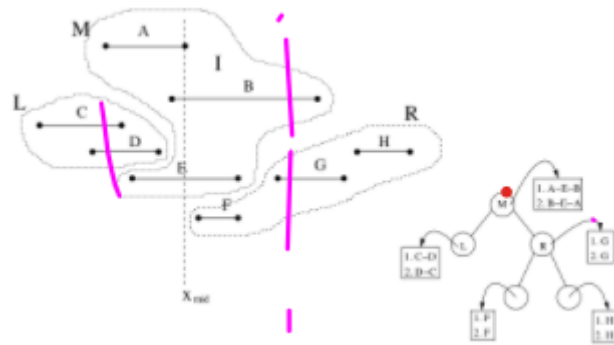
اگر مسئله‌ی بالا در دوبعد تعریف شود بازه‌هایی را می‌خواهیم که شامل x و y باشد. مثلاً y مقدار داده و x مقدار زمان است.



این مسئله می‌تواند از ایده‌ی درخت باینری و اضافه کردن هیوریستیک ایجاد شود. این موضوع می‌تواند مورد پژوهش نیز قرار بگیرد. در این مسئله یکسری بازه وجود و یک نقطه‌ی x وجود دارد. اگر این نقطه به صورت خط عمودی در نظر گرفته شود می‌خواهیم ببینیم که این x شامل کدام یک از بازه‌ها می‌شود. هر بازه با مختصات دو نقطه یک بعدی که ابتدا و انتهای بازه را نمایش می‌دهد نشان داده می‌شود. اگر درخت بازه شامل n بازه باشد در واقع $2n$ نقطه پایانی داریم. برای حل مسئله جستجو کافیست تمام نقاط ابتدایی و انتهایی بازه‌ها مرتب شوند و نقطه‌ی میانی انتخاب شود. خطی به صورت عمودی کشیده می‌شود. تمام بازه‌هایی که سمت چپ قرار می‌گیرند و با آن خط تلاقی ندارند به عنوان یک گره در نظر گرفته می‌شود. همچنین بازه‌هایی که در سمت راست قرار می‌گیرند و تلاقی با خط ندارند به عنوان گره‌ی سمت راست درخت در نظر گرفته می‌شود. ریشه درخت نیز تمام بازه‌هایی است که با خط برخورد داشته‌اند. در شکل زیر گره‌ی میانی یا ریشه‌ی درخت شامل A, B, E است. حال در هر گره دو نوع مرتب‌سازی وجود دارد. در یک مرتب‌سازی نقاط انتهایی سمت چپ هر گره و سپس نقاط انتهایی سمت راست هر گره به طور مجزا مرتب می‌شود و در نهایت Interval Query را ایجاد می‌کنند. این کار آنقدر ادامه می‌یابد تا تقسیم‌بندی امکان‌پذیر نباشد. در نهایت با منطق BST، اگر عنصری مورد جستجو قرار بگیرد از بالای درخت عملیات جستجو انجام می‌شود.



داده ساختار پیشرفته





جلسه دهم

مسئله: در اینجا یکسری بازه وجود دارد. می‌خواهیم ببینیم نقطه‌ای مانند x در کدام یک از بازه‌ها قرار دارد.

راه اول: هر بازه خط دارای یک نقطه‌ی ابتدایی و یک نقطه‌ی انتهایی است. نقاط ابتدایی در یک آرایه و نقاط انتهایی در یک آرایه ذخیره و مرتب می‌شوند و نقطه‌ی x در دو آرایه مورد جستجو قرار می‌گیرد. مرتبه‌ی پیدا کردن یک نقطه از طریق آرایه‌های مذکور برابر با $2n + \log n$ است که این زمان خوب نیست.

تحلیل حل مسئله به صورت زیر است:

راه دوم: بازه‌ها مرتب می‌شوند و تبدیل به BST می‌شود و در نهایت جستجو در BST انجام می‌شود. ابتدا $2n$ نقطه مربوط به n بازه خط مرتب سازی می‌شوند و نقطه‌ی میانی پیدا می‌شود. اگر این نقطه x در نظر گرفته شود بازه‌های به سه دسته تقسیم می‌شوند. برخی از بازه‌ها به طور کامل در سمت راست میانه و برخی از آن‌ها به طور کامل در سمت چپ میانه قرار دارند. همچنین برخی از بازه‌ها شامل نقطه‌ی میانه می‌شوند. با همین منطق یک BSE ساخته می‌شود به طوریکه بازه‌های شامل x در ریشه، بازه‌های کمتر از x در زیر درخت سمت چپ و بازه‌های بیشتر از x در زیر درخت سمت راست قرار می‌گیرند. هر گره‌ی BST شامل یک داده ساختار مشابه با آنچه گفته شد قرار دارد. در هر گره علاوه بر اطلاعات گفته شده، اطلاعات دیگری قرار دارد. به طور مثال نقاط ابتدایی و انتهایی بازه‌ها به طور مجزا و به ترتیب مرتب می‌شوند و به عنوان اطلاعات گره ذخیره می‌شوند. بر حسب مقدار کوثری سه حالت پیش می‌آید

- ۱- اگر مقدار کوثری کمتر از میانه باشد جستجو در سمت چپ انجام می‌شود.
- ۲- اگر مقدار کوثری بیشتر از میانه باشد جستجو در سمت راست انجام می‌شود.
- ۳- مرتبه‌ی جستجو از $O(\log n + k)$ است که n تعداد بازه‌ها و k تعداد بازه‌های چک شده است.

راه حل دوم: Segment tree به عنوان راه حل دیگر به دو دلیل استفاده می‌شود:

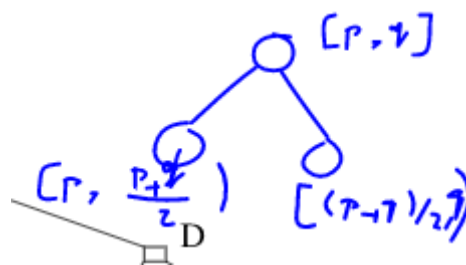
- ۱- برای جستجوی یک نقطه در یکسری بازه مناسب است.
- ۲- برای جستجو یک بازه در یکسری بازه مناسب است.

Segment trees درخت‌های جستجوی دودویی و متوازن هستند. این درخت یکسری هزینه از جمله نگهداری درخت دارد. مزیت آن این است که هزینه‌های نگهداری درخت از جمله حذف یا اضافه کردن یک گره مناسب است. بنابراین اگر بازه‌ای به این درخت اضافه یا کم شود ویژگی بالانس بودن درخت باید حفظ شود. هزینه‌ی بالانس شدن درخت به هزینه‌های قبلی درخت (هزینه‌ی پیدا کردن یک نقطه در یکسری بازه) اضافه می‌شود. Segment tree یک درخت جستجوی دودویی است که هر کدام از برگ‌ها و گره‌ها بازه‌ای را نشان می‌دهند. هر گره دو فرزند دارد. اگر گره بازه‌ی $[p, q]$ را نشان دهد فرزند چپ $[p, (p+q)/2]$ و فرزند



داده ساختار پیشرفته

راست $[(p+q)/2, q]$ را نشان می‌دهد. اگر از پایین به بالا نگاه شود گرهی سطح بالاتر حاصل اجتماع گره‌های فرزند است.

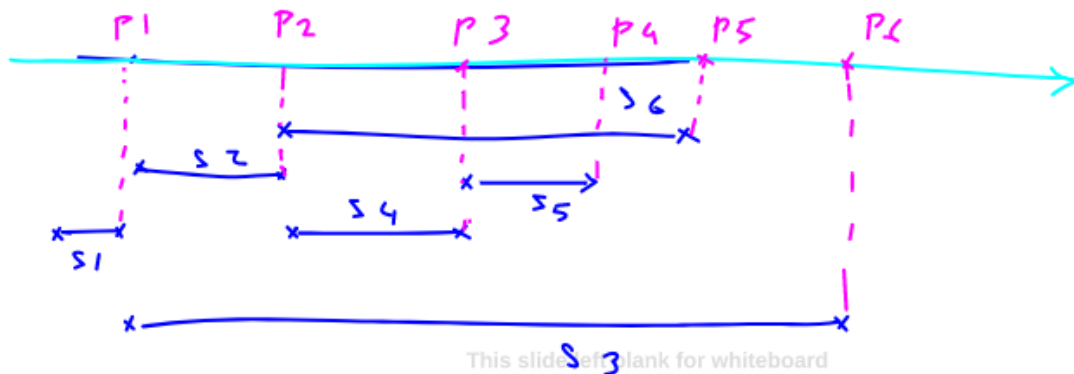


سگمنت تیری برای ذخیره کردن بازه‌ها و پیدا کردن یک نقطه در یک بازه استفاده می‌شود. سگمنت تیری شکل‌های مختلف دارد. در بعضی از اشکال سگمنت تیری به حدی خرد می‌شود که برگ‌ها بازه‌هایی به طول یک را نشان می‌دهند. به دلیل ارتفاع زیاد این درخت، نوع دیگری از سگمنت تیری پیشنهاد می‌شود که برگ‌ها بازه‌های به طول یک ندارند.

نحوه‌ی پرس‌وجو به این‌صورت است که اگر ریشه بازه‌ای به طول $[a, b]$ را نشان بدهد و x بین $[a, (a+b)/2]$ باشد به زیر درخت چپ می‌رویم و در غیر این‌صورت $[(a+b)/2, b]$ به سمت راست می‌رویم.

مثال: اگر بازه‌های زیر را داشته باشیم:

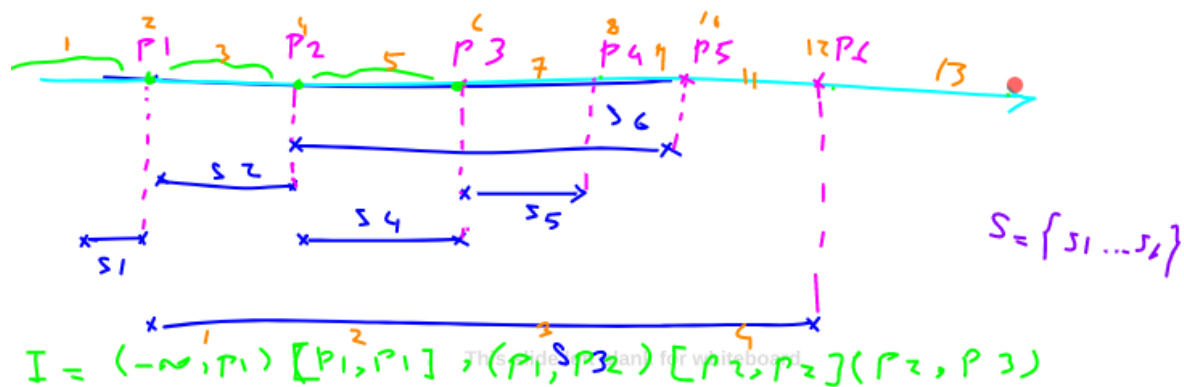
در این مثال نقاط انتهایی مشخص و مرتب‌سازی بر اساس نقاط انتهایی انجام می‌شود.



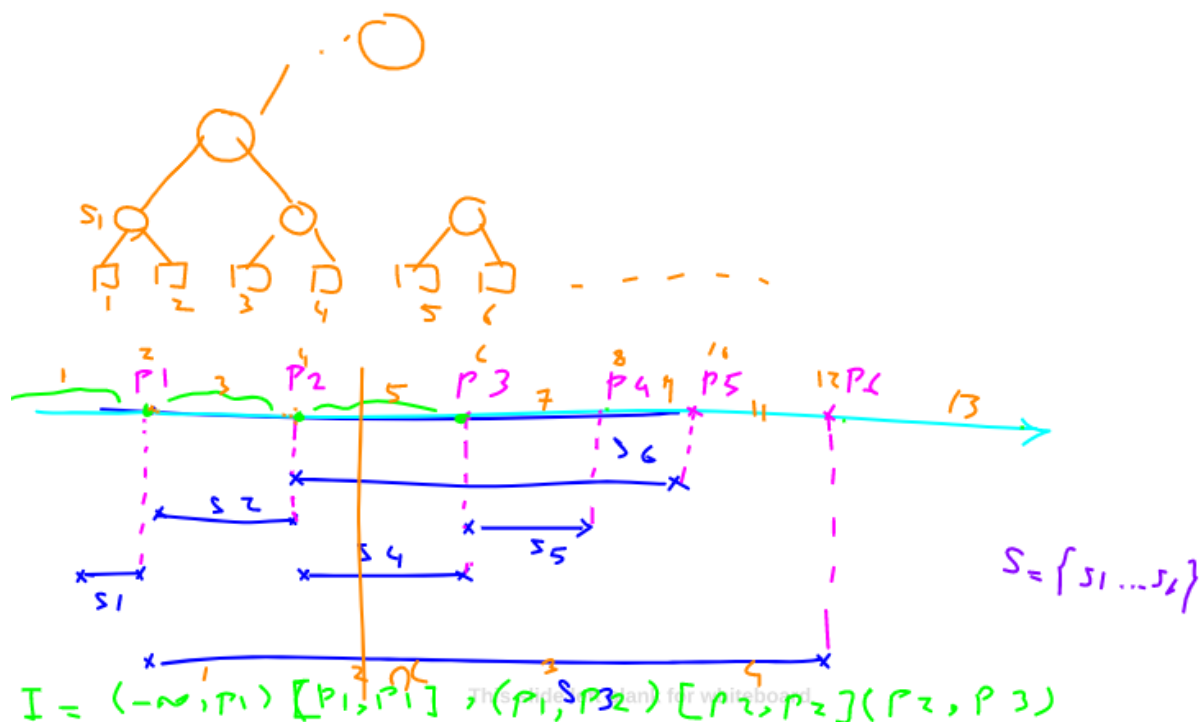
در اینجا یک پارتیشن‌بندی خطی انجام داده‌ایم که شامل نقاط انتهایی هر نقطه از بازه‌هاست. مجموعه‌ی بازه‌ها به صورت $s = \{s_1, s_2, \dots, s_6\}$ داریم. یکسری Elementary interval داریم که با مجموعه‌ی a نشان داده شده است و به صورت زیر و با رنگ سبز نشان داده شده‌اند. در اینجا با شش بازه، ۱۳ Elementary interval شناسایی شده است:



داده ساختار پیشرفته



در گام بعد درخت بازه ترسیم می‌شود که به صورت زیر است. هر Elementary Interval یک برگ را نشان می‌دهد. درخت با رنگ قرمز ترسیم شده است. رسم درخت با تجمیع بازه‌ها ادامه می‌یابد و آنقدر به سمت بالا می‌رویم تا به ریشه برسیم:



حال با توجه به درخت نشان داده شده، عملیات جستجو برای نقطه‌ای مانند a مثال زده شده است.

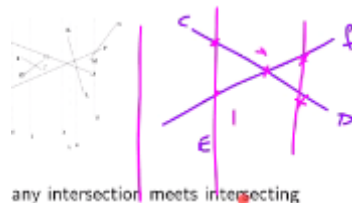
مسئله جدید: تا الان بازه‌های موجود خوش‌تعریف بودند. یعنی موازی محور مختصات هستند. در این مسئله مجموعه‌ی S از پاره خط‌های دلخواه داریم که موازی محور مختصات نیستند و می‌خواهیم ببینیم کدام یک از آن‌ها یکدیگر را قطع می‌کنند. این مسئله در **visibility** کاربرد دارد.



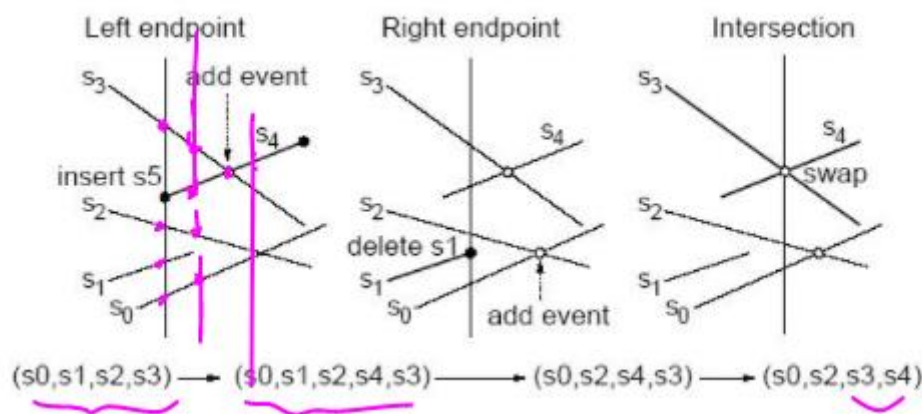
داده ساختار پیشرفته

راه اول: هر دو پاره خط به صورت دوبه دو مجزا در نظر گرفته می شوند. سپس زوج نقاط ابتدایی و انتهایی هر دو پاره خط از لحاظ منطبق بودن با یکدیگر مورد بررسی قرار می گیرند. در صورتی که این زوج نقاط منطبق باشند دو خط متلاقی هستند. جایگشت انتخاب به صورت $\frac{1}{2} \binom{n}{2}$ و از مرتبه n^2 است.

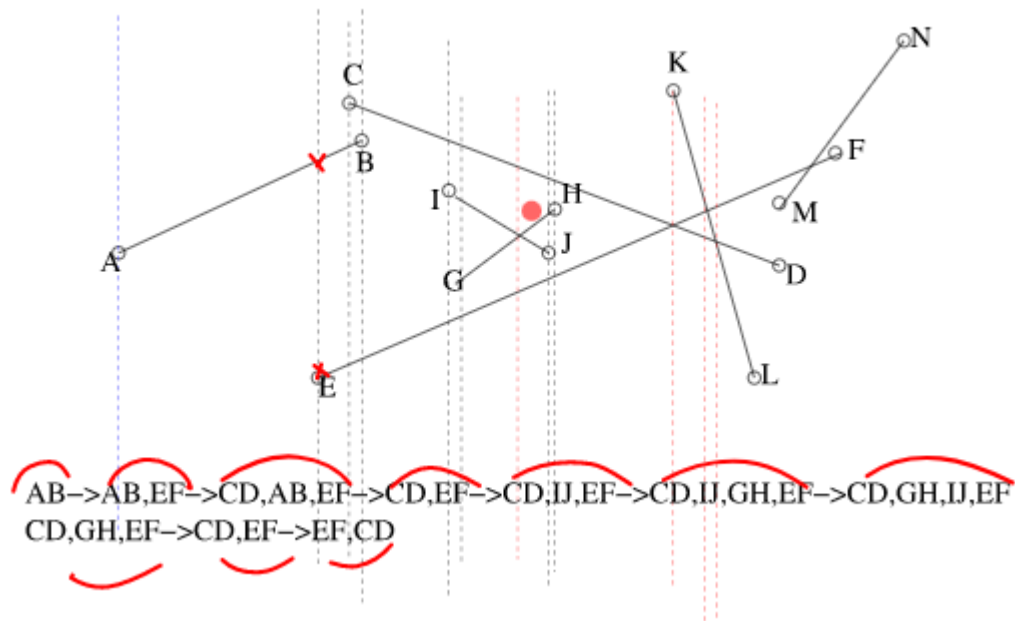
راه دوم: جارو زدن خطی (Line swiping): یک سری خط عمودی می کشیم و نقاط برخورد با پاره خطها درون یک داده ساختار ثبت می شوند. در مثال زیر اگر دو خط متقاطع CD و FE در نظر گرفته شوند خطوط عمودی در ابتدا به CD و سپس به FE برخورد می کنند. ترتیب نوشتن برخورد در یک داده ساختار برای این خطوط، قبل از نقطه برخورد به صورت CD-EF و بعد از رد شدن از تقاطع به صورت EF-CD است. همانطور که ملاحظه می شود این ترتیب برعکس شده است که این برعکس شدن نمایش دهنده تقاطع است.



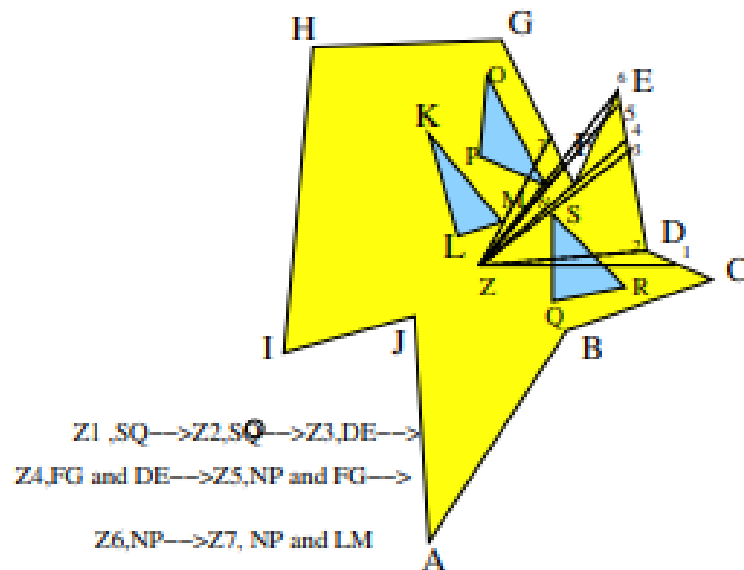
برای ثبت نقاط نیاز به یک داده ساختار وجود دارد. یک ساختار داده مرتب شده می تواند این کار را تسهیل کند. در مثال زیر نقاط تقاطع در یک داده ساختار نوشته می شوند. سپس به جلو می رویم تا وضعیت تغییر کند. وضعیت جدید در داده ساختار جدید نوشته می شود. با مقایسه داده ساختار قبلی و فعلی و مشاهده نقاط معکوس شده از قبیل S_4S_3 که بعداً به S_3S_4 تبدیل شده است می توان تقاطع این دو خط را تشخیص داد.



از هر داده ساختار می توان برای نگهداری ترتیب تقاطعها استفاده کرد. در اینجا از Heap استفاده شده است. از BST نیز می توان استفاده کرد. مثال زیر اتفاقات swping را نشان می دهد: مثلاً IJ و JH را می توان بررسی کرد.



ایده‌ی قبل می‌تواند در مسائل پیچیده گسترش یابد. در شکل زیر یکسری نقاط وجود دارد. می‌خواهیم ببینیم کدام یک از نقاط از نقطه‌ی Z قابل رویت است. تکنیک جارو حالت شعاعی پیدا می‌کند. در این شکل، اشکال آبی موانع هستند. اگر نقاط از نقطه‌ی Z پاره‌خطی به سمت لبه‌ی شکل زردرنگ کشیده شود و این پاره‌خط به یکی از پاره‌خط‌های شکل آبی رنگ برخورد کند در نتیجه لبه‌ی شکل زردرنگ از نقطه‌ی Z قابل رویت نیست. مثلاً از نقطه‌ی Z پاره‌خطی به سمت لبه‌ی ED از شکل زردرنگ کشیده می‌شود. اگر در میانه‌ی راه به پاره‌خطی مانند SR از شکل آبی رنگ برخورد شود در واقع ED از نقطه‌ی Z قابل رویت نیست.

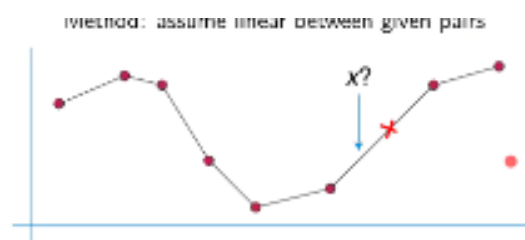




جلسه یازدهم

مسئله ی جستجو: یکسری کلید وجود دارد. می خواهیم کلیدی خاصی را مورد جستجو قرار دهیم. جستجو می تواند موفق یا ناموفق باشد. در جستجوی ناموفق به این نتیجه می رسیم که کلید مورد جستجو وجود ندارد. در این مسئله داده ها می توانند مرتب باشند یا نباشند. اگر مسئله به صورت exact search در نظر گرفته شود می توانیم مسئله ی جستجو را به این صورت مطرح می شود که آیا کلیدی که به صورت کوئری مطرح شده است می تواند در داده ساختار ما وجود داشته باشد یا خیر؟ برای حل این مسئله می توان از هشینگ یا درهم سازی استفاده کرد. همچنین از درخت های جستجو نیز می توان استفاده کرد.

مسئله ی جستجوی باینری، مسئله ای است که کلیدهای آن یک مجموعه ی مرتب شده هستند. می خواهیم عملیات جستجو را انجام بدهیم. این جستجو به این صورت است که می خواهیم یک کلید را به طور دقیق پیدا کنیم یا محدوده ی یک کلید را در درخت جستجو پیدا نماییم. دو تا از کاربردهای معروف مسئله ی جستجوی باینری درون یابی یا طبقه بندی به روش نزدیک ترین همسایه است. درون یابی امکان پیدا کردن یک نقطه بین دو نقطه از یک تابع را به ما می دهد. به طوریکه این نقطه ی سوم باید بر روی همان تابع قبلی باشد.



در مسئله ی جستجوی باینری عملیاتی داریم. یک مجموعه از کلیدها داریم. کلیدها مرتب هستند یا رابطه ی ترتیب هستند. کلیدها می تواند عدد رشته ها و ... هستند. عملیات قابل اعمال عبارتند از:

- جستجوی q
- کوچکترین کلید بعد از q (successor)
- بزرگترین کلید که قبل از q وجود دارد (predecessor)
- نزدیکترین همسایه: می تواند کلید، successor یا predecessor باشد.

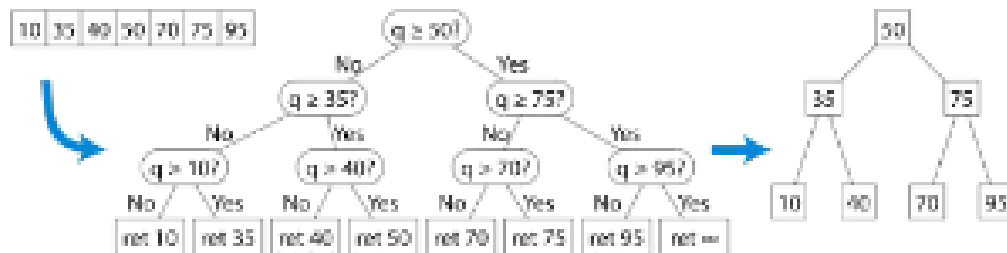
درخت های جستجوی باینری برای حل مسائل جستجوی باینری استفاده می شوند. درخت های جستجوی باینری یک درخت دودویی است که هر گره می تواند دو فرزند داشته باشد و محتوی فرزند سمت چپ از ریشه کوچکتر و سمت راست از ریشه بزرگتر است.

باینری سرچ به عنوان یک مدل فرمال محاسباتی تعریف می شود. در MIT درخت جستجوی باینری به عنوان یک مدل محاسباتی تعریف می شود به صورتی که یک زیر مجموعه از pointer Machin هستند. BST یک مدل محاسباتی است که داده ها باید به صورت یکسری کلیدهای مرتب ذخیره شده اند و هر کلید یک اشاره گر

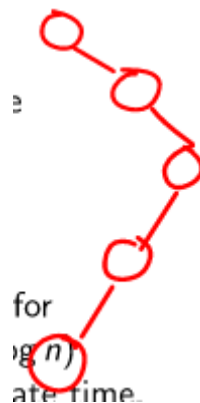


داده ساختار پیشرفته

به فرزندان سمت چپ، راست و پدر دارد. مقادیر کلید سمت چپ کوچکتر از ریشه و کلید سمت راست بزرگتر از ریشه است. این مدل محاسباتی از یک نوع کوئری به نام $Seach(x)$ پشتیبانی می‌کند. عملیات قابل پشتیبانی عبارتند از: حرکت فرزندان به سمت چپ و راست، حرکت به سمت پدر، چرخش یا جابه‌جایی کلید یک درخت جستجوی باینری داده‌ساختاری است که یک‌سری مقایسه را در خود اینکد کرده است. مفاهیم ساکسیسو و پریدیسور در آن ذخیره شده است. مثال نوشته شود.



مفهوم دیگری که در اینجا باید تعرف شود درخت جستجوی دودویی متوازن است. مفهوم توازن بر روی پارامترهایی مختلفی تعریف می‌شود. اگر داده‌های ما ایستا باشند می‌توانیم یک آرایه‌ی مرتب‌داشته باشیم که جستجو در مرتبه‌ی $O(\log n)$ انجام می‌شود. الگوریتم جستجوی باینری این کار را در این مرتبه انجام می‌دهد. اگر یک درخت جستجوی دودویی داشته باشیم مرتبه‌ی جستجو ارتفاع درخت است. ارتفاع درخت لزوماً $O(\log n)$ نیست. ممکن است ارتفاع درخت به صورت زیر است که از مرتبه‌ی $O(n)$ است. به همین دلیل مفهوم توازن بر روی درخت مطرح می‌شود. مفهوم زمان جستجو به معنی تعداد ملاقات نودهایی است که برای پیدا شدن یک نود خاص پیمایش می‌شوند.



امروزه با توجه به مفهوم کلان داده، بحث توازن و زمان جستجو بسیار مهم است. زیرا با افزایش حجم داده، مفهوم زمان می‌تواند یک کاربرد را شونی و یا نشونی کند.

درخت‌های جستجوی دودویی متوازن: درخت جستجویی است که سعی می‌کند مرتبه‌ی جستجو را در $O(\log n)$ نگه دارد. در تعریف قبلی داده‌ها ثابت هستند و تغییری نمی‌کنند. اگر داده‌های پویا داشته باشیم می‌خواهیم زمان جستجو در بدترین حالت (امید ریاضی و یا به صورت سرشکن) از مرتبه‌ی $O(\log n)$ باشد.



داده ساختار پیشرفته

بحث update، یا تغییر درخت در اینجا مطرح می‌شود. به عبارتی با حذف یا تغییر داده‌ها باید درخت به‌روزرسانی شود.

توازن: دو بحث به وجود می‌آورد:

- **Rebuild**: اجازه می‌دهد طی یکسری عملیات، درخت نامتوازن می‌شود ولی این نامتوانی از یک حد خاصی که عبور کند باید زیردرخت‌های درخت متوازن شود.
- **Rotation**: زمانی که عنصر جدیدی حذف یا اضافه شود درخت را مجبور می‌کند که ساختار درخت به طور مرتب تغییر می‌کند تا توازن درخت حفظ شود. تغییرات به طور محلی رخ می‌دهند. شبیه درخت AVL است.

توازن در درخت متوازن به این معنی است که با اضافه یا کم کردن نودها، ارتفاع $\log(n)$ برقرار است. مفهوم توازن می‌تواند با استفاده از وزن‌ها تعریف شود از جمله این موارد می‌توان به موارد زیر اشاره کرد:

Weighted-balanced tree بر روی یکسری وزن متوازن می‌شود. درخت قرمز سیاه ، WAVL و ... از انواع دیگر درختان متوازن هستند.

B-tree: یک ساختار مهم عملی است که برای داده‌هایی استفاده می‌شود که نمی‌تواند به طور کامل درون کش قرار بگیرند. هدف کمینه کردن cache miss است. انواع مختلفی از B-tree‌ها وجود دارند.

Optimal binary search trees: آیا درخت باینری بهینه داریم یا خیر؟ ما می‌دانیم یکسری درخت داریم که خود را متوازن می‌کنند. این درختان برای هر پرس‌وجو $O(\log n)$ زمان مصرف می‌کنند. آیا داده‌ساختاری بهتری داریم که مرتبه‌ی آن از $O(\log n)$ بهتر باشد؟

Static optimality: یک مجموعه از کیلدها داریم و می‌دانیم که هر کلید با چه فرکانسی مورد جستجو قرار می‌گیرند. اگر این چنین مشخصاتی از یک داده داشته باشیم می‌توانیم کیفیت درخت را اندازه بگیریم که به صورت زیر است:

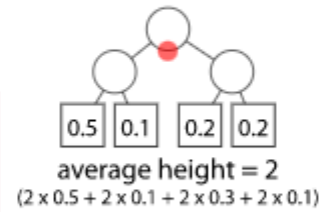
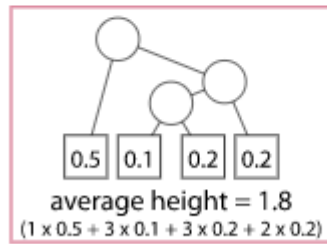
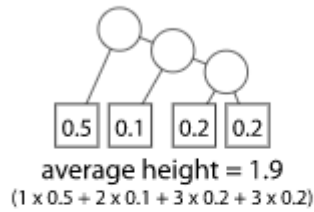
$$= \sum_i p_i \times (\text{length of path to } x_i)$$

در واقع این تعریف میانگین مسیر جستجو برای هر کلید را محاسبه می‌کند. هر چه عدد میانگین کوچکتر باشد درخت با کیفیت‌تر است.

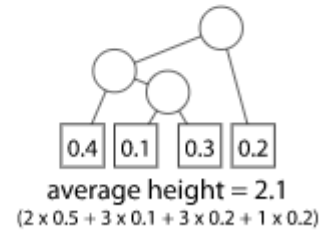
یکسری کلید با یکسری کد فرکانس داریم. میانگین‌ها را حساب کرده است. به این نتیجه رسیده است که کیفیت درختی که دور آن خط کشیده شده است درخت بهینه‌تری است.



With external node frequencies 0.5, 0.1, 0.2, 0.2:



Optimal!





جلسه دوازدهم

داینامیک آپتیمالیتی: در این مسئله فرض می شود همانطور که عملیات جستجو انجام می شود عملیات اضافه و حذف کردن آیتم ها تحت عنوان به روز رسانی اضافه شده است. این همان مفهوم رشد است. فرض دیگری که وجود دارد این است که برخی از آن ها آیتم های داغ هستند. به عنوان مثال در مراسم اسکار یکی از بازیگران سیلی به گوشی مجری زده است بنابراین افراد زیادی این دو آیتم را مورد جستجو قرار می دهند. این دو آیتم، آیتم داغ هستند. با این مفروضات آیا می توان مرتبه ی جستجو را نسبت به حالت ایستا (درختان ایستا) مقایسه و بهبود داد؟

الگوریتم برنامه ی پویا: وقتی مقدار n زیاد است هزینه ی آن بالا است. از مرتبه ی $O(n^3)$ است. همچنین الگوریتم های وجود دارد.

بهترین الگوریتمی که برای ساخت آپتیمال تیری وجود دارد از چه مرتبه ای است؟ و آیا می توان درختی از نوع داینامیک وجود دارد که با درخت ایستای آپتیمال هم مرتبه باشد.

ایده ی ساخت درخت داینامیک آپتیمال: به طور مرتب درخت ارنج شود و آیتم هایی که داغ هستند به ریشه نزدیک شوند. مبنای مقایسه یا baseline به عنوان Competitive ratio شناخته می شود. نرخ رقابتی برای مقایسه ی درخت ساخته شده پویای بهینه پیشنهاد شده است.

اگر A یک الگوریتم آنلاین باشد (منظور عملیات جستجو کردن بر روی یک داده ساختار باشد) که هیچ اطلاعاتی در مورد اینکه در آینده چگونه با فرض کنیم یک الگوریتم آنلاین و یک داده ساختار داریم و هیچ ترتیبی برای عملیات را نداریم. همچنین یک الگوریتم B داریم که همان عملیات قبلی را انجام می دهد اما یکسری اطلاعات قبلی در مورد آن داریم. حال الگوریتم B یک الگوریتم آپتیمال است. برای تمام رشته عملیات ممکن که می تواند بر روی داده ساختار کار کند مانند زیر:

$1 \sim \text{insert}(n_1)$
 $2 \sim \text{insert}(n_2)$
 $3 \sim \text{search}(n_1)$
 $4 \sim \text{search}(n_3)$
 $5 \sim \text{delete}(n_2)$ (one that is
 $6 \sim \text{insert}(n_4)$
 Let B be a
 optimal



داده ساختار پیشرفته

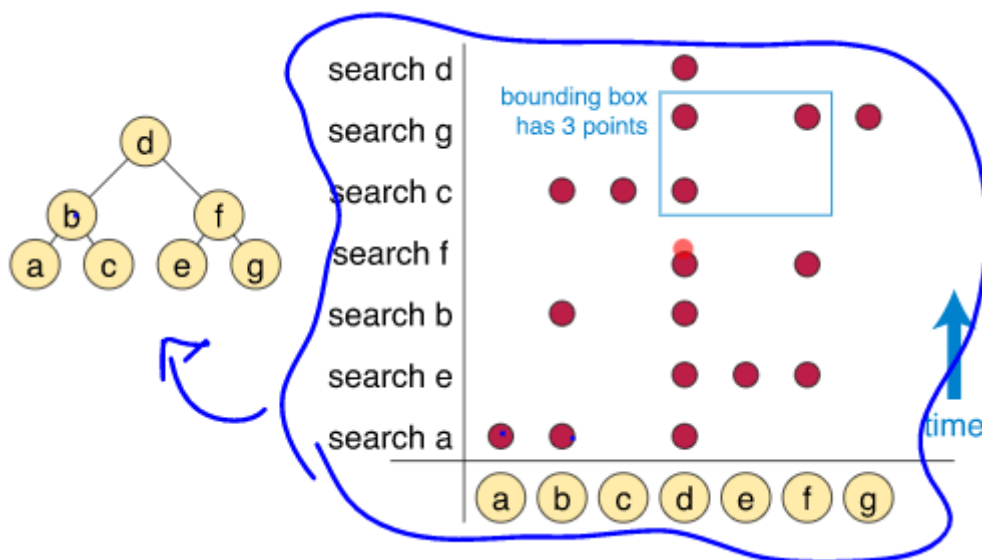
به ازای هر کدام از رشته عملیات بالا، نسبت هزینه به ازای الگوریتم A و B را بدست می آوریم و ماکسیمم را بدست می آوریم. منطقی این است که هزینه الگوریتم A را نسبت به الگوریتم B به دست آوریم. در یک داده ساختار

بحث Dynamic optimality conjecture: به صورت یک قضیه نیست. فرض می کنیم که ما درخت های جستجوی داینامیک داریم و اجازه می دهیم خود را بازسازی کنند. دو مورد هزینه برای بازسازی وجود دارد.

۱- طول مسیر همه ی علیات

۲- سبب مرتب سازی

Greedy ASS trees: این راه حل به این صورت محاسبه می شود: فرض کنید یک درخت جستجوی دودویی داریم. فرض کنید یک سلسله عملیات داریم. در محور X عملیات و در محور Y زمان را نشان می دهد. عملیات می تواند جستجو، حذف و اضافه باشند. به ازای هر عملی، کلیدهایی مورد جستجو قرار می گیرند. یک درخت جستجوی دودویی را به صورت زیر نمایش می دهیم:



درخت جستجوی ASS به این صورت ایجاد می شود که اگر هر دو نقطه ای را پیدا کنیم که سطر و ستون آنها یکی نباشند نقطه ی سوم را پیدا کنیم که سطر و ستون نقطه ی سوم با ستون و سطر دو نقطه ی دیگر یکی نباشند



جلسه سیزدهم: اسلاید دانشگاه استنفورد

معرفی splay tree: یک درخت Bst است که مرتبه‌ی جستجوی آن بسیار موثر است.

مسئله جستجو: در مسئله‌ی جستجو تعدادی کلید داریم و کلیدها دو حالت استاتیک و داینامیک هستند. می‌خواهیم آن‌ها را در یک داده ساختار ذخیره کنیم به طوریکه به بهترین شکل جستجو شوند. چند مدل وجود دارد.

مدل اول: یک تعداد کلید وجود دارد و کوثری‌هایی که بر روی کلیدها در قالب BST اتفاق می‌افتد می‌تواند هر شکلی داشته باشد. تحلیل مورد استفاده تحلیل در بدترین حالت است. بدترین حالت در جستجو این است که ارتفاع درخت زیاد است و کلید جستجو در برگ درخت باشد و یا حتی ممکن است این کلید در برگ‌ها هم وجود نداشته باشند. در این حالت کوثری‌ها در بدترین حالت طراحی می‌شوند.

در درخت صفحه‌ی ۶ بالانسی وجود دارد.....

مدل دوم: کوثری‌های ما حالت نمونه‌برداری شده دارند. اینکه یک کوثری چقدر تکرار شده باشد بسیار مهم است. فرض کنید یکسری کلید داریم و احتمال دسترسی به کلیدها یکسان نیست. مثلاً کلید شماره‌ی ۳ در ۴۰ درصد مواقع احتمال دارد مورد دسترسی قرار بگیرد ولی کلیدهای ۴ و ۵ در ۱ درصد مواقع مورد دسترسی قرار می‌گیرند. در بحث Dynamic Optimality سوال ما این است که درخت بهینه‌ی BST چه درختی است به طوریکه بهترین جستجوها در آن انجام می‌شود. در شکل زیر درخت با توجه به کلیدها بالانس شده است اما چون بحث احتمالات وجود دارد نمی‌توانیم بگوییم درخت سمت چپ بالانس است. در واقع این نوع بالانس زمانی موثر بود که احتمال دسترسی به تمام کلیدها یکسان باشند. برای این مثال اگر به صورت حریصانه به درخت نگاه شود درختی که ارتفاع بیشتری دارد بالانس‌تر و بهینه‌تر است زیرا کلیدهایی که احتمال بالاتری دارند در ریشه یا نزدیک به ریشه قرار دارند. اگر احتمال کلیدها با هم برابر باشند درخت نشان داده شده بهینه نیست (شکل سمت چپ). به طوریکه اگر تحلیل سرشکنی انجام دهیم احتمالاً درختی که از لحاظ ارتفاع بالانس‌تر باشند مناسب‌تر است.

شکل باید قرار داده شود.....

به طور کلی مدل اول بیان می‌کند که ارتفاع درخت باید حداقل نگه داشته باشد. همچنین مدل دوم بیان می‌کند که کلیدهایی که احتمال بیشتری دارند باید نزدیک به ریشه نگه‌داری شوند.

اسلایدها میزان تراکنش را محاسبه می‌کند و ما را به سمت درخت بهینه می‌فرستد.

مدل سوم: می‌خواهیم یک locality مشخص کنیم. به طوریکه locality می‌تواند Temporal یا Special باشد. Spatial یک نوع locality مکانی است که در این مدل مورد بررسی قرار می‌گیرد. Temporal نیز



در مدل چهارم مورد بررسی قرار می‌گیرد. در این حالت منظور از **locality** این است که زمانی که یک کلید مورد جستجو قرار می‌گیرد کلیدهایی نزدیک به آن نیز مورد دسترسی قرار می‌گیرند. فرض کنید رویدادی در زمان ۹ رخ داده است. احتمالاً برای جستجوکننده مهم است که بداند در زمان‌های قبل و بعد از آن یعنی در زمان‌های ۸ و ۱۰ چه اتفاقاتی می‌افتد. زیرا مفهوم توالی برای زمان یک مفهوم معنی‌دار است و منطقی است که یکسری از کلیدها را نزدیک به هم بدانیم. با پیمایش **Inorder** یک یا دو کلید قبل و بعد از کلید شماره‌ی ۹ از نظر **spacially** یا مکانی نزدیک به این کلید تعریف می‌شوند. بنابراین مفهوم **locality** برای کلیدهایی که مورد جستجو قرار می‌گیرند یکی دیگر از حالت‌هایی است که می‌تواند در **BST** اتفاق بیفتد. بنابراین کلیدهایی که از نظر مکانی به مکان جستجو نزدیک هستند آن‌ها را در سطوح بالاتری از درخت قرار می‌دهیم و به عبارتی برای آن‌ها اهمیت قائل می‌شویم.

در مثال مدل ۳ مفهومی به نام **finger** نمایش می‌دهد. ویژگی انگشت پویا جایی است که کلید مورد جستجوی ما یک پوینتر است و کلید ما نزدیک به این محلی است که مورد جستجو قرار می‌گیرد. به عنوان مثال کلید ۳۷ در مرتبه‌ی بسیار پایین‌تر نسبت به کلید ۸۷ قرار می‌گیرد. بنابراین **Dynamic finger tree** می‌گوید اگر **BST** به گونه‌ای تعریف شود که می‌خواهیم n جستجو بین یکسری کلید انجام دهیم اگر یک کلید مورد جستجو قرار بگیرد سایر کلیدهای اطراف آن نیز بر اساس خاصیت **locality** مورد جستجو قرار می‌گیرند و ویژگی انگشت پویا مطرح می‌شود.

تصویر

در اینجا با یک ضریب ۲ کلید بعدی، دو تا کلیدی بعدی و چهارتا کلید بعدی را مارک می‌کند. در واقع این‌ها محدوده‌ی **Spatial locality** را تعریف می‌کند. در این مثال کلیدها ۱۷ تا ۳۴ به عنوان محدوده‌ی **Spatial locality** شناخته می‌شوند. بنابراین یکی از این کلیدها را با مرتبه‌ی $O(\log n)$ جستجو می‌کنیم و سایر آن‌ها به دلیل آنکه به صورت متوالی پشت سرهم قرار گرفتند از مرتبه‌ی $O(1)$ جستجو می‌شوند. به عبارتی سایر نودهایی که خاصیت **locality** دارند مجدداً از ریشه مورد جستجو قرار نمی‌گیرند. در واقع در اینجا ویژگی **Dynamic finger** یک درخت جدید را به ما معرفی می‌کند به طوریکه برگ‌های این درخت خاص کلیدهای جستجو هستند و گره‌های داخلی یک محدوده را به ما نشان می‌دهند.

مدل چهارم: همان بحث محلی بودن در آن است فقط محلی بودن از لحاظ زمانی مطرح است. یعنی اگر کلیدی مورد جستجو قرار گرفت احتمال دارد در آینده هم مورد جستجو قرار بگیرد. مثلاً در دنیای واقعی اتفاقی می‌افتد و این اتفاق در صفحه‌ی گوگل به شدت مورد جستجو قرار می‌گیرد. در اینجا احتمال می‌دهیم در روزهای آینده، این اتفاق مورد جستجو قرار می‌گیرد. بنابراین در درخت جستجو، نودی که مربوط به این اتفاق است در سطح بالاتری از درخت قرار می‌گیرد. حدس ما ممکن است درست باشد که این اطلاعات در



داده ساختار پیشرفته

سطوح بالای درخت باقی می ماند اما اگر درست نباشد بعد از مدتی از سطوح بالای درخت به سمت سطوح پایین آن حرکت می کند.

چهار مدل تعریف شده است که هر کدام از چهار مدل یکسری ویژگی خاص داشتند:

۱- همه ی نودهای درخت از لحاظ ارتفاع بالانس هستند. جستجو در این درخت از مرتبه ی $O(\log n)$ است. BSTهای متوازن سنتی این ویژگی را دارند.

۲- ویژگی دوم آنتروپی است که از نمونه های آن درختان دودویی وزن دار هستند.

۳- Dynamic Finger یا ویژگی انگشت پویا، ویژگی Level-linked BST with finger را معرفی می کند. در این مدل ویژگی محلی بودن از لحاظ مکانی مورد بررسی قرار گرفت. یعنی اگر کلیدی مورد جستجو قرار می گیرد احتمال می رود سایر کلیدهایی که از لحاظ منطقی و مکانی به آن نزدیک هستند در آینده نیز مورد جستجو قرار بگیرند این مجموعه working set نامیده می شود و در سطح بالایی از درخت قرار می گیرد تا زمان دسترسی به این گره ها کاهش یابد.

۴- Working set: ویژگی محلی بودن را با توجه به زمان را بیان می کند و گره های محلی با این مفهوم در یک Working set قرار می گیرند.

Working set مفهومی به نام همبستگی یا correlation را به وجود می آورد. این سوال مطرح می شود که چگونه گره های مرتبط به هم را شناسایی کنیم؟ در اینجا همبستگی باید تفسیر شود. به طوریکه اگر محلی بودن با توجه به زمان در نظر گرفته شود همبستگی داده ها با توجه به زمان تفسیر می شود. یعنی گره ها با توجه به مفهوم زمان مقایسه و مکان یابی می شوند. همبستگی شکل های پیچیده تری نیز به خود می گیرد همبستگی می تواند با توجه به کلمات نیز تعریف شود مثلاً اگر بخواهیم یک دیکشنری پیاده سازی کنیم همبستگی می تواند با توجه ریشه ی کلمات یا شباهت کلمات تعریف شود.

سوالی که در اینجا مطرح می شود این است که آیا یک BST داریم که تمام ویژگی های بالا را گارانتی کند؟ زمانیکه صحبت از گارانتی می شود باید موضوعی اثبات شود. در اینجا این موضوع اثبات نمی شود.

تنها BST که همه ی این موارد بالا را پوشش می دهد Splay tree است.

مسئله: فرض کنید یک درخت جستجو داریم که شامل نودهای $\{x_1, \dots, x_n\}$ است. می خواهیم یک رشته ی جستجو مانند $x_1 \dots x_n$ را در این درخت مورد جستجو قرار بدهیم به طوریکه به طور متوسط بهترین زمان جستجو را برای هر یک از عملیات جستجو داشته باشیم.

در اینجا ویژگی به نام دسترسی متوالی یا Sequential access property تعریف شده است.



داده ساختار پیشرفته

برای پایان نامه‌های دکتری: (به طور کلی یک مسئله باید تعریف شود سپس یکسری قضیه تعریف می‌شود و این قضایا که راه حل ما هستند به طور جداگانه اثبات می‌شوند تا مسئله در نهایت حل شود. در اینجا باید مفهوم سازی کرد).

ویژگی اول همان BST معمولی است.

ویژگی دوم: دسترسی متوالی: یک BST ویژگی دسترسی متوالی را دارد اگر رشته‌ی جستجوی $\{x_1, \dots, x_n\}$ به طور متوسط و سرشکنی به ازای هر عمل $O(1)$ طول بکشد. در اینجا یکی از کلیدها از $O(\log n)$ جستجو می‌شود و بقیه آن از $O(1)$ جستجو می‌شوند. می‌توان درخت را به صورت پیمایش Inorder نگه‌داری کرد. با این کار کلیدها به صورت مرتب شده در دسترس هستند. حال در هنگام اجرای عملیات جستجو اولین عملیات از مرتبه‌ی $o(\log n)$ است و سایر دسترسی‌ها از مرتبه‌ی $O(1)$ است.

ویژگی انگشت پویا: یک BST ویژگی انگشت پویا دارد اگر برای هر رشته عملیات جستجو از x_1 تا x_m زمان سرشکنی دسترسی به x_k برابر با $O(|x_k - x_{(k-1)}|)$ باشد. فاصله‌ی بین گره‌ها باید قابل تخمین زدن باشد. یعنی بتوانیم از جریان عملیات از 1 تا m حرکت کنیم و فاصله‌ی پرش‌های بیش از یک داشته باشیم. زمان جستجو بسیار کمتر از زمانی است که از ریشه بخواهیم جستجو کنیم. وقتی جستجوها مربوط به کلیدهایی باشند که در یک فضا باشد مرتبه‌ی جستجو برابر با $O(1)$ است. زمانیکه از ویژگی انگشت پویا استفاده شود لازم نیست برای جستجوی هر کلید از ریشه این کار را انجام دهیم. بنابراین سرشکن عملیات با ضربی از $O(1)$ قابل انجام است. مفهوم دیگر ویژگی انگشت پویا این است که کلیدهایی که نزدیک کلید اخیرا مورد دسترسی قرار گرفته هستند سهل الوصول هستند (Working set).

ویژگی یکنواختی: اگر کلیدی باشد که اخیرا مورد دسترسی قرار گرفته باشد و حوالی فضای جستجوی ما باشد هزینه جستجوی آن بسیار پایین است و سهل الوصول هستند.

بهینگی پویا: بهینگی پویا بیان می‌کند که شکل درست کردن آن خاص باشد و هزینه‌ی تقریبی عملیاتی که درون آن انجام می‌شود ضریب ثابتی از الگوریتمی باشد که در درخت ایستا ساخته می‌شود. در بهینگی پویا به خوبی الگوریتم‌های ایستا کار می‌کند فقط با یک ضریب ثابت.

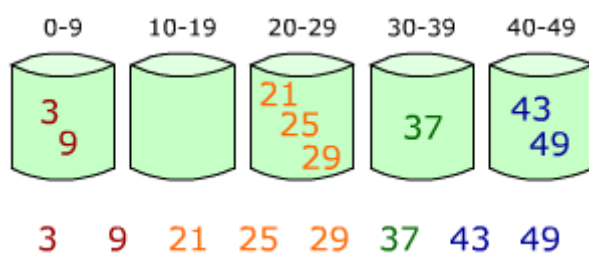


جلسه چهاردهم: مبحث درهم سازی

مقدمه: درخت جستجوی باینری مسئله‌ی جستجو را به نحو مناسبی حل می‌کند. شکل‌های مختلفی از باینری سرچ تعریف می‌شود. در این شکل‌های مختلف سعی می‌شود با حالت بهینه این مسئله حل شود. شکل‌های مختلف درخت باینری سعی می‌کنند با مینیمم کردن ارتفاع درخت، مسئله‌ی جستجوی باینری را در زمان بهینه حل کنند. در بسیاری از این موارد مرتبه‌ی پیچیدگی از $O(\log n)$ است.

زمانیکه فرکانس جستجو زیاد است مرتبه‌ی $O(\log n)$ برای ما جذابیت ندارد. ما می‌خواهیم در مسائل کلان داده، مسئله جستجو را از مرتبه‌ی $O(1)$ حل کنیم. مسئله‌ی هشینگ سعی می‌کند بازیابی و ذخیره در مرتبه‌ی $O(1)$ انجام شود.

فرض کنید یک آرایه‌ی باکت داریم. آرایه‌ی باکت آرایه‌ای است که سایز محدودی دارد و قرار است مقداری داده در خود ذخیره کند. زمانیکه محدودیت طول داریم و طول آرایه m باشد. اگر بخواهیم $m+1$ عنصر ذخیره کنیم لازم است در یک خانه بیش از دو عنصر قرار گیرد. در اینجا زمانیکه برخورد پیش می‌آید هر خانه دارای یک سطل است که اطلاعات اضافی در آن ذخیره می‌شود. این سطل‌ها می‌توانند آرایه، درخت باینری، لیست و ... باشند. شکل زیر آرایه‌ی باکت را نمایش می‌دهد.



نحوه‌ی نگاشت داده‌ها به خانه‌های آرایه مسئله‌ی دیگری است که باید مورد توجه قرار بگیرد. در واقع مکانیزم ذخیره‌سازی آیتم‌ها باید بررسی شود. این مکانیزم توسط تابعی به نام هش فانکشن اعمال می‌شود که که نحوه نگاشت داده‌ها را مشخص می‌کند. Hash function یک عدد را می‌گیرد و محل ذخیره‌سازی آن را از 0 تا $m-1$ مشخص می‌کند. زمانیکه Hash Function محل ذخیره‌سازی یک عدد را مشخص می‌کند دو حالت به وجود می‌آید:

- ۱- در محل ذخیره‌سازی هیچ عنصری وجود نداشته است و عنصر مورد نظر در این خانه قرار می‌گیرد.
- ۲- در محل ذخیره‌سازی از قبل عنصری درج شده است و عنصر مورد نظر با عنصر قبلی برخورد می‌کند.

فرض شود سایز آرایه به اندازه‌ی بزرگ در نظر گرفته شود که برخورد رخ ندهد. همچنین فرض می‌شود چالش فضای ذخیره‌سازی وجود ندارد. با انتخاب تابع فانکشن مناسب می‌توان عناصر مورد نظر را در خانه‌های آرایه جایابی کرد. اگر Hash Function مناسب وجود داشته باشد و هر عنصر در یک خانه از آرایه قرار گرفته



داده ساختار پیشرفته

باشد و هر خانه آرایه‌ی یک اندیس داشته باشد با استفاده از این Hash Function می‌توان عملیات بازیابی را در مرتبه‌ی $O(1)$ انجام داد. زیرا این Hash Function اندیس خانه‌ای که مورد جستجو است را به ما بر می‌گرداند. در اینجا تنها هزینه جستجو، هزینه‌ی تابع است. به طور کلی دسترسی به خانه‌ای مانند $x[0]$ از مرتبه‌ی $O(1)$ است. بهتر است تابع هشینگ پیچیده نباشد. بنابراین می‌توان به ایده‌آل مورد نظر یعنی $O(1)$ می‌رسیم.

از جمله مشکلاتی که در اینجا وجود دارد این است که داده‌ها معمولاً پویا هستند و ممکن است به صورت stream بیاید. داده‌ها می‌توانند تکراری باشند. موضوع حذف و اضافه داده‌ها پیش می‌آید و همچنین فرض می‌شود از لحاظ ذخیره‌سازی مشکلی نداریم. چالش اصلی در اینجا زمان است. همچنان مسئله‌ی فضای ذخیره‌سازی در مسائل کلان داده می‌تواند از جمله چالش‌های پیش رو باشد. چالش دیگر تابع رمزگذاری است که بهتر است از پیچیدگی کمتری برخوردار باشد.

برای مواجهه با برخوردها راه‌های متفاوتی وجود دارد که عبارتند از:

- **آدرس‌دهی باز:** آیتم‌ها در آرایه ذخیره می‌شود و اگر برخورد اتفاق افتاد در خانه‌ی بعدی خالی قرار می‌دهیم. در بدترین حالت از مرتبه‌ی $O(n)$ می‌بایست هزینه جستجو داد. زیرا ممکن است در اثر برخورد مکان یک عنصر در خانه‌ای نباشد که Hashing function اولین بار مشخص می‌کند.

نکته: در Cuckoo hashing می‌توان به طور سرشکنی مرتبه‌ی $O(1)$ را بدست آورد.

- **آدرس‌دهی بسته:** عناصری که برای آن‌ها برخورد رخ می‌دهد در یک داده ساختار مانند لیست پیوندی، درخت باینری و ... ذخیره می‌شوند.

- **Perfect hashing:** تابع رمزگذاری انتخاب می‌شود که هیچ برخوردی ایجاد نکند.

تمرکز ما بر روی طراحی تابع درهم‌سازی و داده ساختار است. تابع درهم‌سازی خوب آن است که برخورد کمتری ایجاد کند. در ساده‌ترین حالت، زمانیکه از جستجوی خطی استفاده می‌شود در بدترین حالت مرتبه‌ی جستجو از $O(n)$ است.

فرض شود که هر زوج داده‌ی ما به صورت (k, v) باشند به طوریکه k ، کلید و v مقدار یک کلید باشد:

1) Linear proving:

$(key, value) \rightarrow A[n]$ (be reserved)

$A[(n+1) \bmod N]$

.....



داده ساختار پیشرفته

در این روش چنانچه برخوردی رخ دهد به اولین خانه‌ی خالی بعد از آن برخورد رفته و عنصر مورد نظر را در آن ذخیره می‌کنیم. این مورد هزینه دارد. همچنین یک هزینه هم زمان جستجو به وجود می‌آید. در این حالت در بدترین حالت جستجو در $O(n)$ اتفاق می‌افتد.

2) Quadratic Probing:

$A[n+f(j) \bmod(N)] :$

این روش جستجوی دو گانه نام دارد. در اینجا تابع جلو رفتن پیچیده‌تر می‌شود. به طوریکه به جای آنکه یک واحد یک واحد جلو برود از تابع $A[n+f(j) \bmod(N)]$ برای جلو رفتن استفاده می‌شود. نشان می‌دهد که دفعه‌ی چندمی است که پیمایشی انجام می‌شود تا خانه‌ی خالی مناسب برای عنصر مورد نظر پیدا شود. $F(j) = j^2$ یک تابع مناسب است. زیرا در هر بار یک جهش برای چک کردن انجام می‌دهد و پشت سر هم این چک کردن اتفاق نمی‌افتد.

3) double hashing:

در این روش دو تابع برای هشینگ تعریف می‌شود. دلیل تعریف این دو تابع رخ دادن مسئله‌ی کلاسترینگ است. در این مسئله، اگر داده‌ها شکل خاصی داشته باشند در یک مکان جمع می‌شوند. مثلاً داده‌هایی که باقیمانده‌ی آن‌ها پنج است در یک بخش از آرایه جمع می‌شوند. این حالت موجب کلاسترینگ و برخورد می‌شود. دابل هشینگ به دلیل ممانعت از مسئله‌ی کلاسترینگ رخ می‌دهد. به طوریکه جایابی عناصر به یک تابع هش محدود نباشند. فرض کنید تابع هش اولیه h و تابع ثانویه‌ی h' را داریم اگر دو تابع h و h' وجود داشته باشند. فرض شود یک کلید در پیمایش اول با استفاده از تابع h مکان‌یابی شود و در این پیمایش مکان مناسبی برای جایابی این کلید یافت نشود. در این حالت در پیمایش دوم سعی می‌شود از تابع h' استفاده شود.

شیوه‌ی دیگر مواجهه با مسئله‌ی برخورد در آدرس‌دهی باز، استفاده از الگوریتم کوکوهشینگ است. کوکوهشینگ جایگزین برای متدهای بالا است. رفتاری که در کوکوهشینگ رخ می‌دهد از فاخته‌ها الگو می‌گیرد. یعنی بدترین مکان برای تخم‌گذاری انتخاب می‌شود.

به طور کلی سه استراتژی برای مسئله‌ی برخورد معرفی شده است.

آدرس‌دهی بسته: تمام داده‌هایی که برخورد دارند در یک داده‌ساختار کمکی قرار می‌گیرد.

آدرس‌دهی باز: به المان‌ها اجازه می‌دهد در خانه‌ای بعد از، خانه‌ای که قرار سرریز رخ داده است ذخیره شوند.

درهم‌سازی عالی: زمانی اتفاق می‌افتد که یک تابع درهم‌سازی انتخاب شود که اصلاً برخورد به وجود نیآورد.



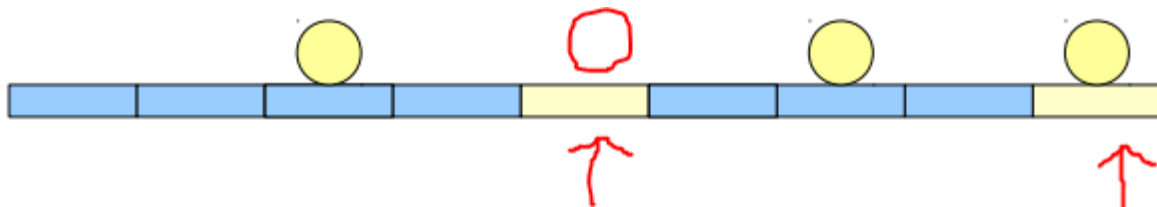
داده ساختار پیشرفته

Load factors: نسبت تعداد آیتم‌های ذخیره شده در آرایه‌ی هشینگ به کل آرایه است که نقش کلیدی در سرنوشت هشینگ دارد. اگر سایز آرایه و تعداد داده‌ها یکسان باشد می‌توان درهم‌سازی عالی داشته باشیم و مشکلی وجود ندارد. اما اگر سایز داده‌ها از آرایه بزرگتر باشد مشکل به وجود می‌آید.

تکنیک‌های برخورد با هشینگ (از دیدگاه دیگر) و بررسی مسئله‌ی برخورد

درهم‌سازی چند انتخابی یا Multy-Choice Hashing:

Second-choice hashing: فرض شود n توپ وجود دارد که می‌خواهیم این‌ها را در یکسری ظرف پخش کنیم. کافیهست دو خانه از آن‌ها را به صورت تصادفی انتخاب کنیم. توپ را در خانه‌ای قرار می‌دهیم که تعداد توپ کمتری داشته باشد. به این استراتژی تصادفی بودن می‌گویند.



قضیه: اگر استراتژی ما به صورت بالا باشد امید ریاضی مقدار تصادفی بیشترین تعداد توپی که در یک خانه قرار می‌گیرد از مرتبه‌ی $O(\log \log n)$ است. این استراتژی مربوط به درج است و مسئله‌ی جستجو نیز مطرح است. امید ریاضی ماکسیمم تعداد توپی که در یک خانه قرار می‌گیرد که برابر $O(\log \log n)$ است. $O(\log \log n)$ تحلیل سرشکنی یا پیچیدگی اجرایی هزینه‌ی جستجو را نشان می‌دهد. در اینجا هزینه‌ی جستجو در بدترین حالت بیشترین تعداد توپ‌هایی است در یک خانه قرار می‌گیرد.

این مسئله می‌تواند ادامه پیدا کند.

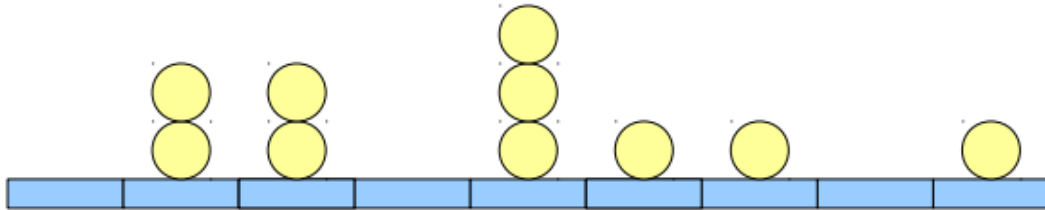
Expected value: Sum (احتمال وقوع * مقدار حالت)

$$E(x) = \sum(x \cdot P(x))$$



Second-Choice Hashing

- **Theorem:** The expected cost of a lookup in such a hash table is $O(1 + \alpha)$. (Why?)
- **Theorem:** Assuming truly random hash functions, the expected worst-case cost of a lookup in such a hash table is $O(\log \log n)$. (Why?)
- **Open problem:** What is the smallest k for which there are k -independent hash functions that match the bounds using truly random hash functions?





جلسه پنزدهم

در جلسه قبل روش Multi-choice hashing و Second-choice hashing معرفی شد. فرض کنید می‌خواهیم چند توپ را در چند ظرف قرار بدهیم. به طوریکه برای هر توپ دو ظرف در نظر گرفته می‌شود و سپس توپ در ظرفی قرار می‌گیرد که توپ کمتری دارد. در همین روش اثبات می‌شود امید ریاضی برای بیشترین تعداد توپ در هر محل از مرتبه $O(\log \log n)$ می‌شود. می‌خواهیم مسئله هشینگ را بهبود دهیم. در هشینگ مرتبه $O(\log n)$ قابل دسترسی است. می‌خواهیم این مرتبه را بهبود دهیم. این مرتبه باید بهتر شود به طوریکه در خوش‌بینانه‌ترین حالت به $O(1)$ برسد. اگر در مسئله هشینگ مرتبه $O(\log n)$ بهبود یابد این بهبود برای ما مناسب است. با مرتبط کردن مسئله توپ‌ها به مسئله هشینگ هر آیتم (توپ) به یک جا معطوف می‌شود. در حالت معمول نمی‌توان تصور کرد که در این مسئله برخورد رخ نمی‌دهد. از این رو داده‌هایی که تحت برخورد قرار می‌گیرند در باکت آرایه قرار می‌گیرند و جستجو درون باکت به صورت خطی انجام می‌شود. در بدترین حالت تعداد توپ‌های درون باکت از $O(\log \log n)$ است که مرتبه‌ی جستجو در بدترین حالت از $O(\log \log n)$ است که به صورت ترتیبی انجام می‌شود.

تکنیک دیگر Cuckoo hashing دو جدول درهم‌سازی ساخته می‌شود. دو تابع هش به نام‌های h_1 و h_2 در نظر گرفته می‌شود. زمانیکه می‌خواهیم برای آیتمی مکانی پیدا کنیم از h_1 یا تابع هش اول استفاده می‌شود تا مکانی برای آن عنصر پیدا شود. اگر مکان پیدا شده در جدول اول خالی بود این داده در آن مکان قرار می‌گیرد اما اگر آن مکان خالی نبود و عنصر جدید از لحاظ نزدیکی، نسبت به عنصری که در آن اسلات قرار دارد به این اسلات نزدیک‌تر باشد داده‌ای که در آن مکان قرار دارد برداشته شده و عنصر جدید در آن مکان قرار می‌گیرد و برای عنصر برداشته شده مکان جدیدی یافت می‌شود. مکان این عنصر از طریق تابع دوم هشینگ یا h_2 جایابی می‌شود.

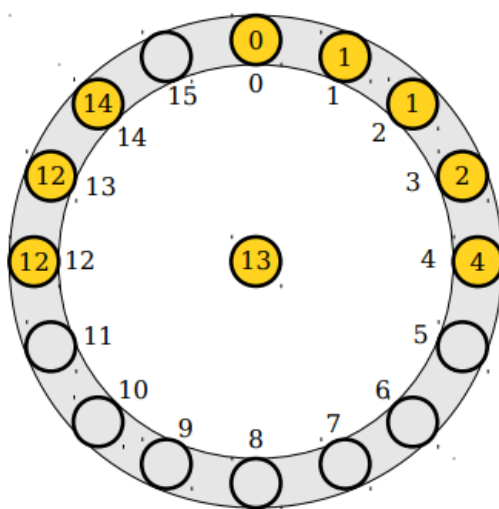
	97
32	
	26
84	
59	41
93	23
58	
	53

T_1 T_2

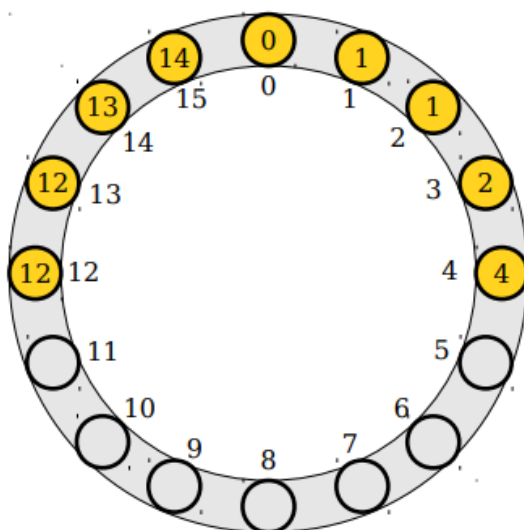


Hashing with relocation

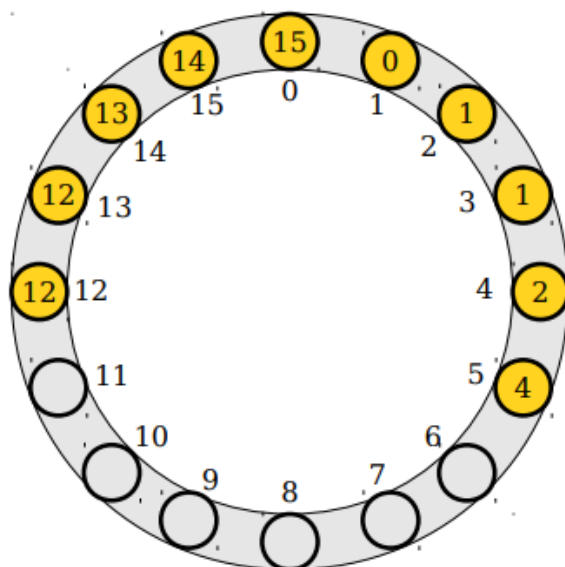
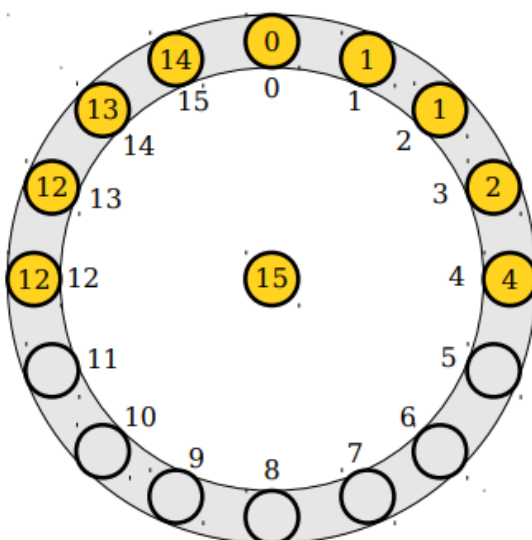
Rabin hood hashing: یک نوع هشینگ است که کلیدها می‌توانند پس از آنکه درون خانه‌ای قرار گرفتند جابه‌جا شوند. الگوریتم این جابه‌جایی به صورت زیر است: اگر یک کلید جدید بخواهد درج شود و مکانی را که تابع هشینگ نشان می‌دهد توسط یک کلید ثانویه اشغال شده باشد و کلید جدید به این مکان نسبت به کلید ثانویه نزدیک‌تر باشد باید کلید ثانویه این خانه را تخلیه کند و کلید جدید در این خانه قرار بگیرد. در واقع کلید جدید به نزدیک‌ترین محل نسبت به مکان اصلی خود قرار می‌گیرد. هدف اصلی این الگوریتم همین موضوع است. در واقع واریانس یا امید ریاضی این جابه‌جایی باید حداقل شود. همچنین کلید ثانویه باید به نزدیکترین مکان ممکن نسبت به مکانی که از قبل درون آن وجود داشت جابه‌جا شود.



Relocation: فرض شود یکسری آیت‌م به صورت شکل بالا موجود باشند. عنصر شماره‌ی ۱۳ باید در محل شماره‌ی ۱۳ قرار می‌گیرد. همانطور که مشاهده می‌شود مکان شماره‌ی ۱۳ توسط عنصر شماره‌ی ۱۲ پر شده است. بنابراین عنصر شماره‌ی ۱۳ باید در نزدیک‌ترین محل به خانه‌ی ۱۳ یعنی مکان شماره‌ی ۱۴ قرار بگیرد. این مکان، توسط عنصر ۱۴ پر شده است. عنصر شماره‌ی ۱۴ از این خانه برداشته شده و عنصر شماره‌ی ۱۳ جای آن را می‌گیرد. همچنین عنصر شماره‌ی ۱۴ به نزدیکترین خانه نسبت به خانه‌ی اولیه یعنی در خانه‌ی شماره‌ی ۱۵ قرار می‌گیرد. در نهایت تغییرات اعمال شده به صورت زیر است:



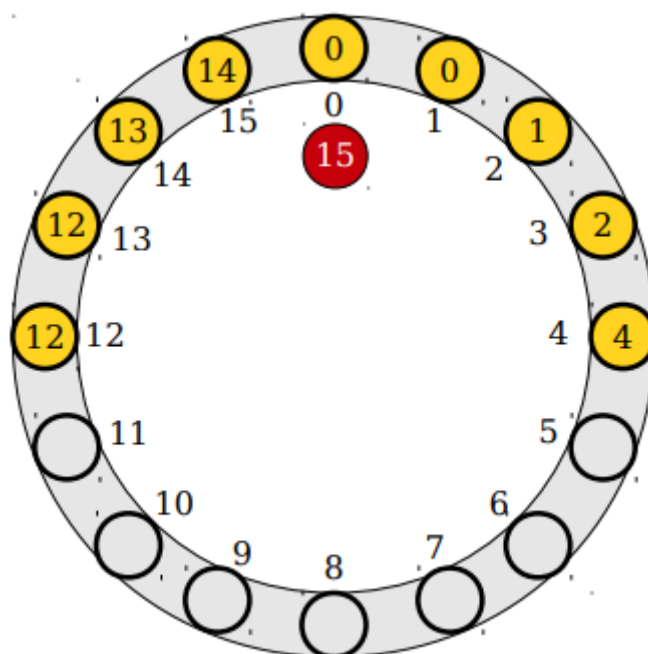
همچنین درج کلید ۱۵ به صورت زیر است (اسلاید شماره ۴۰):





داده ساختار پیشرفته

عملا هر کدام از این کلیدها با یک خانه جابه‌جا شده‌اند. در بالا سعی شده است که واریانس تغییرات بسیار کم باشد. به این دلیل که وقتی امید ریاضی (سرشکنی) حساب می‌شود هزینه‌ی سرشکن شده $O(1)$ شود. در واقع مرتبه‌ی جابه‌جایی از $O(1)$ است. در این الگوریتم مرتبه‌ی جستجو در کمترین زمان انجام می‌شود. فرض کنید می‌خواهیم ۱۵ را جستجو کنیم. فقط مکان اصلی شماره‌ی ۱۵ و خانه‌ی بعد از آن مورد جستجو قرار می‌گیرد. یعنی با دو جستجو نتیجه‌ی یک کوثری مشخص می‌شود. مرتبه‌ی جستجو از $O(1)$ است. هزینه‌ی جستجو به شیف‌ت کردن منتقل می‌شود. نکته‌ی دیگری که وجود دارد این است که این الگوریتم بسته به نیاز مسئله به کار می‌رود. در واقع بسته به کاربرد از یک مسئله به مسئله‌ی دیگر الگوریتم‌های ارائه شده می‌توانند تغییر کنند.



امید ریاضی در روش Robin hood hashing با توجه به عملیات جستجو $O(1)$ است. برای عملیات حذف به مشابه عمل جستجو، به طور میانگین دو مقایسه انجام می‌شود و در صورتی که عنصر مورد نظر یافت شود از داده ساختار حذف می‌شود. در n -dependent hashing اگر مقدار $n=5$ باشد به مرتبه $O(1)$ در بدترین حالت می‌رسیم. در الگوریتم مورد بررسی $n=2$ است.

قضیه: اگر هش فانکشن تصادفی باشد واریانس امید ریاضی تعداد جابه‌جایی‌های تابع هش $O(\log \log n)$ است.

کوکوهشینگ در مسائل بلاک‌چین استفاده می‌شود.

اکنون دو ایده وجود دارد

۱- Second-choice hashing: از دو تابع هش برای مکان‌یابی استفاده می‌شود.



۲- Relocation hashing: در نزدیک ترین مکان به محل خودش قرار می دهیم.

با استفاده از دو ایده‌ی بالا کوکوهشینگ تشریح می‌شود. هر کدام از روش‌های بالا سعی کردند هزینه‌ی جستجو را به سهم خود کاهش بدهند. سعی کردند واریانس جستجو را در توزیع المان‌ها پایین بیاورند. هر دو روش بالا می‌خواهند واریانس تغییر را کم کنند. علت نام‌گذاری این روش به نام کوکوهشینگ رفتار کوکوها است. کوکو تخم خود را در لانه‌ی پرنده‌ی دیگر قرار می‌دهد. حال پرنده‌ی ثانویه به دلیل ازدیاد تخم‌ها بعضی از آن‌ها را از لانه‌ی خود بیرون می‌ریزد. اگر پرنده بتواند تخم کوکو را تشخیص دهد آن را به بیرون از لانه پرت می‌کند در غیر اینصورت برخی از تخم‌ها به تصادف به بیرون از لانه پرت می‌شوند. در واقع کوکو انتظار دارد پرنده‌ی دیگری آن را نگه داری کند.

در این روش دو جدول و دو تابع درهم‌سازی در نظر می‌گیرند. جداول هم‌اندازه‌ی هم هستند. دو تابع h_1 و h_2 در نظر گرفته می‌شود. که این تابع‌ها یک کلید را می‌گیرند و آن کلید را به بازه‌ی $[1...m]$ نگاشت می‌شود. برخی توابع هش عملیات نگاشت را به خوبی انجام می‌دهند ولی هزینه‌ی محاسباتی زیادی دارند و بلعکس. انتخاب دو تابع h_1 و h_2 باید به صورتی باشد که یک توازن بین هزینه و عملیات نگاشت رخ دهد. تا جایی که امکان دارد توابعی انتخاب می‌شود که کلیدها به صورت عادلانه و تصادفی پخش شوند. h_1 مربوط به جدول T_1 و h_2 مربوط به T_2 است و عملیات نگاشت انجام می‌شود. هر المان می‌تواند به صورت زیر باشد:

- Maintain two tables, each of which has m elements.
- We choose two hash functions h_1 and h_2 from $\mathcal{U}$ to $[m]$.
- Every element $x \in \mathcal{U}$ will either be at position $h_1(x)$ in the first table or $h_2(x)$ in the second.

	97
32	
	26
84	
59	41
93	23
58	
	53

T_1
 T_2

در هنگام جستجوی المانی مانند x اندیس h_1 از جدول T_1 و اندیس h_2 از جدول h_2 بررسی می‌شود. اگر عنصر مورد نظر در هر یک از المان‌های پیشنهادی توسط h_1 و h_2 قرار داشت مقدار True و در غیر اینصورت



داده ساختار پیشرفته

مقدار False برگردانده می‌شود. بنابراین با دو جستجو می‌توان نتیجه‌ی یک کوئری را مشخص کرد. در کوکوهشینگ نتیجه‌ی عملیات جستجو از مرتبه‌ی $O(1)$ است.

اگر عنصری که قرار است اضافه شود x در نظر گرفته شود ابتدا سعی می‌شود که این x در جدول شماره‌ی 1 درج شود. بنابراین از $h1$ برای پیدا کردن ایندکس مناسب در جدول $T1$ استفاده می‌شود. اگر این خانه خالی باشد x درون آن قرار می‌گیرد در غیراینصورت کلید قبلی خانه‌ی اشغال شده خارج شده و x درون این خانه قرار می‌گیرد و سپس برای کلید خارج شده از جدول $T2$ با استفاده از تابع هش $h2$ مکان مناسب پیدا می‌شود. مثال : دو جدول $T1$ و $T2$ به صورت پیش‌فرض در نظر گرفته می‌شوند:

32	97
	26
84	
59	41
93	23
58	
	53

T_1 T_2

در گام اول عنصر شماره‌ی 10 درون این جدول درج می‌شود. پس از درج جداول $T1$ و $T2$ به صورت زیر هستند.

	97
32	
	26
84	
75	93
10	23
58	
	53

T_1 T_2



داده ساختار پیشرفته

در گام بعدی با درج عنصر 6 تغییرات دو جدول T_1 و T_2 به صورت زیر است.

	97
53	
	26
6	32
75	93
10	23
58	
	84

T_1 T_2

مشکل این روش این است که امکان Loop یا چرخش دارد یعنی چرخه ادامه دارد. در اینجا باید تابع هش عوض شود که به آن Rehash گفته می شود. به این روش در مجموع Cuckoo Hash گفته می شود. شکل زیر یک نمونه از Loop را نشان می دهد.

	97
53	
	26
6	32
75	93
10	23
58	
	84

T_1 T_2

91



داده ساختار پیشرفته

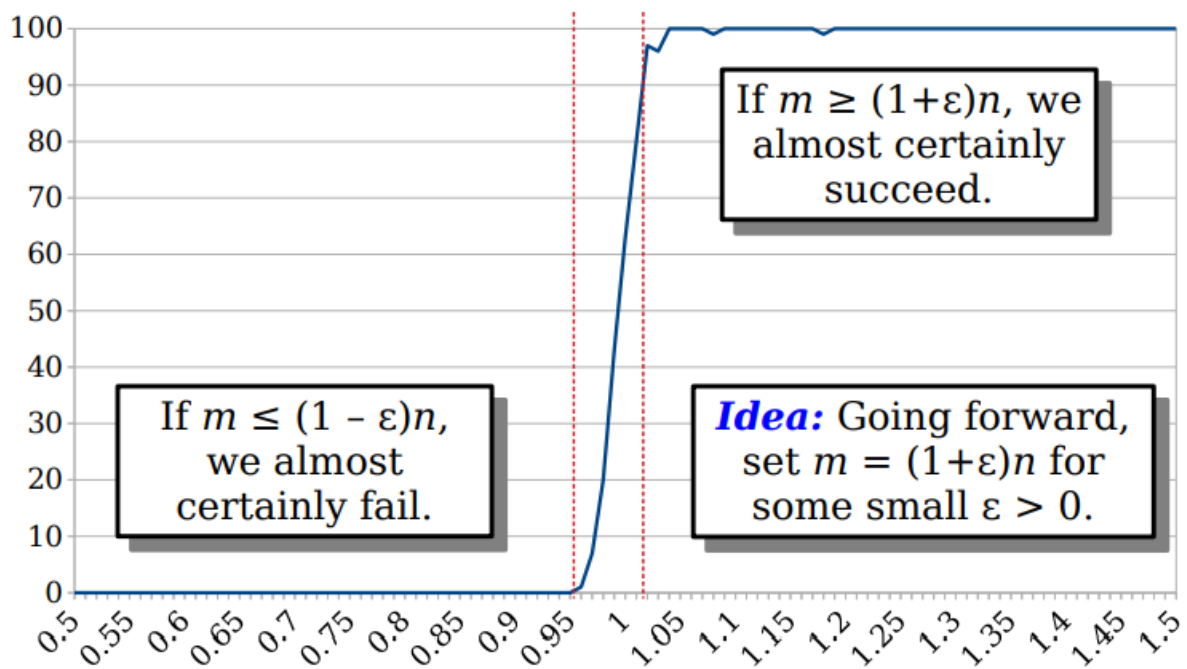
برای ممانعت از Loop یا چرخه سائز هر آرایه باید به اندازه‌ی نصف آیتم‌های باشد. اگر load factor به کمتر از نیم برسد و توابع هش مناسب انتخاب شود احتمال اینکه در لوپ بیفتد کمتر است. راه دیگر برای تشخیص این مشکل که آیا لوپ داریم یا خیر این است که این فرآیند به شکل یک گراف دیده شود. گره‌های گراف برابر با المان‌های هر آرایه هستند و جابه‌جایی نودها به صورت یال دیده می‌شود. سپس بر روی لوپ (Loop) در گراف بحث می‌شود.

اگر لوپ در Rehashing اتفاق بیفتد و درج‌های ما با شکست مواجه شوند باید h_1 و h_2 تغییر کند و عملیات درج از اول انجام شود.

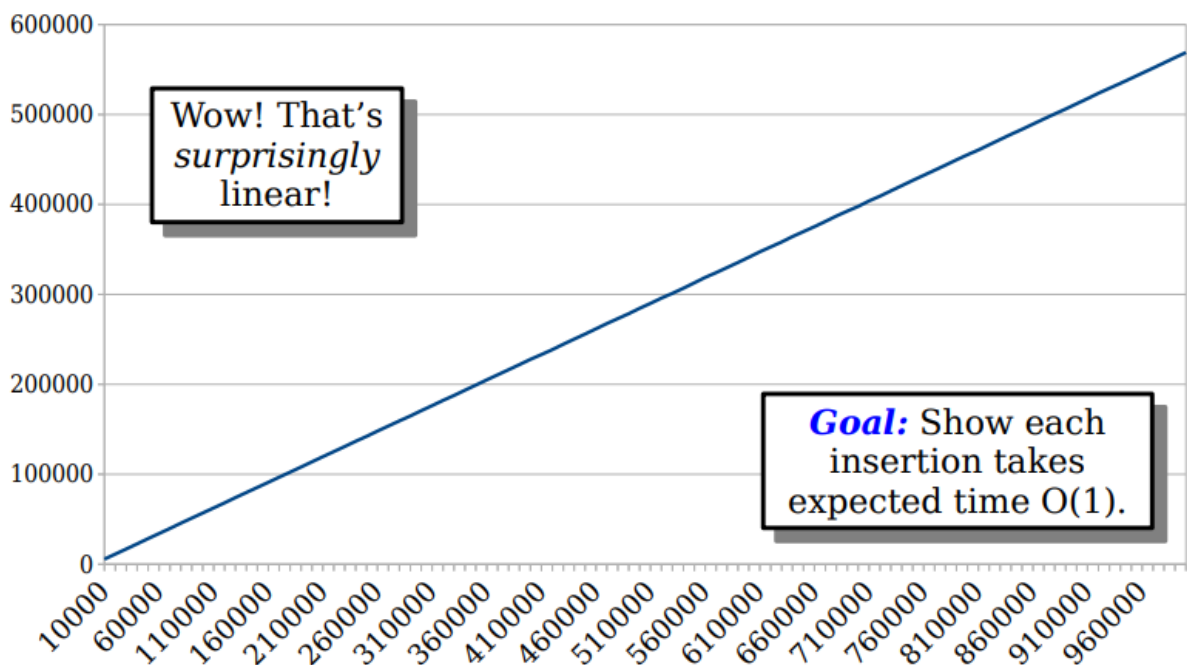
وقتی می‌خواهیم داده ساختاری را تحلیل کنیم خوب است قبل از آن کاری انجام شود. تعداد زیادی حذف، اضافه و به‌روزرسانی با یک مجموعه داده انجام شود تا ببینیم چه اتفاقی می‌افتد. این کار انجام شده که دو نتیجه بدست آمده که در مورد آن بحث می‌شود:

سوال اول: می‌خواهیم بدانیم اگر n عنصر و دو جدول داشته باشیم و هر جدول m اسلات داشته باشد درصد درج موفق چقدر است؟

برای اندازه‌گیری احتمال باید رابطه‌ای بین m و n مفروض باشد. بسته به رابطه‌ی بین m و n این احتمال متفاوت است. فرض می‌شود $m = \alpha n$ باشد. بر روی α یک بررسی انجام شده است. محور عمودی میزان درج و محور افقی load factory را نشان می‌دهد. در وسط نمودار $m = n$ است. در عمل نشان داده شده است که اگر $m \leq (1 - \epsilon)n$ باشد احتمال درج موفق کمتر از یک است در واقع در هنگام درج با شکست مواجه می‌شویم. اگر $m \geq (1 + \epsilon)n$ باشد احتمال درج موفق بیشتر از یک است. می‌خواهیم از هر دو جدول به صورت بالانس شده استفاده کنیم و فرض می‌کنیم همیشه نصف جداول خالی هستند. بنابراین احتمال موفقیت‌آمیز بودن درج‌ها بیشتر و بیشتر می‌شود. به طوریکه اگر مقدار m به ازای هر جدول به صورت $m = 1.5n$ باشد احتمال موفقیت‌آمیز بودن درج‌ها یک می‌شود (برای دو جدول $m = 3n$ می‌شود).



در جدول زیر محور افقی مقدار درج‌ها و محور عمودی مقدار جابه‌جایی‌ها را نشان می‌دهد. در عملی نشان داده شده است که مرتبه‌ی این عمل خطی است. که این آزمایش نیز، آزمایش بسیار خوبی است. در این جدول به ازای 110000 درج 70000 جابه‌جایی داریم.



چالش‌های پیش‌رو:

۱- استفاده از توابع هشینگ مختلف به دلیل ایجاد لوپ (لوپ‌ها به دلیل برخورد اتفاق می‌افتند)

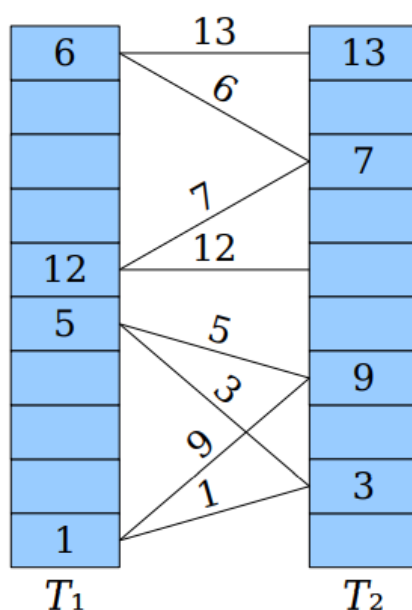


داده ساختار پیشرفته

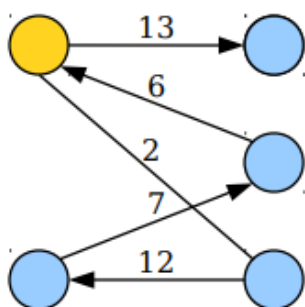
۲- احتیاج به استنتاج زنجیره‌ی جابه‌جایی داریم. در واقع بعد از چند جابه‌جایی به لوپ می‌رسیم؟ آیا زنجیره باید ادامه داده شود.

برای این جابه‌جایی از Cuckoo graph استفاده می‌شود.

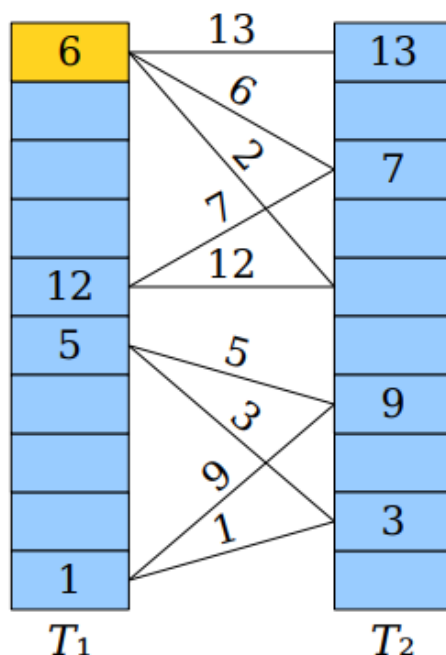
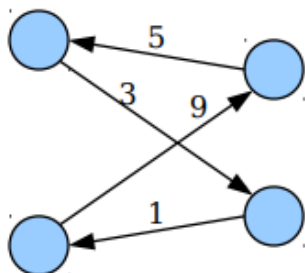
مسئله: می‌خواهیم بدانیم با تنظیمات اولیه آیا درون لوپ گیر می‌کنیم یا خیر؟ در اینجا از کوکوگراف استفاده می‌شود. هر کدام از سلول‌های آرایه یک گره در گراف را نشان می‌دهد. این شکل یک گراف دوبخشی را نشان می‌دهد. گراف دو بخش گره‌ها را به دو دسته تقسیم می‌کند به طوریکه بین گره‌های یک دسته در گراف دو بخشی رابطه‌ای وجود ندارد. یال‌ها بین گره‌های دو دسته برقرار می‌شوند.



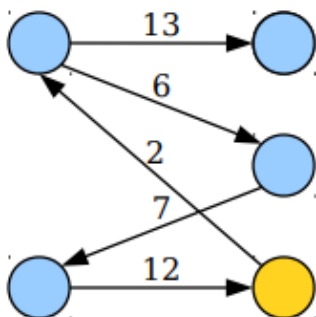
برای شبیه‌سازی گراف، سلول‌های هر آرایه گراف هستند و یال‌ها دو محلی را به هم وصل می‌کنند که یک آیتم می‌تواند در آن ذخیره شود. به عبارتی مکان‌های بالقوه‌ی یک عنصر در دو جدول با یکدیگر لینک می‌شوند. جهت یال‌ها به سمت مکان عنصر مورد نظر در نود مقصد است. به عنوان مثال شکل زیر مکان بالقوه‌ی 6 را از طریق لینک به یکدیگر متصل می‌کند. همچنین جهت یال، مکان ذخیره‌سازی عنصر 6 را نشان می‌دهد.



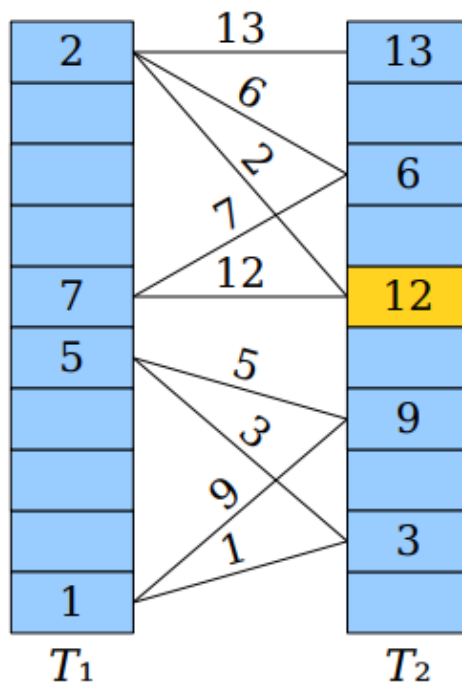
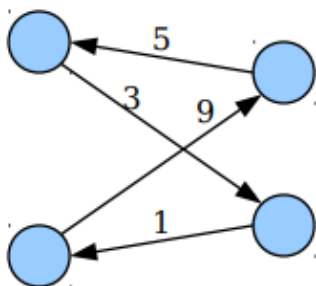
Insertions
correspond to
sequences of
flipping
arrowheads.



همچنین شکل زیر نتیجه‌ی درج عدد 2 را نشان می‌دهد:



Insertions
correspond to
sequences of
flipping
arrowheads.



در اینجا تحلیل گراف با توجه به موجود بودن دور در گراف بررسی می‌شود. اگر گراف به صورت یکسری جنگل باشد (درخت‌های مجزا) بدون مشکل عمل درج انجام می‌شود. اگر درخت نباشد و یک دور وجود داشته باشد می‌توانیم درج موفق انجام دهیم اما اگر بیش از یک دور داشته باشیم نمی‌توانیم درج انجام دهیم. موجودیت دور در کانکت کامپوننت اتفاق می‌افتد. اگر دور ایجاد شد بهتر است Rehashing انجام دهیم. در اینجا با ایجاد گراف و شمارش دورها به راحتی می‌توانیم تشخیص دهیم که آیا زنجیره‌ها در حلقه می‌افتند یا خیر؟



جلسه شانزدهم

زمانیکه یک درج انجام می‌شود یک زنجیره‌ای از تغییرات اتفاق می‌افتد که این زنجیره تغییرات می‌تواند با موفقیت اعمال شود و یا ممکن است در یک لوپ بیفتد. کارایی عمل درج بسته به زنجیره‌ی تغییرات است که وجود دارد. دو چالش اصلی که در این بخش وجود داشتند عبارتند از:

۱- تحلیل زنجیره‌ی تغییرات

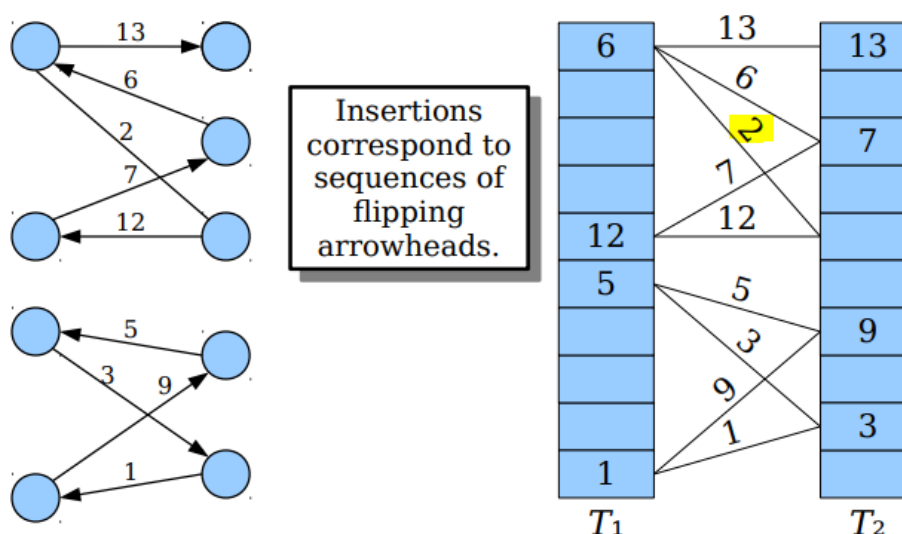
۲- اگر در یک حلقه بیفتیم باید توابع هش را عوض کنیم.

همچنین از یک گراف برای تحلیل زنجیره تغییرات استفاده می‌شود. کوکوگراف، گرافی است که در این بخش مورد استفاده قرار می‌گیرد. کوکوگراف یک مالتی گراف دوبخشی است که نودهای آن سلول‌های آرایه‌های هش هستند و یال‌ها ارتباط سلول‌های جداول هستند. عدد روی هر یال نشان‌دهنده‌ی کلیدی است که در یکی از نودهای مبدا یا مقصد ذخیره شده است و جهت یال نشان‌دهنده نودی است که کلید آن یال در آن ذخیره شده است.

می‌خواهیم از خواص گراف برای استنتاج زنجیره استفاده کنیم. حال می‌خواهیم نحوه‌ی استفاده از کوکوهشینگ را بررسی کنیم. یکسری از ویژگی‌های کوکوگراف عبارتند از:

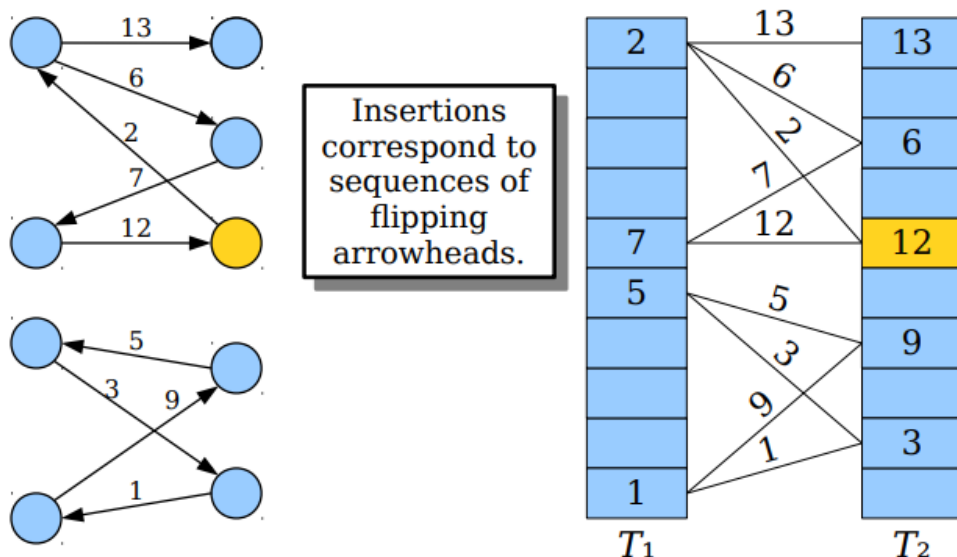
- نوک هر پیکان اشاره به المان ذخیره شده در یک اسلات دارد.
- در هر نود بیش از یک المان قرار نمی‌گیرد. پس به هر نود فقط و فقط یک یال وارد می‌شود.
- درج‌ها مجموعه‌ای از تغییرات در جهت فلش‌ها است. زیرا زمانیکه عنصری در گراف جایابی می‌شود عنصر قبلی جابه‌جا می‌شود و جهت فلش عوض می‌شود. در واقع زنجیره‌ای از فلیپ‌ها داریم.

گراف زیر را در نظر بگیرید.





می‌خواهیم عدد 2 را در آن ذخیره نماییم. نتیجه‌ی نهایی پس از درج عدد 2 به صورت زیر است:



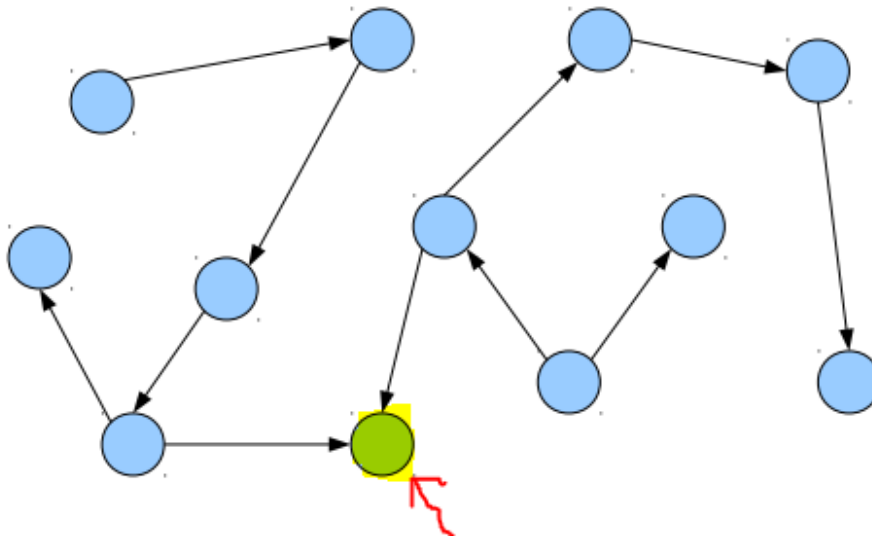
در مثال بالا مکان‌های 2 و 6 جابه‌جا می‌شوند. سپس مکان‌های 7 و 6 جابه‌جا می‌شوند. در نهایت 12 جابه‌جایی می‌شود. در مثال بالا گراف ما دو کامپوننت گراف مستقل دارد که آن‌ها را به اختصار با CC نمایش می‌دهند که ویژگی‌های هر کدام از آن‌ها باید به طور مستقل مورد بحث قرار بگیرد.

ادعا 1: اگر کلید x در جدول درهم‌سازی کوکو درج شود این درج موفقیت‌آمیز خواهد بود اگر کامپوننت گراف ایجاد شده دور نداشته باشد یا فقط یک دور داشته باشد (گرافی که بیش از یک سیکل داشته باشد کامپوننت پیچیده متصل نامیده می‌شود). اگر چنین کامپوننت گراف وجود داشته باشد و عملیات درج اتفاق بیفتد عملیات درج موفقیت‌آمیز است. اگر یک کامپوننت پیچیده نداشته باشیم درج موفقیت‌آمیز خواهد بود.

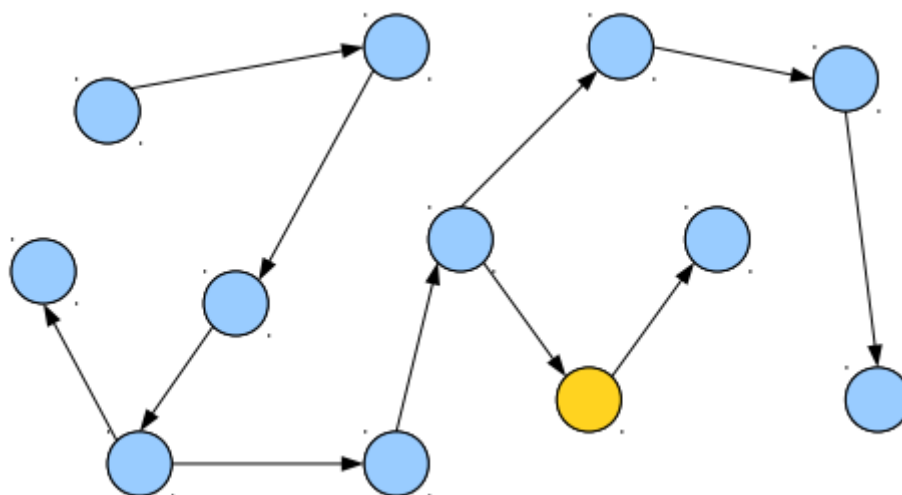
شکل زیر را در نظر بگیرید:



داده ساختار پیشرفته



نود نشان داده شده در شکل بالا به گراف اضافه می شود. بعد از اضافه شدن و انجام تغییرات، گراف به صورت زیر ظاهر می شود.

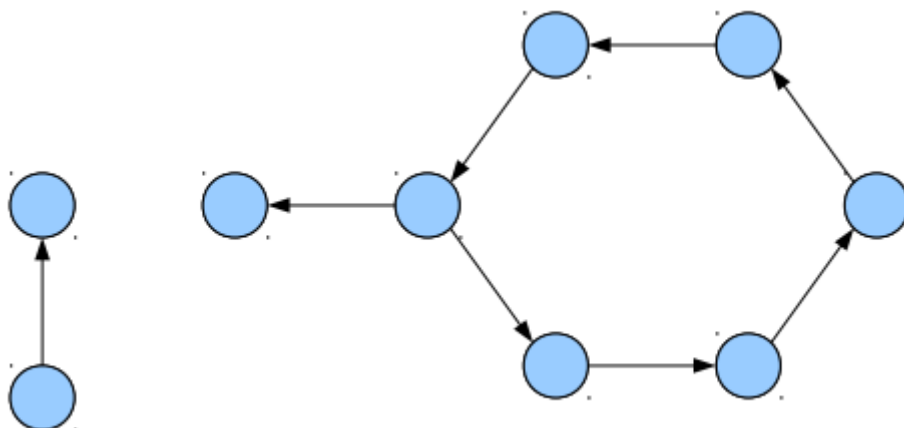


مثال بالا مربوط به زمانی است که اگر لوپ وجود نداشته باشد و یک نود به گراف اضافه شود این درج با موفقیت انجام می شود.

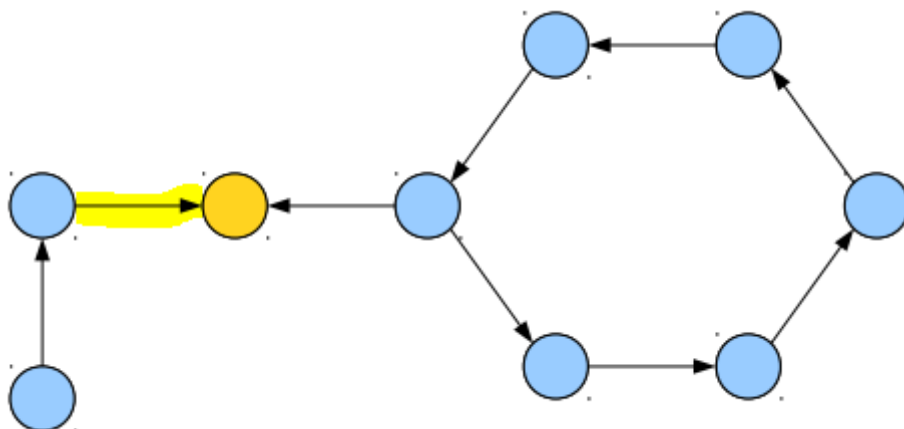
مثال زیر مربوط به حالت دوم است. یعنی یک گراف وجود داشته باشد و در این گراف یک دور وجود داشته باشد. شکل زیر را در نظر بگیرید.



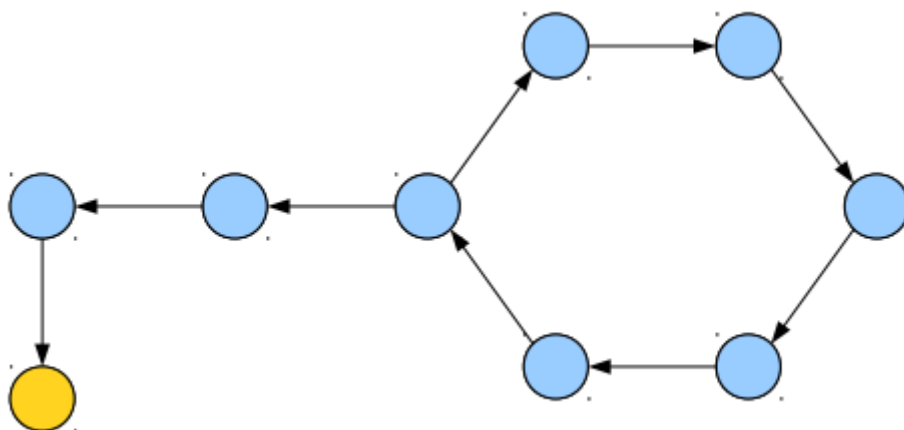
داده ساختار پیشرفته



فرض کنید عنصری مانند عنصر زیر به این گراف اضافه شود:



با تغییر مکان هر عنصر در نهایت شکل زیر بدست می آید:



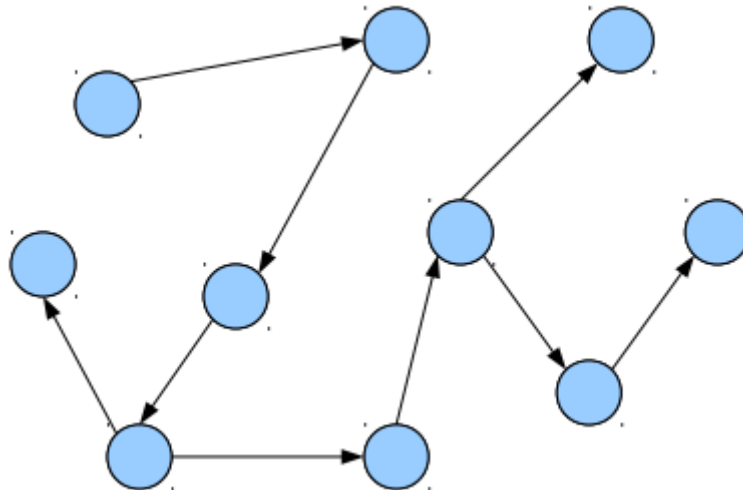
به طور کلی زمانی که درج انجام می شود درون یک سیکل گیر می افتیم یا از شر آن سیکل خلاص می شویم و یا از آن سیکل خارج می شود..



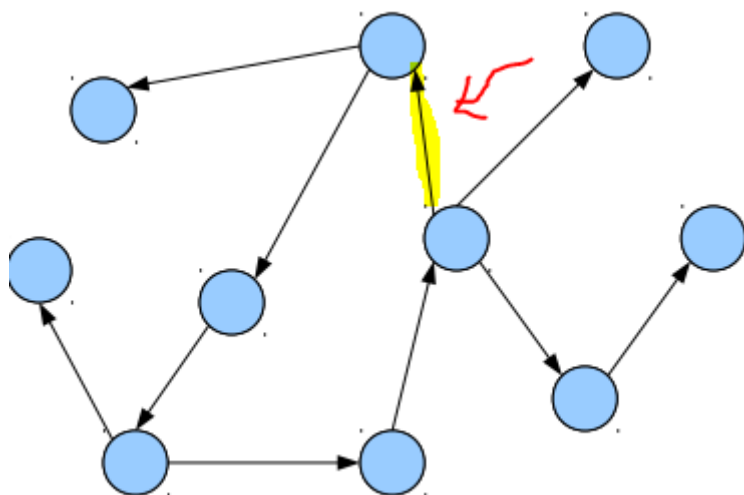
داده ساختار پیشرفته

در ادعای دوم بیان می‌شود اگر X در جدول کوکوه‌شینگ درج شود آن درج با شکست مواجه می‌شود اگر کامپننت کانکتی که حاوی X است بیش از یک دور داشته باشد.

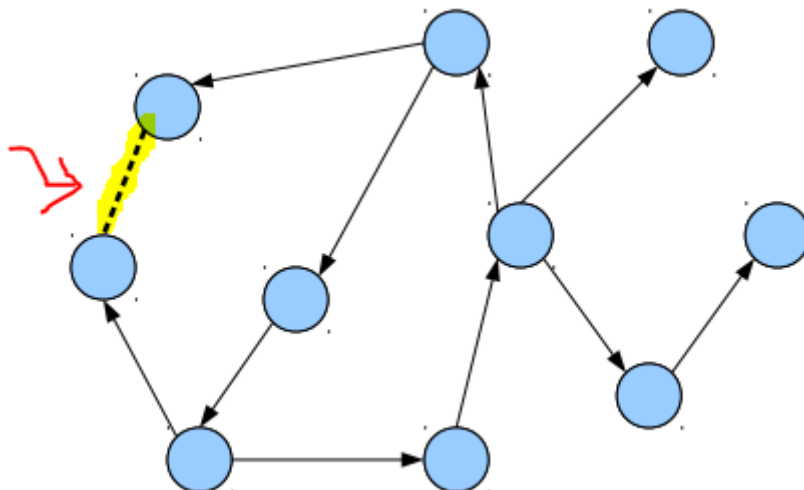
مثال: در گراف زیر هیچ دوری درون گراف وجود ندارد.



سپس عنصری به صورت زیر در گراف اضافه می‌شود و یک سیکل ایجاد می‌شود. از آنجا که در این گراف فقط یک دور وجود دارد مشکلی پیش نمی‌آید.



حال اگر مطابق شکل زیر عنصر دیگری به گراف اضافه شود باعث ایجاد سیکل در گراف می‌شود.



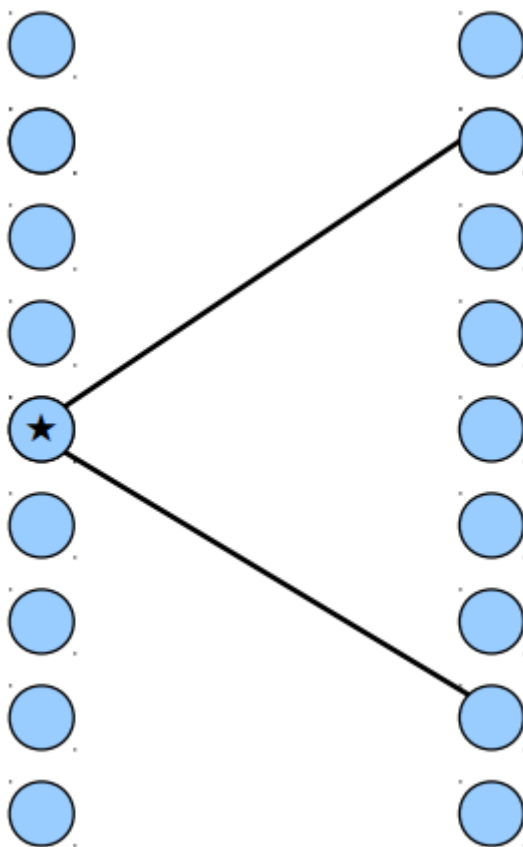
ادعای ۲: اگر یک درج در جدول کوکو وجود داشته باشد با شکست مواجه می‌شود اگر بیش از یک دور وجود داشته باشد.

وقتی یک گراف بدون سیکل داریم گراف k نود و $k-1$ یال دارد. زمانی که یک دور داریم k نود و k یال داریم. اگر k نود و $k+1$ یال داشته باشیم بیش از یک دور داریم.

می‌خواهیم در مورد اندازه‌ی کانکت کامپوننت صحبت کنیم. مثلاً اگر n گره داشته باشیم چه سهمی از گره‌ها به هر کامپوننت تخصیص داده می‌شود و آیا این می‌تواند به اندازه‌ای بزرگ باشد که با بزرگ شدن طول زنجیره، کوکوهشینگ با شکست مواجه شود. همچنین می‌خواهیم بدانیم چقدر شانس شکست زنجیره‌ها در عمل درج وجود دارد.

گام اول: لازم است تخمینی از اندازه کامپوننت کانکت‌ها داشته باشیم. همه‌ی این‌ها به صورت امید ریاضی گفته می‌شود. استفاده از امید ریاضی بیان‌کننده‌ی یک احتمال از اندازه‌ی گراف است. در واقع تخمینی از کانکت کامپوننت‌ها زده می‌شود.

برای تخمین سائز کانکت کامپوننت‌ها (تخمین اندازه‌ی نودها) در یک کوکوگراف به اینصورت عمل می‌کنیم: باید یکی از گره‌ها به تصادف انتخاب شوند سپس یک BFS شبیه‌سازی شود. یعنی می‌خواهیم یک درخت از یال‌ها ایجاد کنیم. این گره‌ی اولیه ریشه‌ی زیرگراف از گراف متصل است. می‌خواهیم بزرگترین زیرگراف از کوکوگراف را تخمین بزنیم و بگوییم گراف از این زیرگراف بزرگتر است. یک اسلات از جدول در نظر گرفته می‌شود. n قلم داده درون جدول وارد شده است بنابراین برداشت می‌شود n یال در گراف داریم. فرض دیگر این است که جداول توابع درهم‌سازی h_1 و h_2 به صورت تصادفی عمل می‌کنند به طوریکه احتمال تخصیص هر گره به هر اسلات مقدار ثابتی است. فرض کنید m گره (اسلات) سمت چپ و m گره (اسلات) سمت راست قرار دارند. احتمال انتخاب یک نود در سمت چپ گراف به عنوان ریشه‌ی درخت توسط هر لبه $1/m$ است. در واقع تمام سمت راستی‌ها با شانس یکسان $1/m$ به گره‌ی ریشه متصل می‌شوند.



همانطور که در شکل بالا مشاهده می شود با توجه به اینکه احتمال تشکیل دو یال شکل گرفته (نشان دهنده ی اعداد درج شده) بیشتر از $1/m$ است این دو یال ایجاد شده اند. ادامه ی درختی نیز به همین صورت ساخته می شود. گره ی سمت راست با احتمال $1/(m-1)$ به گره ی سمت چپ متصل می شوند. این زیرگراف با عملیات متناوب ساخته می شود.

۳۶ دقیقه - مثال به روی تابلو کشیده شده است.

استدلال می شود که در هر انتخاب احتمال باینومیال با فاکتور یک داریم. با توجه به باینومیال بودن درخت و احتمال شکل گیری هر یال $(1/m, 1/(m+1))$ بالتسون و والتسون ثابت کردند امید ریاضی تعداد گره های درخت در سطح های مختلف آن برابر n/m است. به عبارت دیگر مرتبه ی سرشکنی عملیات درج شده برابر $O(1)$ است. به طور کلی سه عمل اصلی کوکوهشینگ و مرتبه ی اجرایی آن به صورت زیر است:

۱- جستجو: هزینه ی آن $O(1)$ است.

۲- حذف: هزینه ی آن از $O(1)$ است.

۳- درج: هزینه ی آن به صورت امید ریاضی برابر $O(1)$ است.

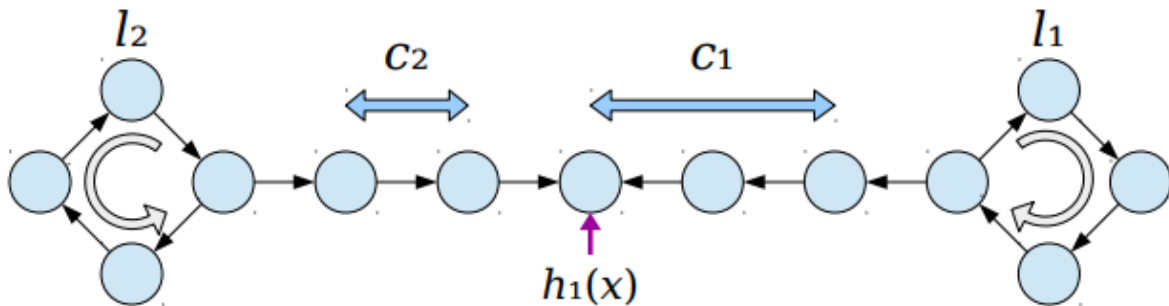
اگر در کاربردی عملیات بازیابی برای ما مهم باشد کوکوهشینگ برای ما مناسب است. زیرا هزینه ی جستجو $O(1)$ است.



داده ساختار پیشرفته

تعمق در ساختار گراف:

زمانیکه کانکت کامپوننت‌های یک گراف پیچیده نباشند عملیات درج با موفقیت انجام می‌شود. زمان درج به سبب کانکت کامپوننت وابسته است که به طور سرشکن شده از مرتبه‌ی $O(1)$ است. سوالی که پیش می‌آید این است که در چند درصد مواقع کانکت کامپوننت‌های پیچیده نداریم؟ به طور کلی زمانی یک کانکت کامپوننت پیچیده وجود دارد که شکلی مانند شکل زیر وجود داشته باشد.



در شکل بالا، همانطور که مشاهده می‌شود بیش از دو حلقه وجود دارد. بنابراین وجود دو حلقه باعث گیرافتادن در آن می‌شود. بدتر از این شکل نیز وجود دارد. زمانیکه بیش از دو حلقه وجود دارد. هزینه‌های یک عمل درج به چند بخش تقسیم می‌شود. این هزینه‌ها به ترتیب برابر مجموع چهار هزینه‌ی C_1 ، C_2 ، l_1 و l_2 است. در اثبات این بخش ثابت شده است که

- ۱- امید ریاضی درج موفق برابر با $O(1)$ است.
- ۲- زمانیکه عملیات درج به صورت ناموافق انجام می‌شود و مجبور می‌شویم Rehash انجام دهیم امید ریاضی آن برابر $O(1/(m^2))$ است.
- ۳- زمانیکه یک Rehash انجام می‌دهیم و مشکل حل می‌شود مرتبه‌ی اجرای عملیات به صورت $O(m)$ است. شکل زیر این موضوع را نشان می‌دهد:

$$O(1) + O(1 / m^2) \cdot O(m)$$

$O(1)$

Expected cost of successful insertion.

$O(1 / m^2)$

Expected number of rehashes.

$O(m)$

Cost of doing one rehash.

یکی از فرض‌هایی که در اینجا انجام می‌شود این است که در هر اسلات یک خانه وجود دارد می‌توان گفت یکی از کارهای آینده این است که هر خانه می‌تواند یک داده ساختار باشد. یکی از صحبت‌هایی که می‌شود



داده ساختار پیشرفته

محدود کردن حرکت و طول زنجیره‌ها است. کفایت یک آستانه در نظر گرفته شود و اگر این تغییرات بیش از حد آستانه باشد تغییرات متوقف شوند این داده درون داده ساختاری اضافه شود که داده‌های سرریز را نگهداری می‌کند. در نهایت زمانیکه جستجو انجام می‌شود این داده ساختار سرریز نیز در نظر گرفته می‌شود.

مبحث هشینگ:

تابع هش: یکسری کلید از مجموعه‌ی جهانی U را به یکسری اسلات در جدولی با اندازه‌ی m نگاشت می‌کند.

در اسلایدهای MIT یکسری تعریف وجود دارد. برخی از این‌ها عبارتند از:

تعریف اول: تا درهم‌سازی نگاشتی از داده‌ها به خانه‌های آرایه است به طوریکه:

$$h:\{0,1,\dots,U-1\} \rightarrow \{0,1,\dots,m-1\}$$

دسته‌های مختلفی از هش فانکشن‌ها وجود دارد که عبارتند از:

۱- تابع درهم‌سازی کاملاً تصادفی: تابعی است که به ازای همه‌ی کلیدهایی که در مجموعه‌ی مرجع

وجود دارد و کلیدها مستقل هستند با احتمال یکسانی کلیدها به یک اسلات نگاشت می‌شوند. در واقع احتمال نگاشت یک کلید به همه‌ی خانه‌ها برابر است.

تعریف استاد: شانس نگاشت شدن همه‌ی کلیدها به عناصر مختلف مجموعه‌ی مرجع جدول درهم‌سازی یکسان است.

۲- توابع درهم‌سازی سراسری: تابعی که با احتمال یکسان کلیدهای مختلف را به یک مکان نگاشت

می‌کند. در اینجا برخورد در نظر گرفته می‌شود. در واقع احتمال برخورد برای زوج کلیدها یکسان است و مرتبه‌ی آن از $O(1/m)$ است.

تابع h با احتمال یکسان کلیدها را به خروجی‌های ممکن نگاشت می‌کند. به عبارتی احتمال وقوع برخورد در هر یک از m محل موجود برای نگاشت شدن کلیدها، یکسان است.

۳- خانواده‌ی توابع درهم‌سازی قوی: فرض می‌شود حتماً برخورد رخ می‌دهد ولی طوری تابعی هش

تعریف می‌شود که مرتبه‌ی برخورد آن کوچکتر از $O(1/m)$ است.

$$h(x) = \left[\left(\sum_{i=0}^{k-1} a_i x^i \right) \bmod p \right] \bmod m.$$

مثال: $[ax \bmod p] \bmod m$ for $0 < a < p$: برای آنکه برخورد کمتر از $1/m$ باشد باید p مناسب

انتخاب شود. نشان داده می‌شود که این تابع بسیار قوی است. اکثر توابع درهم‌سازی به این شکل هستند. تعیین p و a یک چالش است. اگر بخواهیم این را قوی‌تر کنیم باید $(x \bmod p) \bmod m$



داده ساختار پیشرفته

انجام دهیم. همچنین می‌توان یک ضریب برای x باید در نظر گرفته شود به طوریکه بین a و p یک رابطه برقرار باشد.

تابع درهم‌سازی قوی: $h(x) = (a, x) \gg (\log u - \log m)$

۴- K -wise Independent (توابع کاتایی مستقل): اگر برای هر کدام از توابع هش، احتمال اینکه این تابع هش کلیدی را به مقدار مشخص نگاشت کند و کلید دیگری را به مقدار دیگر نگاشت کند و ... مرتبه‌ی نگاشت کلیدها به یک مقدار مشخص از $O(1/m^2)$ است. در واقع یک نوع رابطه‌ی داخلی بین کلیدها را در نظر می‌گیرد. اگر $k=2$ یا Pairwise independent داشته باشیم به صورت $h(x) = [(ax+b) \bmod p]$ است. در واقع نگاشت کلیدها به یکدیگر را در نظر می‌گیرد. احتمال تخصیص دو کلید متمایز به دو اسلات مجزا از مرتبه‌ی $O(1/m^2)$ است. به عنوان نتیجه در اینجا b اضافه شده است.

درهم‌سازی Prefect یا FKS Perfect Hashing: ایده‌ی آن به صورت دو لایه‌ای انجام می‌شود. روش ما به این صورت است که در دو لایه درهم‌سازی انجام شود. در لایه‌ی اول درهم‌سازی و یک تنظیمات برای آن در نظر گرفته می‌شود. تنظیمات می‌توانند شامل سایز و تابع هش و ... است. در لایه‌ی اول برخورد را می‌پذیریم و یک مرحله نگاشت انجام می‌شود. سپس هر کدام از اسلات‌ها به یک جدول درهم‌سازی وصل باشد به طوریکه کلیدهای دارای برخورد در آن جدول هش ذخیره هستند و در جدول دوم باید مطمئن باشیم که برخورد رخ نمی‌دهد.



جلسه هفدهم

گاهی داده‌ها پویا هستند. یعنی داده‌ها به صورت استریم وارد می‌شوند روش‌های قبل مناسب نیستند. زیرا داده‌های استریم آمار داده‌ها را به طور مرتب تغییر می‌دهد. حال فرض می‌کنیم یک روش پیر هزینہ داریم که به ما جواب دقیق می‌دهد اما آنقدر دفعات تکرار بالا باشد که کوئری گرفتن غیر ممکن می‌شود. در اینجا قبول می‌کنیم داده‌ها دقیق نیستند و داده‌ها تقریب زده می‌شوند. بنابراین داده ساختارهای احتمالاتی معرفی شدند.

این داده ساختارها کاربردهای زیادی در کلان داده و مسائلی که داده‌ی استریمینگ دارند ایجاد کرده‌اند. انواع مختلفی از داده ساختارهای احتمالاتی وجود دارد که عبارتند از: Count-Min Sketch, Bloom Filter و

...

Bloom Filter: بلوم فیلتر در سال ۱۹۷۰ معرفی شد. بلوم فیلترها آرایه هستند. مسائلی که بلوم فیلتر کمک به حل آن‌ها می‌کند مسائل **Membership** هستند. مسئله‌ی **Membership** بررسی می‌کند یک داده عضو آن داده ساختار است یا خیر (**Membership**)؟ پاسخ بلوم فیلتر به صورت بله یا خیر نیست. در واقع زمانی که پاسخ بلوم فیلتر «خیر» باشد به معنی است که آن داده قطعاً عضو داده ساختار نیست. اما جواب مثبت بلوم فیلتر به این صورت تفسیر می‌شود که احتمالاً آن داده ساختار درون بلوم فیلتر قرار می‌گیرد.

جنس این نوع داده ساختار یک آرایه از نوع بیت است. بلوم فیلتر یکسری پارامتر دارد که تنظیم پارامترهای بلوم فیلتر می‌تواند در نتیجه‌ی نهایی موثر باشد. یکی از این پارامترها سایز آرایه‌ی بیتی (m)، تعداد درجه‌های احتمالی (**insert**) و پارامتر دیگر **False positive** یا احتمال قابل قبول در پذیرش جواب مثبت است. در واقع می‌دانیم بلوم فیلتر هر گاه جواب مثبت می‌دهد این جواب مثبت می‌تواند صحیح یا غلط باشد. **False positive** جواب مثبتی است که غلط است و برای آن یک آستانه در نظر گرفته می‌شود که بیشتر از آن آستانه قابل قبول نیست.

اساس اکثر داده ساختارهای احتمالاتی هشینگ است. عدم قطعیت از هشینگ می‌آید در واقع ثبت وجود یا عدم وجود داده باعث اضافه شدن این مسئله (عدم قطعیت) می‌شود. با این ورودی‌ها در مورد بلوم فیلتر تعیین می‌کنیم که می‌توانیم چند تابع هش داریم. تعداد عناصری که قرار است در بلوم فیلتر درج شوند به K تابع درهم سازی داده می‌شوند. همچنین نمی‌خواهیم داده را بازیابی کنیم و بیشتر می‌خواهیم آماری از آن داده را بدست آوریم. مثلاً بودن یا نبودن عدد ۲ مورد بررسی قرار می‌گیرد. مثلاً افرادی در حال ثبت نام هستند. می‌خواستیم ببینیم چند خانم در حال ثبت نام هستند. پس دنبال این نیستیم که خود اطلاعات را بازیابی کنیم.

یک آرایه‌ی m بیتی و k تابع درهم سازی در نظر گرفته می‌شود به طوریکه که کلیدها توسط توابع درهم سازی به خانه‌های آرایه نگاشت می‌شوند. در واقع کلید به عددی از ۰ تا $m-1$ نگاشت می‌شود. زمانی که قرار است



داده ساختار پیشرفته

نگاشتی انجام شود یک عدد به k تابع درهم سازی داده می شود و این k تابع k خانه از آرایه را آدرس دهی می کنند. با مشخص شدن عناصر آرایه توسط توابع هش، این خانه ها باید یک شوند.

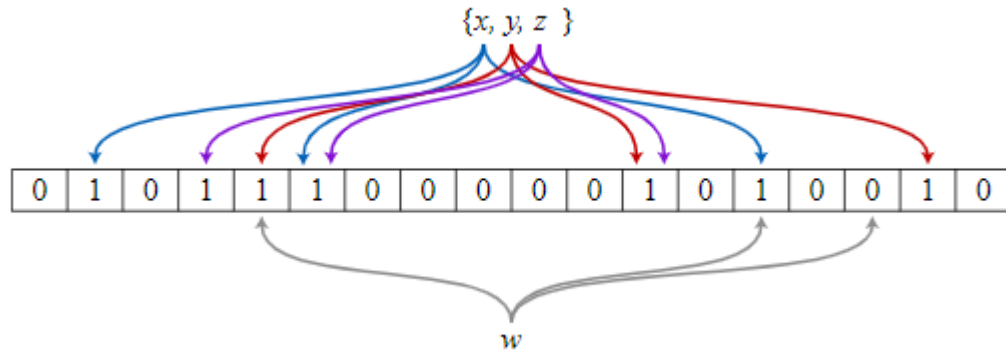
نحوه ی درج در بلوم فیلتر: اگر قلم داده ی جدیدی وارد شود مطابق روال قبل ابتدا k خانه ی آرایه توسط توابع هش مشخص می شوند و سپس خانه هایی که یک هستند بدون تغییر باقی می مانند و خانه هایی که صفر هستند به یک تبدیل می شوند. به اینصورت اثری از قلم بر روی داده می گذاریم. سعی می شود توابع درهم سازی از هم مستقل باشند. به اینصورت که توابع درهم سازی داده ها را در خانه های متفاوت از هم نگاشت کنند.

هزینه ی عملی درج برابر با تعداد توابع درهم سازی است که برابر با k است. هزینه ی هر تابع درهم سازی یک واحد است. بنابراین به ازای در یک قلم داده هزینه ی درج از $O(k)$ است. وقتی n قلم داده درج می شود هزینه ی درج $k.O(n)$ است. هزینه ی درج در این نوع داده ساختار زمانی که مقدار داده ها بالا باشد بالا می رود. همچنین یکی دیگر از مزیت های آن ماهیت داده ی ذخیره شده است. یک رکورد بزرگ از اطلاعات در نظر گرفته می شود (شامل عکس، اطلاعات و ...). به ازای این رکورد فقط یک داده ی یک بیتی ذخیره می شود. در واقع حجم ذخیره سازی پایین می آید.

نحوه ی کوئری گرفتن: مزیت این نوع داده ساختار نوع کوئری است. نوع کوئری آن بودن یا نبودن یک داده را مشخص می کند. حال یکسری درج انجام دادیم و می خواهیم کوئری انجام بدهیم. هنگام کوئری گرفتن توابع درهم سازی دو جا را نشان بدهند. اگر هر دو محل یک باشد امکان دارد داده در این داده ساختار وجود دارد. اما اگر داده در این داده ساختار درج نشده باشد امکان دارد یک یا هر دو مکان صفر باشد. با تنظیم پارامترها می توان گفت با چه احتمالی پاسخ درست پذیرفته می شود.

با احتمال یک درصد خطا به ازای هر المان باید 9.6 بیت مارک شوند (تعداد تابع درهم سازی باید 9.6 باشد).
عملیات حذف: در بلوم فیلتر (Bloom filter) پایه عمل حذف نداریم.

در مثال زیر سه آیت x, y, z ذخیره شده است و $m=18$ است. سه تابع هش داریم. ابتدا x ، سپس y و بعد از آن z درج می شود. زیرا هر آیت سه جا ذخیره شده است. نتیجه ی ذخیره به صورت شکل زیر است.



زمانیکه درج

انجام شد می‌خواهیم از رشته‌ای مانند w کوئری بگیریم. این رشته به سه تابع هش داده می‌شود و این توابع یکسری خانه را نمایش می‌دهند. اگر در یکی از خانه‌ها فقط یک صفر وجود داشته باشد به طور قطعی می‌گوییم که این داده در آن داده ساختار درج نشده است. اما با توجه به اینکه هر سه خانه یک است احتمال می‌دهیم این داده درون داده ساختار درج شده باشد.

نکات:

- False positive ممکن است اما False negative ممکن نیست.
- هزینه کوئری $O(k)$ است. در واقع عملاً از خواص هشینگ استفاده می‌کند. در اینجا برخورد وجود ندارد.
- در اینجا حذف وجود ندارد.

در بیگ‌دیتا بسیار مناسب است و سرعت آن بسیار بالا است. یکسری توابع درهم‌سازی هستند که برای این کار بسیار مناسب است و هزینه‌ی استفاده از آن‌ها پایین است که در ادامه گفته می‌شود.

فرض شود دو لاین جریان داده وجود دارد. خانم‌هایی که از کانال‌های یک و دو ثبت شدند Membership آن‌ها به طور مجزا در بلوم فیلترهای مجزا ثبت شده‌اند. در اینجا اجتماع و در برخی موارد اشتراک جواب‌ها می‌توانند ما را در حل برخی مسائل یاری کند و به پاسخ صحیح برسانند. در واقع ترکیب جواب‌ها می‌تواند نتیجه بدهد.

رابطه‌ی زیر تابع درهم‌سازی را تخمین می‌زند.

.....

سایز آرایه از رابطه‌ی زیر تخمین زده می‌شود.

.....



داده ساختار پیشرفته

P میزان آستانه‌ای است که درصد صحیح بودن False positive را به ما نشان می‌دهد. دو دسته از توابع هش معروف ممرهش و جیم‌کینگ است.

کاربردهای معروف Bloom filter عبارتند از:

- Google BigTable: از داده ساختارهایی است که گوگل برای جستجوی سریع استفاده می‌کند.
- Apache HBase
- Apache Cassandra
- Google chorom: از بلوم فیلتر برای شناسایی آدرس‌های بدخواه استفاده می‌شود.

بلوم فیلتر برای کاهش دسترسی‌ها به دیسک و جستجو بین اقلام داده‌ای متفاوت مورد استفاده قرار می‌گیرد. برای مشخص کردن سطر و ستون‌هایی که مشخص نیستند (Missing information)، از بلوم فیلتر استفاده می‌شود. برای شناسایی آدرس‌های بدخواه استفاده می‌شود. گوگل کروم برای تشخیص آدرس‌های بدخواه از بلوم فیلتر استفاده می‌کرد اما به داده ساختار Prefix set تبدیل شد.

وقتی آرایه‌ی بلوم فیلتر مناسب است که همه‌ی آرایه بتواند در کش جا بگیرد. در واقع آرایه بیش از حد بزرگ نشود. علت آن این است که اگر یک بلوم به عنوان مثال ۵ برابر کش باشد لازم است آرایه‌ی بلوم فیلتر به پنج بخش تقسیم شود و این پنج بخش به طور مجزا درون کش قرار گیرند و هر بار عملیات جستجو به طور مجزا بررسی شود. در واقع زمان اجرای عملیات جستجو حداقل پنج برابر می‌شود. البته تقسیم شدن بلوم فیلتر و ایجاد مشکل یک‌سری راه‌حل از جمله بافرینگ نیز دارد.

برخی از انواع بلوم فیلتر عبارتند از:

- ۱- بلوم فیلتر تضعیف شده: برای حل مسائل کوتاه‌ترین مسیر در گراف‌های بزرگ استفاده می‌شود.
- ۲- بلوم فیلتر Spectral: برای حل مسائل کوثری فرکانس (تکرار اطلاعات) استفاده می‌شود.
- ۳- بلوم فیلتر Counting: شمارش اطلاعات بالا (حل مسئله‌ی کوثری فرکانس)
- ۴- بلوم فیلتر Cascading: یک بلوم فیلتر در دل بلوم فیلتر دیگر قرار می‌گیرد.
- ۵- بلوم فیلتر Scalable

در پایتون داده ساختاری به عنوان بلوم فیلتری تعریف شده است. (Pyblom)



جلسه‌ی هجدهم

در جلسه‌ی قبل داده‌ساختارهای احتمالاتی معرفی شدند و گفته شد این داده‌ساختارها برای کاربردهایی که داده‌های آن‌ها از نوع streaming و بیگ‌دیتا است مناسب هستند. بنابراین وقتی صحبت از استریم داده‌ها می‌شود اولاً عدم قطعیت داریم یعنی ممکن است پاسخ کوئری‌های ما با توجه به جریان داده‌ای که می‌آید متفاوت باشد همچنین از لحاظ سرعت و فضایی که برای پاسخ به کوئری‌ها نیاز است محدودیت‌ها داریم.

برخی از مسائلی که داده‌ساختارهای احتمالی حل می‌کنند عبارتند از:

مسئله‌ی membership: از داده‌ساختار بلوم فیلتر برای حل مسئله‌ی Membership استفاده می‌شود. در این مسئله استریمی از داده‌ها بررسی می‌شود و عضویت یک داده‌ی خاصی مورد بررسی قرار می‌گیرد. اگر پاسخ ارائه شده منفی باشد با قطعیت می‌پذیریم که این داده از طریق جریان داده وارد داده‌ساختار نشده است اما اگر جواب مثبت باشد با یک احتمال وجود داده‌ی مورد جستجو در استریم داده‌های وارد شده به داده‌ساختار پذیرفته می‌شود.

مسئله‌ی Frequency: این مسئله تکرار یک آیتم را در یک داده‌ساختار تخمین می‌زند. قدمت این داده‌ساختار به سال ۲۰۰۳ برمی‌گردد. تفسیر نام Count-min Sketch به معنی خلاصه‌ی فشرده از حجم عظیم داده است که این خلاصه در یک ماتریس ذخیره می‌شود. این داده‌ساختار یک شمارش و یک عمل کمینه کردن انجام می‌دهد. عملکرد آن شبیه بلوم فیلتر است به این معنی که بلوم فیلتر مجموعه‌ای از آرایه‌ها بیتی است ولی این داده‌ساختار مجموعه‌ای از آرایه‌ها است که d سطر و w ستون دارد به طوریکه d تابع درهم‌سازی و w سائز آرایه‌ی درهم‌سازی است و این مجموعه کمک می‌کند تا شمارشی انجام شود.

عملیات درج در Count-min Sketch: کاری که در اینجا انجام می‌شود شبیه بلوم فیلتر است. یعنی تابع درهم‌سازی یک خانه را به ما نشان می‌دهد و یک واحد به مقدار خانه اضافه می‌شود. توجه شود که درون این خانه‌ها بیت وجود ندارد بلکه اعداد درون این خانه‌ها وجود دارد. در حالیکه در بلوم فیلتر یک بیت وجود دارد که این بیت می‌تواند صفر یا یک باشد. به طور کلی d تابع هش d خانه را به ما نشان می‌دهند و به این d خانه به طور مجزا یک واحد اضافه می‌شود.

عملیات جستجو در Count-min Sketch: اگر بخواهیم کوئری از این داده‌ساختار بگیریم از چند تابع درهم‌سازی استفاده می‌کنیم که هر کدام یک خانه را به ما نشان می‌دهند. از آن‌جا که قبلاً داده‌ها به صورت استریم وارد داده‌ساختار شدند برخی از خانه‌های آرایه غیر صفر هستند. خانه‌هایی که تابع درهم‌ساز برای یک کلید خاص نشان می‌دهد بازیابی می‌شوند مقدار کمینه‌ی آن‌ها مورد بررسی قرار می‌گیرد. عددی که نمایش داده می‌شود حداقل تکراری است که نمایش داده می‌شود. یعنی ممکن است مقدار تکرار اصلی بیشتر باشد.

نکات عبارتند از:



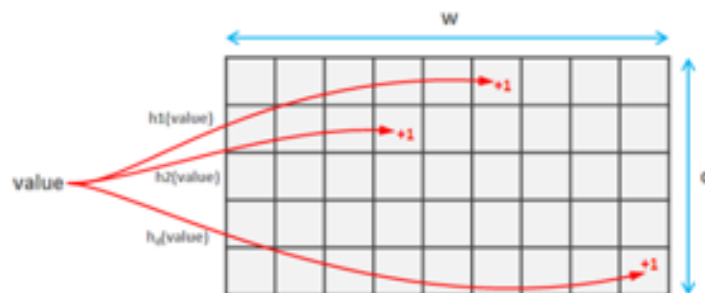
داده ساختار پیشرفته

- هر کدام از خانه‌های ماتریس یک شمارنده است.
- هر کدام از ردیف‌ها متناظر با تابع هش هستند.
- اگر برای یک عنصر خاص، توابع هش ما را به خانه‌های a, b, c, d ببرند فرکانس عنصر خاص برابر با $\min(a, b, c, d)$ است.
- یک خطای کمی این روش دارد که ما آن را می‌پذیریم. این خطا بسته به پارامترهای تعداد توابع (d) درهم‌سازی سایز آرایه‌ی، W و تعداد درج‌ها (n) است. خطای تخمین بار یک حد بالایی دارد که با احتمالات قابل اثبات است. حد بالای خطا عبارت است از:

$$[\text{error} \leq 2 * n/w]$$

d که تعداد تابع درهم‌سازی است میزان خطا را تنظیم می‌کند. اگر مقدار خطا دور از مقدار احتمالی باشد آن را تنظیم می‌کند.

احتمال اینکه تخمین بیشتر از خطا باشد $d^{1-(1/2)}$ است. زمانی که می‌خواهیم خطای ما کمینه باشد باید تابع ما به صورت مورب باشد. به این معنی که داده‌ها زیر هم نباشند. در واقع داده‌ها بهتر است پخش باشند. زیرا ترتیبی برای توابع درهم‌سازی وجود ندارد. به دلیل نبودن ترتیب احتمال **overwrite** افزایش می‌یابد و **overwrite** باعث تخمین اشتباه می‌شود (زمانیکه داده‌ها پراکنده نباشند).



در اینجا یکسری پارامتر وجود دارد که با توجه به آن‌ها می‌توان تعداد تابع درهم‌سازی و آرایه را به مناسب انتخاب کرد. این پارامترها با اپسیلن و لاندا نمایش داده می‌شوند. اپسیلن میزان نشان‌دهنده‌ی دقت یک موضوع یا تخمین خطای آن موضوع است. به طور مثال میزان خطای فرکانس به صورت $4.5 \pm \epsilon$ تخمین زده می‌شود. اگر خطا به اندازه‌ی اپسیلن بالا یا پایین شود مشکلی به وجود نمی‌آید. همچنین لاندا قطعیت رسیدن به یک دقت خاص را نمایش می‌دهد. تعداد توابع درهم‌سازی از رابطه‌ی $d = \lceil \ln(1/\delta) \rceil$ نشان داده می‌شود. همچنین اندازه‌ی آرایه هش از رابطه‌ی $w = \lceil e / \epsilon \rceil$ محاسبه می‌شود. e عدد اولی‌ر است که مقدار آن برابر با 2.7 است. یکسری از موارد مانند جریان داده و ... دست ما نیست. همچنین می‌توان با اجتماع و اشتراک داده‌ساختارها را تلفیق کرد. در واقع هر منبع داده یک استریم باشد و هر کدام از آن‌ها به یک جدول



داده ساختار پیشرفته

برود و در نهایت با استفاده از عملیاتی مانند اجتماع، اشتراک و جمع می‌توان جداول را ترکیب و عملیات مینیمم‌گیری را بر روی آن‌ها اعمال کرد.

اگر داده از منابع مختلف تولید شوند و بتوان هر کدام از آن‌ها را مستقل در نظر گرفته می‌شوند. هر کدام از آن‌ها ... در نظر گرفته می‌شود بعداً عملیات ترکیب بر روی همه‌ی عناصر مستقل اجرا شود.

ترکیباتی از توابع درهم‌سازی بسیار مناسب است. یکی از ترکیب‌های معروف توابع درهم‌سازی مرمرهش و Fowlen-N.... است. انتخاب توابع درهم‌سازی نیز بسیار مهم است.

تخمینی که (Overestimate) می‌زند بیشتر از تعداد تکرار واقعی است. ولی تخمین کمتر از مقدار گفته شده نیست. این داده ساختار شکل بهبودیافته‌ای از بلوم‌فیلتر است. کاربردهای واقعی آن در سوئیچ‌های شبکه AT&T است که بتواند ترافیک شبکه را اندازه بگیرد. ویژگی این داده ساختار و بلوم فیلتر این است که حافظه‌ی زیادی مصرف نمی‌کنند. اگر بخواهیم به یک احتمال خطای $k=5$ باشد. یعنی ۵ تابع درهم‌سازی باید $d=[\ln 1/\delta]$ باشد. اگر بخواهیم خطای ما کمتر 1% باشد باید $k=5$ باشد. یعنی ۵ تابع درهم‌سازی کافی است. در Map-Reduce گوگل نیز مورد استفاده قرار می‌گیرد.

اگر از این داده ساختارها استفاده شود در زمان کوتاه نتیجه‌ی بسیار عالی حاصل می‌شود. هر چه تنوع داده‌ها بالاتر باشد این موضوع بهتر جواب می‌دهد. این داده ساختارها در امنیت استفاده می‌شود.

Cardinality: داده ساختار دیگری که استفاده می‌شود Cardinality است. در کاردینالیتی می‌خواهیم تنوع داده‌ها را بسنجیم. به عنوان مثال مسئله‌ی کاردینالیتی قابل بحث نیست ولی مسئله‌ی فرکانس قابل بحث است. مسئله‌ی کاردینالیتی تعیین تعداد المان‌های مجزا در یک مجموعه مجزا از المان‌های جدا از هم است به طوریکه در اینجا المان‌های تکراری می‌توانند وجود داشته باشند. این داده ساختار در سال ۲۰۰۷ ابداع شده است. مثلاً در مجموعه‌ای 1000 علی و 500 فاطمه و 200 محمد داریم. به طوریکه 1000 ، تعداد فرکانس‌های المان علی و مقدار 3 کاردینالیتی این داده‌ها است. این موضوع در سال ۲۰۰۷ ابداع شده است.

برای شمارش کاردینالیتی راه‌حل‌های دقیق و غیردقیق وجود دارد که یکی از آن‌ها لینیر کانتینگ (Leaner counting) است که در آن داده‌ها استیریم نیستند. این روش دقیق نیست (تخمینی است). در این روش از یک شمارنده‌ی خطی برای تخمین استفاده می‌شود که یک Bit map است و شبیه جداول درهم‌سازی می‌باشد. سائز آن m است که ابتدا m خانه صفر می‌شود. لینیر کانتیر به اینصورت انجام می‌شود که هر المان به یک هش فانکشن داده می‌شود و این هش فانکشن ما را به یک خانه‌ی آرایه می‌برد در صورتی که آن خانه‌ی آرایه یک باشد یک باقی می‌ماند و در صورتی که صفر باشد تبدیل به یک می‌شود. در اینجا ممکن است برخورد به وجود بیاید که با ایجاد برخورد آن خانه‌ی آرایه یک باقی می‌ماند. در این روش هر چه اندازه‌ی آرایه بزرگتر



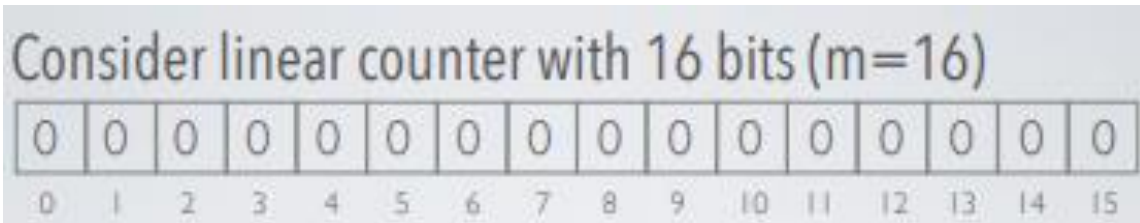
داده ساختار پیشرفته

باشد بهتر است. بعد از آنکه داده‌ها در جدول درهم‌سازی ثبت شدند نسبتی مانند V محاسبه می‌شود. این مقدار از تقسیم خانه‌های خالی به کل بیت‌ها بدست می‌آید. سپس کاردینالیتی از رابطه‌ی زیر محاسبه می‌شود:

$$n \approx -m \ln V$$

چون مقدار V از یک کوچکتر است یک منفی قبل از V می‌گذارند. این با توجه به اینکه یک روش ساده است.

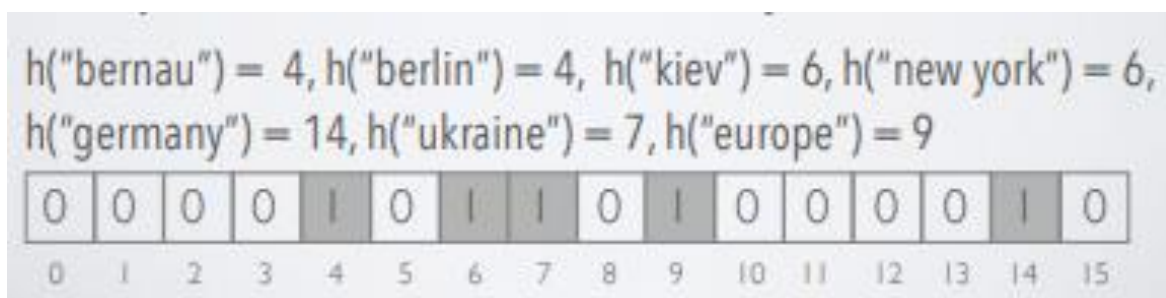
مثال: یک آرایه‌ی با سایز $m=16$ در نظر گرفته می‌شود. از تابع درهم‌سازی مرمهش ۳ استفاده می‌شود. حال اگر خروجی این تابع درهم‌سازی، در محدوده‌ای که مورد انتظار نباشد خروجی آن در نظر گرفته می‌شود و باقیمانده‌ی آن به ۱۰ محاسبه می‌شود. مقدار این باقیمانده‌ای خانه‌ای را نشان می‌دهد که کلید در آن تاثیر می‌گذارد. در این مثال اسامی یکسری از شهرها به عنوان ورود وارد داده ساختار لینیر کانتر می‌شود. می‌خواهیم تنوع اسامی را مورد بررسی قرار دهیم.



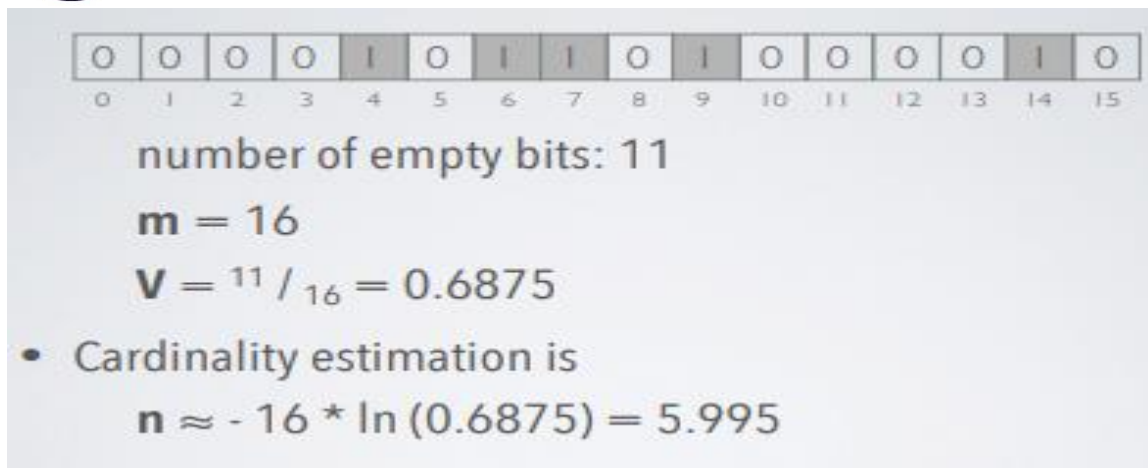
مجموعه شهرهای مورد بررسی به صورت زیر است:

Set of 10 elements: "bernau", "bernau", "bernau", "berlin", "kiev", "kiev", "new york", "germany", "ukraine", "europe" (NOTE: the real cardinality $n = 7$)

اگر نداشت با استفاده از تابع مرمهش ۳ انجام شود نتیجه‌ی هشینگ به اینصورت است:

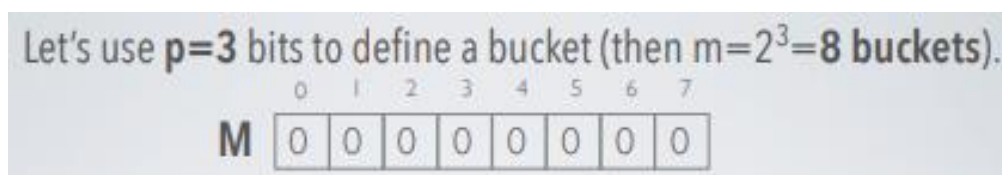


نتیجه‌ی تغییر جدول هش به صورت جدول بالا است. حال الگوریتم leaner counter به اینصورت عمل می‌کند که تعداد بیت‌های خالی شمرده می‌شود (یعنی 11 بیت) و سپس تعداد کل خانه‌ها 16 است که مقدار V برابر با $11/16$ است که در نهایت برابر با $n = m \ln 0.6875$ است ($n=5.995$)



تخمین مونت کارلو نیز به اینصورت عمل می کند که تخمینی از یک جواب زده می شود و سپس از روی تخمین به جواب اصلی نزدیک می شویم. اگر داده ها به صورت استیریم باشند و انتظار دقت بالاتری داشته باشیم از داده ساختار HyperLogLog استفاده می شود. مثال زیر برای داده ساختار HyperLogLog بررسی می شود:

مثال HyperLogLog: فرض شود داده ها تکراری نباشند یعنی برخورد وجود ندارد. می بینیم چقدر داده ها خوب عمل می کنند (برای روش قبل). این روش بر مبنای یک تابع درهم سازی است که یکسری مقادیر هش تولید می کند و این مقادیر هش یکنواخت هستند. مقادیر بیت و طول آرایه درهم سازی در اینجا باید تعیین شود. نحوه ی تعیین تابع درهم سازی و طول آرایه با توجه به نرخ خطا تعیین می شود. فرض کنید هر باکت با سه بیت نمایش داده می شود. در اینجا تابع درهم سازی به گونه ای انتخاب می شود که خروجی آن مجموعه ای از بیت ها صفر و یک است. این خروجی ها می توانند به صورت اندیس یک آرایه و یا مجموعه ای از بیت ها در نظر گرفته شوند. فرض می شود از سه بیت برای تعریف هر خانه ی آرایه استفاده می شود. در واقع طول آرایه ۸ بیت می شود. فرض می شود با سه بیت می توان عناصر صفر تا هفت را نمایش داد.



خروجی تابع درهم سازی به دو بخش تقسیم می شود. سه بیت اصلی که برای آدرس دهی نیاز است یک بخش و بقیه ی آن بخش دوم را تشکیل می دهد. در بخش دوم مقدار صفرها شمارش می شوند و به اول عدد غیر از صفر که برخورد می شود میزان صفرهای شمارش شده ی قبل از آن رنگ را نشان می دهند.



Let's use $p=3$ bits to define a bucket (then $m=2^3=8$ buckets).

	0	1	2	3	4	5	6	7
M	0	0	0	0	0	0	0	0

Consider $L=8$ bits hash function h

Index elements "berlin" and "ferret":

$h(\text{"berlin"}) = 0110111$

$h(\text{"ferret"}) = 1100011$

Define buckets and calculate values to store:

(use first $p=3$ bits for buckets and least $L-p=5$ bits for ranks)

- $\text{bucket}(\text{"berlin"}) = 011 = 3$ $\text{value}(\text{"berlin"}) = \text{rank}(0111) = 2$
- $\text{bucket}(\text{"ferret"}) = 110 = 6$ $\text{value}(\text{"ferret"}) = \text{rank}(0011) = 3$

	0	1	2	3	4	5	6	7
M	0	0	0	2	0	0	3	0

مثال بالا را در نظر بگیرید. برلین به صورت $h(\text{"berlin"}) = 0110111$ رمزگذاری شده است. سه بیت اول این رشته یعنی 011 محل ذخیره سازی و 0111 مقدار ذخیره سازی را نشان می دهند. برای تعیین مقدار ذخیره سازی از مفهوم leading zero استفاده می شود. برای محاسبه ی leading zero تعداد صفرهای بخش دوم یعنی 0111 شمارش می شود تا به یک برسیم. این مقدار در اینجا 2 است که این دو در خانه ی 011 یعنی خانه ی شماره ی 3 ذخیره می شود.

در اینجا زمانیکه برخورد رخ می دهد مقدار فعلی و مقدار سابق مقایسه می شود و هر کدام بزرگتر بود در آن خانه درج می شود. در مثال زیر برخورد در خانه ی ششم رخ می دهد و به دلیل اینکه مقدار 4 بزرگتر از 3 است این مقدار در خانه ی ششم قرار می گیرد:



Index element "kharkov":

- $h(\text{"kharkov"}) = 1100001$
- $\text{bucket}(\text{"kharkov"}) = 110 = 6$ $\text{value}(\text{"kharkov"}) = \text{rank}(0001) = 4$
- $M[6] = \max(M[6], 4) = \max(3, 4) = 4$

	0	1	2	3	4	5	6	7
M	0	0	0	2	0	0	4	0

Estimate the cardinality be the HLL formula ($C \approx 0.66$):

$$DV_{HLL} \approx 0.66 * 8^2 / (2^{-2} + 2^{-4}) = 0.66 * 204.8 \approx 135 \neq 3$$

NOTE: For small cardinalities HLL has a strong bias!!!

در اینجا سه قلم داده ثبت شدند. حال می‌خواهیم کاردینالیتی را حل کنیم. بر حسب M یک عدد ثابت محاسبه می‌شود. عدد ثابت محاسبه شده 0.66 است. در نهایت مقدار آن به این صورت است:

- Estimate the cardinality be the HLL formula ($C \approx 0.66$):

$$DV_{HLL} \approx 0.66 * 8^2 / (2^{-2} + 2^{-4}) = 0.66 * 204.8 \approx 135 \neq 3$$

NOTE: For small cardinalities HLL has a strong bias!!!

مقدار تخمین در این روش بسیار بد است. تخمین برابر با ۱۳۵ است در حالیکه کاردینالیتی ۳ است. زیرا داده‌ها استریم نیستند. این روش برای داده‌های استریم بسیار مناسب است. اگر داده‌های به حد کافی بزرگ باشد. مقدار خطا به صورت زیر است:

- The standard error can be estimated as:

$$\sigma = \frac{1.04}{\sqrt{2^p}} \quad \text{so, if we use 16 bits (p=16) for bucket indices, we receive the standard error in 0.40625\%}$$

در رابطه‌ی بالا p تعداد بیت‌هایی است که باید محاسبه شود. اگر داده‌های به حد کافی بزرگ باشد به طوریکه کاردینالیتی بیشتر از ۱۰۹ باشد این دقت با دقت نسبی ۲ درصد و مصرف 1K بایت ذخیره می‌شود. اگر یک روز بیگ‌دیتایی داشته باشیم و بخواهیم در زمان کوتاه کوئری بزنیم این داده ساختار مناسب است.